射频神经场 FPGA 加速系统

摘要

复杂室内环境无线空间谱预测能够刻画多径传播和遮挡条件下的信号空域分布，为室内无线网络的波束赋形、动态频谱共享等应用提供支撑，一直以来都是无线通信领域的研究热点。近年来，射频神经辐射场 NeRF² 方法应用于无线空间谱预测取得了较好的效果，但该方法使用了大量重复计算，当部署到性能相对较低的边缘设备上时，推理速度难以满足要求。同时，边缘设备相对于电脑还存在存储资源受限、定点推理精度下降和推理速度下降等问题。这些问题都亟待解决。

针对上述问题，本文从计算复用、存储压缩、量化精度和推理加速四个方面进行优化。首先，针对 NeRF² 推理过程中的重复计算问题，通过结构约束和受约束微调解耦衰减分支与信号分支，预计算并复用固定接收端相关的衰减中间量，减少重复计算。其次，针对边缘设备存储资源受限的问题，采用稀疏权重存储方法，删除整数模型中的分组全零权重列，降低存储开销。在此基础上，使用训练集定标和硬件精确量化感知训练，使训练前向过程匹配目标硬件的整数运算与射线渲染规则，以减少定点推理时引起的精度损失。最后，针对边缘设备的推理速度下降的问题，使用并行处理，设计双 INT8 乘法 DSP 打包和双射线并行引擎，并在可编程逻辑中实现定点射线渲染，以提高推理速度。

在此基础上，本文基于 PYNQ-Z2 平台构建 95 MHz 软硬件协同推理系统，通过运行时加载模型参数实现多场景切换，根据发射端三维坐标生成 90×360 的完整空间谱，每条方向射线包含 64 个采样点。实验覆盖 s23、Meetingroom、Office1 和 Office2 四个场景，共包含 1725 幅测试空间谱，其中 s23 采用实测数据，其余场景采用仿真数据。在模型装载和固定接收端缓存准备完成后，完整测试集三轮推理的单幅平均时间为 1.101～1.140 s，较 PC 浮点参考缩短 12.62%～18.08%。FPGA 结果相对真值的平均结构相似度较 PC 浮点参考下降 0.0075～0.0130。各场景固定测试位置连续推理 50 次的中位时延为 0.961～1.004 s，整板输入功率中位数为 1.974～2.043 W。结果表明，该系统能够在资源受限平台上完成多场景无线空间谱预测，可为空间传播分析和信道知识地图构建提供支持。

目录

摘要.....1
第一部分 绪论.....4
1.1 研究背景与问题提出.....4
1.2 系统功能与研究目标.....4
1.3 应用场景与输出含义.....5
1.4 关键技术与系统特点.....5
1.5 性能指标概览.....6
1.6 主要研究内容.....7
1.7 研究流程.....7
第二部分 系统设计与实现.....7
2.1 系统总体架构.....7
2.1.1 系统功能层次与数据关系.....8
2.1.2 处理系统与可编程逻辑的数据通路.....9
2.1.3 硬件工程模块与系统架构的对应关系.....10
2.1.4 完整空间谱推理流程.....10
2.1.5 场景切换与显示机制.....11
2.2 硬件系统设计.....12
2.2.1 硬件架构.....12
2.2.2 样机结构与连接.....14
2.2.3 硬件模块设计.....14
2.3 软件系统设计.....24
2.3.1 软件总体架构.....24
2.3.2 软件模块与数据流程.....27
2.4 设计原理与软硬件任务划分.....51
2.4.1 空间谱生成原理.....51
2.4.2 计算任务与数据依赖.....51
2.4.3 接口协议与状态管理.....52
2.5 模型和硬件优化分析.....52
2.5.1 优化思路与比较方法.....52
2.5.2 量化感知训练的精度影响.....53
2.5.3 稀疏存储的权重压缩效果.....53
2.5.4 运行时可配置设计的资源开销.....53
2.5.5 计算复用与流水调度.....54
第三部分 实验结果与分析.....54
3.1 实验系统与验证内容.....54
3.1.1 系统实物与连接.....55
3.2 系统实现与功能验证.....56

3.2.1 样机组成 56
3.2.2 硬件实现 56
3.2.3 软件实现与功能验证 57
3.3 性能测试与结果分析 60
3.3.1 评价指标与测试对象 60
3.3.2 完整测试集预测精度 60
3.3.3 精度分布与空间谱对比 61
3.3.4 固定位置连续推理时延 62
3.3.5 完整测试集平均推理时间 63
3.3.6 板端功率测量方法 63
3.3.7 活动窗口功率与积分能耗 64
3.3.8 不同测量范围下的单图能耗 65
3.3.9 功率采样与能耗估算 66
3.3.10 整数输出复核 67
第四部分 结论与展望 68
4.1 研究结论 68
4.2 后续研究方向 68
参考文献 68
附录 A 大模型协作记录与技能包提炼 70
A.1 大模型协作方式 70
A.2 典型协作案例 70
A.3 NeRF² 模型转换技能包 nerf2-checkpoint-fpga-export 72
A.4 PYNQ Overlay 验证技能包 pynq-overlay-validation 72
附录 B 实验复现方法 73
B.1 复现输入与工程文件 73
B.2 运行环境准备 73
B.3 输入校验与硬件重建 74
B.4 模型转换与完整测试集评估 74
B.5 RTL 验证与部署文件生成 74
B.6 板端部署与服务启动 75
B.7 应用启动与完整测试 75
B.8 功率测量与数据归档 75
B.9 复现结果判据 76

第一部分 绪论

1.1 研究背景与问题提出

复杂室内环境中的墙体、设备和人员会对无线信号产生反射、绕射和散射，使信号经多条路径到达接收端，并形成随空间位置变化的角域分布。获取不同发射位置对应的接收空间谱，有助于分析主要来波方向和多径传播特征。逐点测量需要在候选位置采集数据，射线追踪则依赖环境几何和材料参数。为提高已建模环境中的空间谱查询效率，本文研究固定接收端条件下由发射端位置预测角域空间谱的方法。

NeRF²利用神经网络表示无线传播场，并通过射线渲染生成接收结果[1]。信道知识地图将位置与信道信息关联，可为环境感知通信提供先验知识[2]。NeRF²所采用的连续场表示与射线采样方法源自 NeRF[3]，信道知识地图的构建和应用也得到进一步研究[4,5]。然而，将射线采样与神经网络推理部署到资源受限平台时，需要同时处理计算量、存储容量和定点误差。模型压缩与 FPGA 并行计算为解决上述问题提供了方法基础[6,7]。

基于上述研究，本文以 Zynq-7020 为目标器件，围绕模型结构适配、定点计算、稀疏数据组织和软硬件协同开展设计。系统在已建模场景中，根据发射端坐标预测固定接收端的空间谱，可为主要来波方向分析和接收方向候选筛选提供依据。图 1-1 和图 1-2 分别说明空间谱的物理含义与信道知识地图的应用背景。

表 1-1 已有方法与本作品的任务定位

方法	可提供的能力	需要的条件或工作量	本作品的取舍
逐点测量	取得实际位置的传播标签	需要采集设备及位置遍历	离线训练仍依赖测量或仿真数据。查询阶段复用模型
几何射线追踪	按环境模型计算传播路径	需几何、材料和传播参数	本系统直接读取训练所得参数，不在线重建几何环境
原始 NeRF² 浮点推理	学习并查询连续无线场	在 PC/GPU 执行浮点网络与射线累积	保留其 FP32 参考，另构建可在 PL 执行的整数路径
本作品整数部署	由 Tx 生成 90×360 完整方向谱	已训练场景、固定 Rx 及配套运行时资源	在当前器件完成约 1 s 级驱动推理，并支持场景参数切换

表 1-1 比较相关方法与本文的任务定位。针对固定接收端和可变化发射端的应用条件，系统设计需要兼顾模型预测能力、硬件资源约束与运行效率。下文依次介绍研究目标、系统实现和实验结果。

1.2 系统功能与研究目标

1. 系统支持 s23、Meetingroom、Office1 和 Office2 四个场景。用户可通过统一界面选择场景、提交查询并查看结果。
2. 在接收端位置与姿态固定的条件下，系统接受场景范围内的连续发射端三维坐标，并预测该位置对应的完整角域空间谱。

3. 输出空间谱包含 90×360 个方向采样值，可用于观察相对信号强度、主要来波方向和多径角域分布。
4. 系统支持在浏览器中显示、保存和重新读取空间谱，并可通过板卡 HDMI 接口输出结果。
5. 系统以 PYNQ-Z2 为边缘推理平台。在所测四个场景中，整板输入功率中位数约为 2 W。

1.3 应用场景与输出含义

本系统面向已建模的环境，在接收端 Rx 位置与姿态固定时，根据发射端 Tx 坐标生成 90×360 角域空间谱。空间谱描述各接收方向的相对信号强度，可用于主要来波方向判断、多径角域分布分析和接收波束候选筛选，例如预估智能工厂中移动设备在规划位置发射时，固定接收端的空间谱。

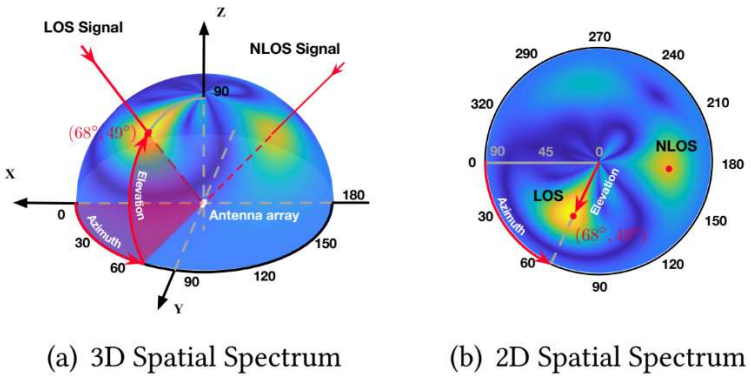


图 1-1 空间谱与来波方向示意（引自文献[1]，Fig. 5）

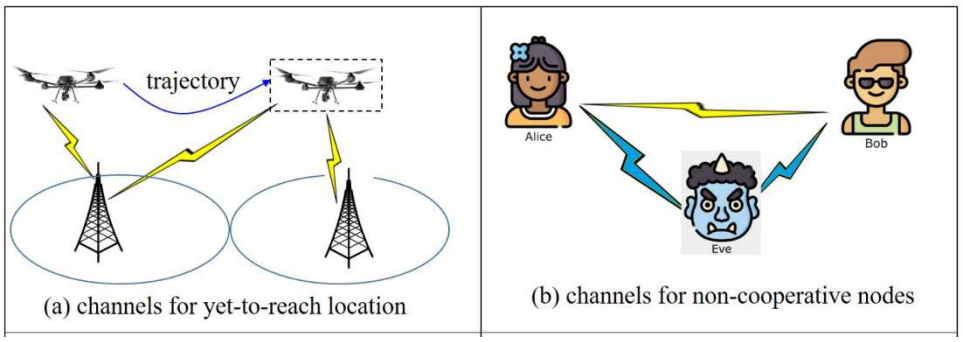


图 1-2 CKM 一般应用背景：未到达位置及非合作节点（引自文献[2]，Fig. 5(a)(b)）

将不同发射位置与对应的预测空间谱关联，可为空间相关的信道知识地图提供角域信息，进而支持已建模环境中的传播特征分析。在位置和几何条件适用的情况下，相关结果还可为接收方向筛选及干扰分析提供参考。

1.4 关键技术与系统特点

1. 运行时场景配置。四个场景共用一份 95 MHz 推理位流，切换场景时更新模型参数及配套运行时数据。
2. 硬件精确量化感知训练。利用训练集定标，并在训练前向中匹配 INT8 权重与激活、INT16 偏置、移位、饱和和射线渲染规则。
3. 分支解耦与中间量复用。通过结构约束及受约束微调分离衰减与信号计算，预计算固定接收端对应的衰减中间量，在线更新与发射位置和方向

相关的信号分支。

4. 稀疏存储与并行计算。删除冻结整数模型中整组全零列，使用稀疏索引和执行描述符组织数据，并采用双射线矩阵运算单元减少重复访存与无效计算。
5. 完整空间谱生成。系统对 90×360 条方向射线分别进行 64 点采样和渲染，通过定点复数累加与幅值计算生成空间谱。
6. 软硬件协同。PC 负责模型训练、参考推理、任务组织和可视化，处理系统中的 ARM 负责模型装载与 DMA 调度，可编程逻辑完成整数推理和射线渲染。

1.5 性能指标概览

表 1-2 作品主要性能指标

指标	结果	说明
支持场景	运行时可配置，已验证 4 个场景	s23、Meetingroom、Office1、Office2
完整测试集	1725 幅	1225 + 200 + 200 + 100
输出分辨率	90×360	每幅 32400 条方向射线（90*360）
每射线采样	64 点	每幅共 2073600 个采样点（90*360*64）
推理时钟频率	95 MHz	PL 推理时钟为 95 MHz
FPGA 平均时延	1.101-1.140 s	各场景完整测试集运行 3 轮，统计单幅平均推理时间
FPGA 中位时延	0.961～1.004 s	各场景选定固定测试位置，连续 50 次实测
FPGA P95 时延	0.983～1.046 s	同上述 50 次测试。P95 表示 95%的测量耗时不超过该场景对应值
平均 SSIM 差	0.00752～0.01302	PC 与 FPGA 分别预测的空间谱和真实谱图的 SSIM 之差的绝对值
FPGA 整板功率	1.974～2.043 W	FNB58 测量 5 V 板输入端
FPGA 单图能耗估算	1.900～2.051 J	各场景推理功率中位数×该场景 50 次推理的中位时延

1.6 主要研究内容

1. 根据固定接收端条件分析网络输入依赖，通过结构约束实现分支解耦，并复用衰减中间量。
2. 设计双 INT8 乘法打包和双射线计算结构，提高网络推理的并行度。
3. 采用分组全零列删除和稀疏索引组织权重，减少模型存储与无效运算。
4. 构建运行时可配置的推理硬件，使四个场景共享同一份位流。
5. 建立与目标整数算术和射线渲染规则一致的量化感知训练流程。
6. 将相位递推、复数累加和整数开方映射到可编程逻辑，生成完整空间谱。

1.7 研究流程

1. 整理模型参数、数据划分与配置，建立 PC 浮点基准。
2. 分析分支依赖，施加结构约束并微调，确定固定 Rx 缓存方案。
3. 约定硬件数值规则，采用训练集定标和硬件精确 QAT，导出稀疏参数、执行描述符及查找表。
4. 设计双射线整数计算、射线渲染与 AXI 数据通路，完成 RTL 仿真。
5. 在 95 MHz 时钟约束下完成综合与布局布线，并检查实现结果。
6. 集成 PS 调度、DMA、App 及 HDMI，完成四场景精度、时延和整板功率测试。

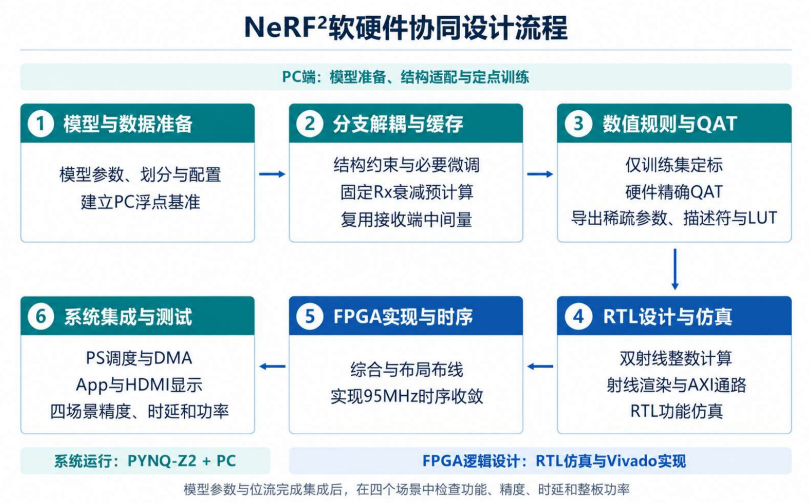


图 1-3 NeRF²软硬件协同设计流程

第二部分 系统设计与实现

2.1 系统总体架构

本系统在 PYNQ-Z2 平台上实现 NeRF²无线空间谱推理。对于已完成离线建模的室内场景，接收端 Rx 的位置与姿态保持固定，用户输入发射端 Tx 的三维坐标，系统生成 90×360 角域空间谱。谱中每个位置对应一个接收方向，数值表示该方向的相对信号强度。PYNQ-Z2 平台结构与接口依据参考手册[8]和原理图[9]，Zynq 处理系统与 AXI DMA 的配置分别参考文献[10]和[11]。

系统按功能分为模型准备层、PC 交互层、Zynq 处理系统层和可编程逻辑加速层。前两层运行在电脑上，后两层位于同一块 PYNQ-Z2 的 XC7Z020 芯片中。处理系统 PS 包含双核 ARM 处理器，运行 Linux 和 PYNQ 软件。可编程逻辑 PL

由 RTL 描述的专用数据通路构成。ARM 组织任务并控制硬件，PL 完成主要整数推理与射线渲染，两者通过共享 DDR 和 AXI 接口协同工作。

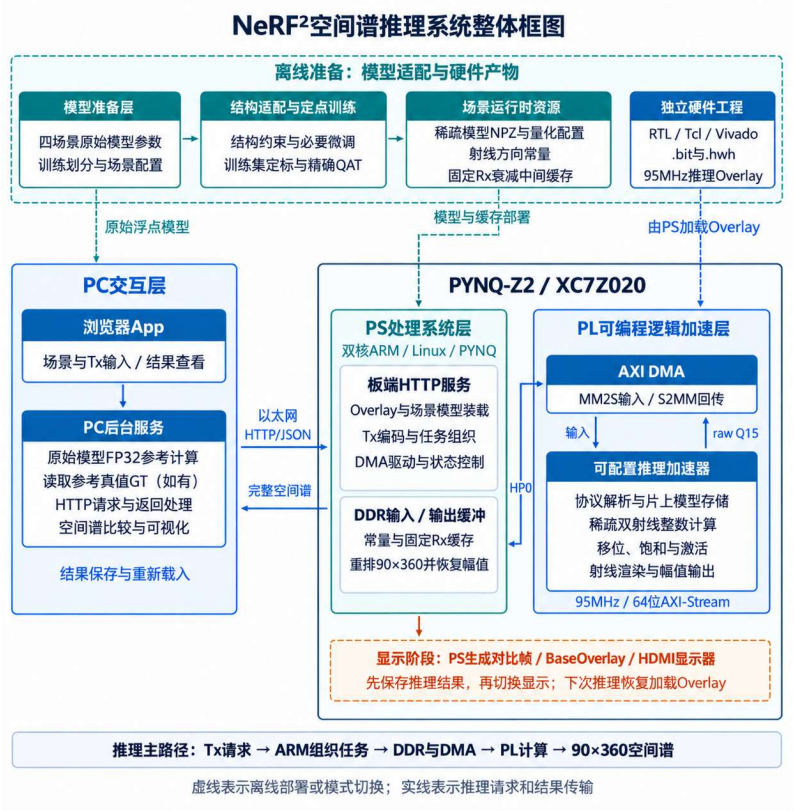


图 2-1 系统整体框图及离线准备与在线推理的关系

图 2-1 上部表示离线准备，虚线连接表示文件部署或模式切换。下部表示系统运行，实线连接表示请求、输入数据和结果的传输。模型准备与 RTL 实现共同确定硬件可执行的数值规则，浏览器 App 和板端服务在这些产物准备完成后组织实际查询。

2.1.1 系统功能层次与数据关系

（1） 模型准备层

模型准备层将 PC 训练得到的模型参数转换为加速器能够装载的数据。其输入包括 s23、Meetingroom、Office1 和 Office2 四个场景指定的最终模型参数、原始数据及训练划分、接收端配置、场景边界和射线采样设置。原始浮点模型保留为 PC 参考，部署模型则经过结构约束、必要的受约束微调、训练集定标和硬件精确量化感知训练。结构约束使衰减计算与 Tx 相关信号计算分离。QAT 使训练前向匹配目标硬件的整数算术、移位、饱和、激活和射线渲染规则。

该层输出运行时模型 NPZ、量化配置、射线方向常量和固定 Rx 衰减中间缓存。运行时模型包含稀疏权重、索引、偏置、执行描述符、移位量及渲染查找表。描述符说明计算单元下一步处理哪一层、哪一组数据及对应存储位置。衰减缓存保存当前模型和固定射线采样配置对应的中间结果，不保存某个 Tx 的最终预测图。因此，新 Tx 查询仍需执行信号计算和完整方向网格的射线渲染。

（2） PC 交互层

PC 交互层由浏览器 App 和本地后台服务组成。浏览器负责场景选择、测试

位置选择、合法连续 Tx 坐标输入及空间谱显示。后台服务负责读取当前场景配置、运行原始模型的 FP32 参考推理、向板端发送请求，并接收和保存板端结果。参考推理在电脑上执行，CUDA 可用时使用 GPU，也可使用 CPU。浏览器本身不执行 PyTorch 神经网络计算。

对于数据集中有参考真值 GT 的测试位置，PC 可组织 GT、PC 预测和 FPGA 预测的比较。对于没有对应 GT 的自由 Tx 坐标，界面显示 PC 与 FPGA 两种预测。GT 只用于结果比较，不是 PL 推理的数据来源，PC 参考结果也不参与加速器的空间谱数值计算。

（3） Zynq 处理系统层

PS 侧 ARM 运行板端 HTTP 服务和 PYNQ 驱动。它接收场景及 Tx 请求，检查坐标是否处于当前场景范围，装载推理 Overlay 和场景运行时模型，读取固定方向常量与衰减缓存，并组织 DMA 输入缓冲。Overlay 中的.bit 文件配置 PL 逻辑，配套.hwh 文件向 PYNQ 描述 IP、地址与接口，.hwh 本身不执行计算。Jupyter Notebook 用于启动和管理服务，不在每次查询时重新训练模型，模型提前就在电脑中训练完成。

（4） 可编程逻辑加速层

PL 内包含 AXI DMA、AXI 互连、AXI GPIO、复位模块和自定义推理加速器。加速器内部负责协议解析、片上模型存储、稀疏参数读取、双射线整数运算、定点激活及射线渲染。配置阶段可计算固定 Rx 相关中间量。正常缓存推理阶段复用这些中间量，计算当前 Tx 对应的信号分支，并按每条射线的 64 个采样点完成复数累加和幅值计算。

2.1.2 处理系统与可编程逻辑的数据通路

图 2-2 展示推理 Overlay 的 Vivado Block Design，并标注 PS7、GP0 控制通路、AXI GPIO、HP0 数据通路、AXI DMA 和自定义推理核心。各标注用于定位主要模块与接口，具体功能对应关系见表 2-1。

控制通路由 PS 的 M_AXI_GP0 发起，经 ic_gp0 连接 DMA 的 AXI4-Lite 控制寄存器和 AXI GPIO。ARM 通过 MMIO 配置传输地址和字节数，启动 DMA 并查询状态。GPIO 的 1 位输出用于推理核心软复位，32 位输入用于读取核心状态。模型参数和推理任务不通过逐个 GPIO 引脚传送。

数据通路以 DDR 缓冲区为起点和终点。AXI DMA 的 M_AXI_MM2S 和 M_AXI_S2MM 经 ic_hp 连接 PS 的 S_AXI_HP0，从 DDR 读取输入并将结果写回 DDR。DMA 与自定义核心之间使用 64 位 AXI4-Stream：MM2S 把内存数据变成输入流，S2MM 接收核心输出并写回内存。DMA 硬件 IP 位于 PL，ARM 运行的是控制该 IP 的驱动程序。

同一 AXI-Stream 接口承载模型装载与推理数据，软件通过协议头区分命令。系统时钟由 PS 的 FCLK_CLK0 提供，工作频率为 95 MHz，rst0 负责相关复位信号分发。图 2-2 中的蓝色虚线用于引出功能标注，硬件连接沿 Block Design 中的接口和信号线阅读。

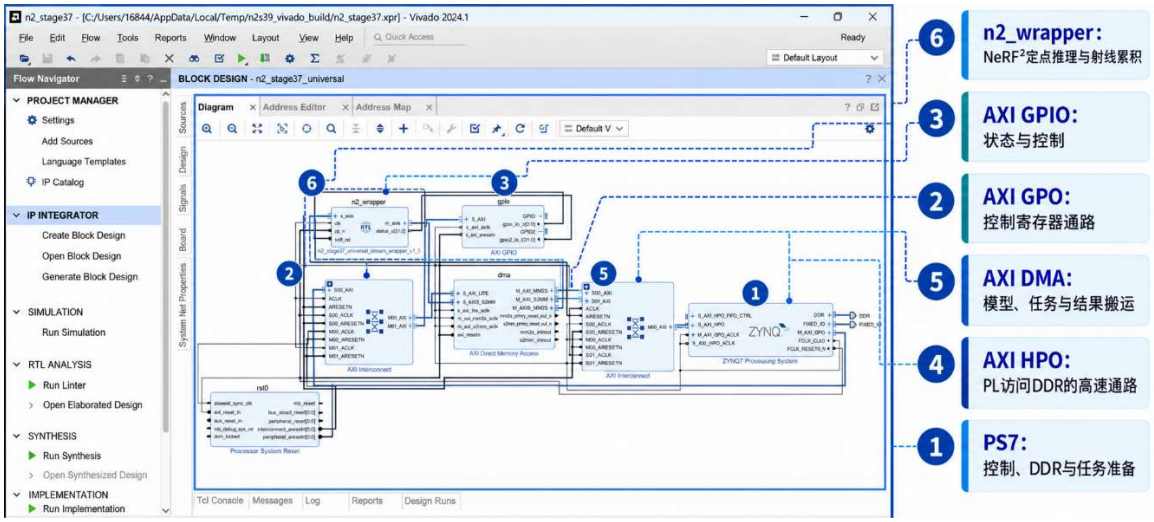


图 2-2 推理 Overlay 的 Vivado Block Design 与功能标注

2.1.3 硬件工程模块与系统架构的对应关系

图 2-2 中的功能标注与工程模块可按表 2-1 对应。其中，GP0 与 HP0 是 PS 侧接口，其通路分别经过 ic_gp0 与 ic_hp。rst0 负责复位分发。Block Design 展示硬件连接，电脑端训练、浏览器 App 和 Linux 服务不会作为 IP 出现在图中。整机关系应结合图 2-1 阅读。

表 2-1 Block Design 模块与功能对应

工程模块	承担的工作	主要连接
ps7	ARM 处理系统与 DDR 控制	GP0 控制访问，HP0 数据访问，提供时钟和复位
ic_gp0	控制互连	PS 的 GP0 连接 DMA 及 GPIO 控制寄存器
dma	DDR 与流接口之间双向搬运	MM2S 送入任务，S2MM 回传结果。内存端经 ic_hp 接 HP0
ic_hp	高带宽数据互连	汇聚 DMA 两条内存访问通路并连接 PS 的 HP0
gpio	软复位与核心状态访问	1 位输出至 soft_rst，32 位输入读取 status_o
n2_wrapper	可配置整数推理与射线渲染	接收输入流，输出每条射线的定点幅值
rst0	时钟域复位管理	分发外设和互连所需的复位信号

2.1.4 完整空间谱推理流程

以用户在 App 中输入一个合法 Tx 坐标为例，在线数据在 PC、ARM、DDR、DMA 和 PL 之间按以下顺序流动。数据集位置查询使用同一主推理通路，只是在请求中附带测试位置编号，并可读取该位置的 GT 用于比较。

1. 接收查询并计算 PC 参考。

PC 后台读取当前场景的模型与 Rx 配置，检查 Tx 坐标，并执行 FP32 参考推理。随后通过以太网向板端 HTTP 服务发送场景、Tx 或测试编号以及所需显示选项。浏览器通过 PC 后台查看任务进度和结果。

2. 准备 Overlay 和场景资源。

   ARM 检查当前 PL 是否运行推理 Overlay。首次启动或从 HDMI 模式返回时，先加载.bit 及.hwh。模型参数以带地址的记录经 AXI-Stream 写入 PL，完成后提交生效。DDR 同时准备方向常量和衰减缓存。同一场景且资源有效时复用已有配置，场景改变时装载相应模型及缓存。

3. 生成 Tx 编码并构造输入流。

   ARM 对 Tx 坐标作位置编码，按该场景冻结的尺度量化为 63 维 INT8 输入，再填入已准备好的射线常量记录。固定 Rx 下的射线方向编码和衰减中间量可以复用。软件把命令头、每对射线的常量及 64 点缓存组成若干传输批次。

4. 通过 DMA 送入计算。

   软件把输入写入 DDR 缓冲并执行缓存刷新，先准备 S2MM 接收，再启动 MM2S 发送。两个发送缓冲轮换使用，当前批次运行时准备下一批数据。DMA 经 HP0 访问 DDR，再通过 AXI-Stream 向加速器输送输入。

5. 执行完整方向网格计算。

   PL 解析命令，按照执行描述符读取稀疏参数，并使用双射线引擎计算当前 Tx 与方向对应的信号参数。随后将信号参数与每条射线 64 点衰减中间量结合，完成定点射线渲染、复数累加和模值计算。分批处理结束后，全部 32400 条射线均得到结果。

6. 接收结果并组织显示。

   S2MM 将 64 位输出传输字写回 DDR。驱动在传输完成后使 CPU 缓存失效，提取每个传输字低 32 位的无符号幅值，按射线顺序排列为 90×360 数组。显示值由原始定点整数直接恢复，再传回 PC 保存和绘图。需要 HDMI 显示时另行进入板端显示阶段。

本作品的 Q15 幅值以 2 的 15 次方为缩放因子，转换关系为：

$$A = \frac{q}{32768}$$

其中 q 表示 PL 输出的无符号定点整数，A 表示恢复后的幅值，缩放因子为 2 的 15 次方。该幅值表示预测空间谱的相对强度。

2.1.5 场景切换与显示机制

四个场景共享同一份推理位流。位流确定计算单元、接口、存储组织和执行能力，场景模型决定这些单元使用的权重、索引、偏置、描述符、移位量与查找表。切换场景时，软件同步更新当前模型、方向常量和固定 Rx 缓存，避免不同场景的数据混用。在推理 Overlay 仍驻留且状态正常的情况下，这一场景切换不需要重新综合或更换推理硬件结构。

系统的三类存储分别承担长期保存、运行时缓冲和片上计算任务。SD 卡保存操作系统、模型文件、配置及结果，DDR 保存运行时数组、射线常量、固定

Rx 缓存和 DMA 缓冲，PL 片上存储供加速器读取模型与中间数据。固定 Rx 缓存随模型、Rx 或几何配置更新，具体数据分工见表 2-2。

表 2-2 数据存放位置与作用

存放位置	典型内容	在推理中的作用
SD 卡	系统、运行时模型、场景配置、保存的结果	为程序装载和后续浏览提供持久文件
DDR	方向常量、衰减缓存、输入输出缓冲	ARM 与 DMA 共享的运行数据空间
PL 片上存储	稀疏权重、索引、偏置、描述符及中间数据	按硬件周期向并行计算单元提供数据

HDMI 显示在空间谱生成后进行。PS 将当前结果整理为对比帧：数据集测试位置有 GT 时可显示 GT、PC 和 FPGA 三幅图。自由 Tx 没有对应 GT 时显示 PC 与 FPGA 两幅图。HDMI 输出提供现场查看途径，不改变此前生成的原始空间谱数值。

HDMI 显示采用独立的 BaseOverlay。启用显示时，系统保存本次推理结果并加载显示位流。下一次推理前，服务恢复推理 Overlay 及相应的模型与缓存。上述流程使推理和显示能够在同一可编程逻辑上分时运行。

完整空间谱的板端驱动时延与应用请求的总响应时间对应不同的处理范围。应用请求还可能包含 PC 参考计算、网络通信、模型装载、绘图和 HDMI 模式切换。计算结果可保存在 PC 或板端，并供后续查询和分析使用。

在线查询使用离线准备的场景模型。输入新的发射端坐标后，系统在当前模型的空间范围和固定接收端配置下执行推理。对于新环境，需要先完成数据准备、模型训练、定点部署及运行时资源生成。

2.2 硬件系统设计

2.2.1 硬件架构

本作品采用 PYNQ-Z2 开发板构成空间谱推理终端，核心器件为 XC7Z020-1CLG400C。芯片内部集成处理系统 PS 与可编程逻辑 PL：PS 运行 Linux、PYNQ 和板端服务，承担模型装载、数据准备与外设控制。PL 实现自定义整数神经网络和射线渲染数据通路。二者位于同一颗 Zynq 器件内部，通过 AXI 互连和板载 DDR 共享任务与结果。

硬件系统由 PC、PYNQ-Z2、供电设备、以太网线和 HDMI 显示器组成。PC 通过以太网向板端提交场景和 Tx 坐标，板端完成推理后回传空间谱。需要现场显示时，再将已生成的图像通过板卡 HDMI OUT 输出。图 2-3 展示的是本作品的实际连接现场。

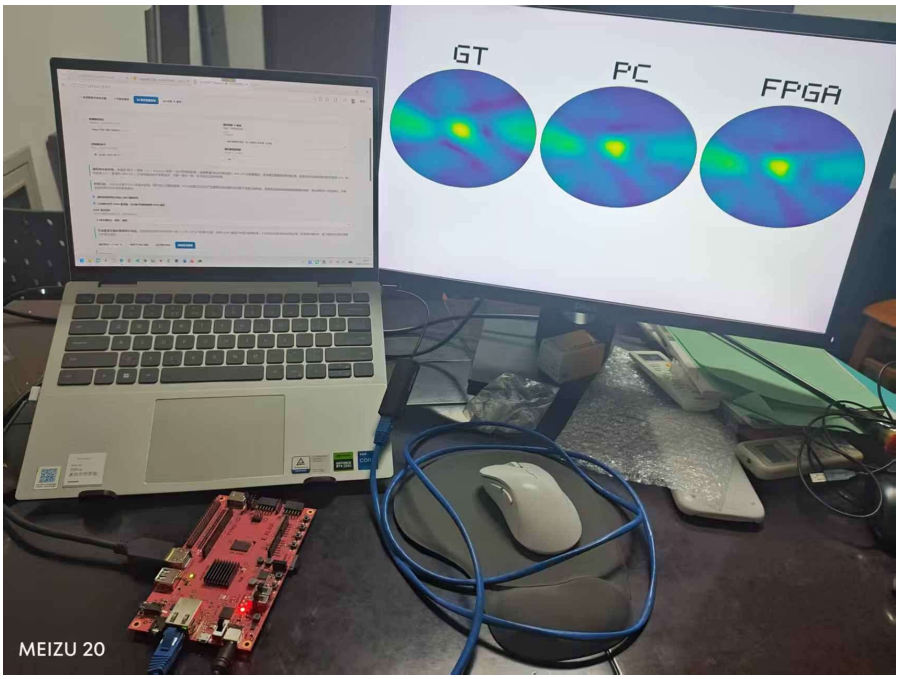


图 2-3 作品实际连接现场与 HDMI 显示效果

PYNQ-Z2 开发板提供 DDR、网口、MicroSD、供电和 HDMI 等基础电路。本文的硬件设计主要包括 PL 计算核心、片上存储、AXI 接口集成及运行时模型装载，板级系统通过现有接口完成连接。系统以预训练模型为基础执行空间谱推理。

2.2.1.1 硬件平台与外部接口

PYNQ-Z2 提供双核 Cortex-A9 处理器、512 MB 板载 DDR3、MicroSD 接口和可重配置 PL。图 2-4 为所用平台的板卡外观，用于定位芯片、DDR 和连接器。实际线缆连接见图 2-3。板载 DDR 是芯片外部的运行内存，PL 内部 BRAM 是计算单元就近访问的片上存储，两者用途和带宽组织不同。板卡基础配置与接口依据厂家资料。

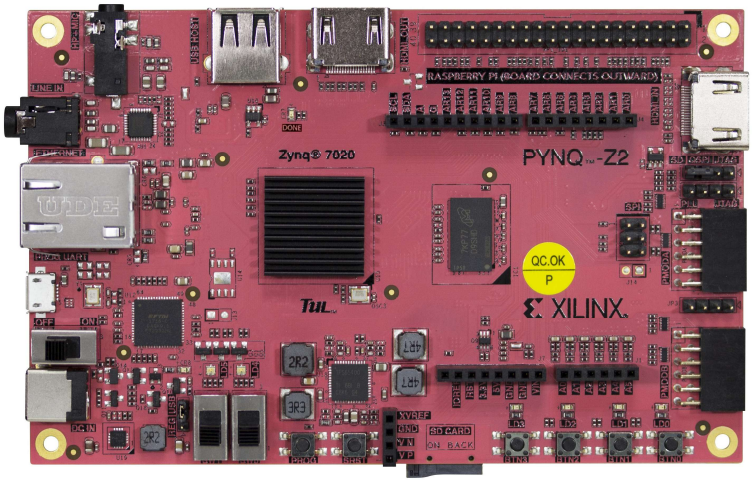


图 2-4 PYNQ-Z2 平台外观与主要器件位置

表 2-3 硬件部件及外部接口分工

部件或接口	本作品中的作用	连接或数据方向
XC7Z020 PS	运行控制软件和网络服务	PC 请求→ARM 任务组织
XC7Z020 PL	整数网络、射线渲染和流接口	定点输入→原始幅值
512 MB DDR3	常量、衰减缓存与 DMA 缓冲	ARM 与 DMA 双向访问
MicroSD	系统启动、模型与结果持久保存	文件→运行内存
RJ45 以太网	PC 与板端之间的通信	请求下发、结果回传
Micro-USB 与 HDMI OUT	供电及结果显示	5 V 输入。图像输出

表 2-3 概括了系统的硬件部件与接口分工。其中，PC 承担任务组织与浮点参考计算，PYNQ-Z2 执行定点推理，外接仪表用于测量板卡输入功率。

2.2.2 样机结构与连接

系统采用桌面式样机，由 PYNQ-Z2 开发板、电脑和外接显示屏组成，通过电源适配器、USB 转网口适配器、网线及 HDMI 线连接。各设备的空间布置和连接关系构成系统运行的物理基础。

常规运行时，PYNQ-Z2 通过开发板套件配套的电源适配器和 DC IN 接口供电。功率测试则采用外接仪表与相应的 USB 供电链路，具体接线见供电模块说明。

通信链路由电脑 USB 接口、USB 转网口适配器、网线和开发板 RJ45 接口构成，承担电脑端任务下发及板端推理结果回传。

显示链路通过 HDMI 线连接 PYNQ-Z2 的 HDMI OUT 接口与外接显示屏。电脑用于操作应用界面，外接显示屏用于展示板端生成的结果画面。

样机以 PYNQ-Z2 为连接中心，电源适配器、电脑和显示屏分别通过供电线、网络链路和 HDMI 线接入。整体设备布置及连接现场见图 2-3。上述装配利用开发板原有连接器完成，结构说明以实际设备和线缆连接为依据。

2.2.3 硬件模块设计

硬件系统包括板级基础电路、Zynq 内部互连和自定义计算核心。板级电路提供供电、启动、存储和外部连接，内部互连承担处理系统控制及 DDR 数据传输，自定义核心执行协议解析、稀疏整数网络计算、定点激活和射线渲染。下文由顶层接口展开各计算与外设模块的设计。

推理 Overlay 的顶层包含 ps7、ic_gp0、dma、ic_hp、gpio、rst0 和 n2_wrapper 七个模块。图 2-2 标注了 GP0 与 HP0 接口及其通路，表 2-4 列出顶层的关键连接。

表 2-4 顶层关键输入输出连接

通路	关键连接	承载内容
控制	PS M_AXI_GP0→ic_gp0→DMA / GPIO	传输地址、长度、启动和状态

通路	关键连接	承载内容
DDR 数据	DMA M_AXI_MM2S、M_AXI_S2MM→ic_hp→PS S_AXI_HP0	读取输入、写回结果
输入流	DMA M_AXIS_MM2S→核心 s_axis	模型记录和推理记录
输出流	核心 m_axis→DMA S_AXIS_S2MM	装载响应或射线幅值
复位与状态	gpio_io_o→soft_rst。status_o→gpio2_io_i	1 位软复位、32 位状态
时钟	PS FCLK_CLK0→互连、DMA、GPIO 和核心	95 MHz 推理时钟域

2.2.3.1 PS 控制通路与 DMA 数据接口

PS 通过 M_AXI_GP0 发起内存映射访问。ic_gp0 的两个主端口分别连接 DMA 的 AXI4-Lite 控制口和 AXI GPIO。当前 HWH 中，DMA 基地址为 0x40400000，GPIO 基地址为 0x41200000。软件通过 HWH 识别 IP 及其地址，避免将地址与其他 Overlay 混用。自定义核心通过流协议装载参数，没有另设直接承载模型权重的 AXI4-Lite 寄存器文件。

DDR 数据通路由 DMA 连接内存与计算核心。MM2S 通道从 DDR 读取输入并生成 AXI4-Stream，S2MM 通道接收核心输出并写回 DDR。两条内存主接口经 ic_hp 接入 PS 的 S_AXI_HP0，ARM 通过驱动准备地址并控制 DMA 传输。

表 2-5 自定义加速器对外端口

核心顶层端口	位宽与方向	用途
s_axis_tdata	64 位，输入	每拍接收一个命令或数据字
s_axis_tvalid / tready	各 1 位，输入/输出	输入流有效与可接收握手
m_axis_tdata	64 位，输出	推理幅值放在低 32 位
m_axis_tvalid / tready	各 1 位，输出/输入	输出有效与接收端反压
m_axis_tlast	1 位，输出	标记当前输出事务的最后一拍
soft_rst / status_o	1 位输入/32 位输出	软复位及运行状态
clk / rst_n	各 1 位，输入	时钟及低有效复位

当 VALID 与 READY 同时为 1 时，一个数据拍完成传输。输出端在下游暂不可接收时保持待发送数据及末拍标记。输入端的记录边界由命令头中的数量与内部状态机共同确定，因此软件按照相同格式和长度组织输入。

ARM 向 DMA 缓冲写入输入后刷新 CPU 缓存，接收完成后使相应缓存失效，再读取结果。驱动先安排 S2MM 接收，再启动 MM2S 发送，并使用两个发

送缓冲交替准备数据。整图 32400 条射线以 64 位传输字回传时共 259200 字节。提取低 32 位后形成 90×360 的原始无符号整数数组。

2.2.3.2 协议解析与运行时模型装载

n2_wrapper 首先解析输入流的命令头。最高 4 位表示操作类型。模型装载命令随后接收地址字与数据字，推理命令随后接收射线常量和缓存记录。协议解析器将命令转换为片上存储写使能、引擎启动、渲染输入和结果回传等控制信号，因此同一组 AXI4-Stream 端口可在不同阶段承载不同内容。

表 2-6 核心输入流支持的操作类型

操作码	功能	主要行为
0x8	装载模型	接收写入目标、地址、分段与数据，装载时撤销模型有效标志
0x9	提交模型	记录模型标识并返回提交响应，未出现协议错误时置模型有效
0x6	构建衰减缓存	使用当前模型计算固定 Rx 相关中间量并回传
0x5	缓存推理	计算当前 Tx 信号分支，结合已准备的衰减缓存渲染
0x4	完整计算模式	接收采样点记录并执行衰减与信号计算
0xF	回显	回传输入字，用于接口连通检查

模型写入目标包括权重、稀疏索引、偏置、执行描述符、逐层移位量以及渲染查找表。装载器先完成各区域写入，再提交生效。正常推理要求模型有效且射线数非零、为偶数，以匹配双射线组织。切换场景时更新参数和配套常量、缓存。计算结构与推理位流保持相同。

核心通过 32 位 status_o 返回错误汇总、空闲状态、网络引擎状态、模型有效性、协议状态、命令类别及剩余射线对数量。控制软件据此判断事务进度并执行相应恢复操作。软复位经寄存器同步后作用于核心状态。

输出侧设置 16 项结果 FIFO，以解耦内部结果生成与 DMA 接收握手。推理结果的 TLAST 由剩余输出数量产生。驱动按事务顺序接收模型装载响应和空间谱幅值，保证控制响应与推理结果分别处理。

2.2.3.3 双射线稀疏网络计算

网络引擎 u_factor 由两组 128 路矩阵向量计算单元 g0、g1，共享运行时模型存储与描述符，并复用 32 路激活流水线。两条射线使用相同权重、各自的输入激活，减少重复存储。中间激活通过两组缓冲交替读写。图 2-5 由本作品 RTL 在 Vivado 中展开并导出，保留真实实例名和端口。

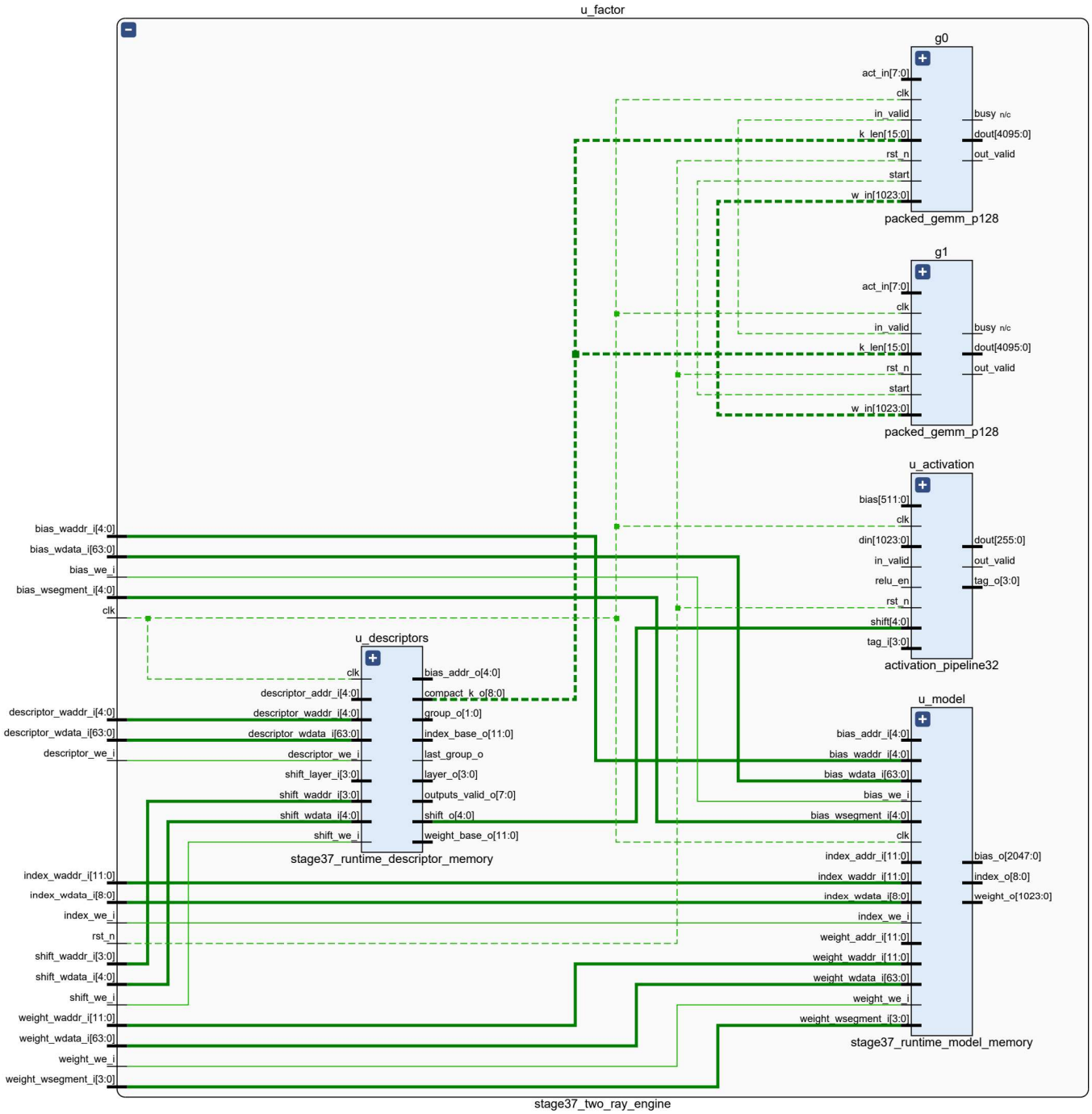


图 2-5 双射线引擎的 Vivado RTL 模块关系视图

u_model 输出 1024 位权重字、9 位输入索引和偏置字。u_descriptors 给出层号、输出分组、压缩输入长度和存储基址。g0、g1 分别形成 128 个 32 位累加结果，再由 u_activation 按 32 个通道分块完成重定标。图为所选层级的模块视图，外围调度和激活缓冲仍由该层 RTL 实现。

2.2.3.4 模型存储与描述符调度

片上权重存储划分为 16 个 64 位存储体。运行时按 64 位数据字分段写入，计算时同地址的 16 段并行读出，组成 1024 位权重字，即 128 个 INT8 权重。每个存储体深度 3072，因此权重区的逻辑容量为 384 KiB。这是片上结构容量，与

SD 卡容量无关。同一时刻装载当前场景的模型，而非将四个场景的全部权重同时驻留。

表 2-7 给出运行时模型与中间量的片上组织。

表 2-7 运行时模型与中间量的片上组织

存储区域	RTL 组织	计算时用途
权重	16×3072×64 位	每次提供 128 个 INT8 权重
稀疏索引	3072×9 位	将压缩列映射回原始输入位置
偏置	32 组×32 段×64 位	每个输出分组提供 128 个 INT16 偏置
描述符	32×53 位	规定层号、分组、长度、基址等
移位量	16×5 位	规定各层算术右移位数
激活缓冲	按射线与输出分组组织的双缓冲	保存前一层整数输出
渲染 LUT	6 组 256 项及 64 项路径损耗表	衰减、幅度、相位与距离因子

稀疏格式以 128 个输出为一组，删除组内所有权重均为零的输入列。每个保留列对应一个 1024 位权重字和一个输入索引。调度器根据索引选择激活值，并将其广播至对应输出乘法通道。压缩输入长度 compact_k 确定该分组需要处理的列数。

执行描述符记录层号、输出分组、权重与偏置的存储位置、有效输出数以及层结束标志。当前模型使用 23 条描述符，硬件空间可容纳 32 项。场景模型按照相同的结构、数值格式和片上容量约束组织。

对冻结整数模型删除全零列可保持计算结果一致，结构约束、微调与 QAT 对预测精度的影响则通过实验评价。固定 Rx 衰减缓存驻留 DDR，并按批次送入渲染器。

2.2.3.5 双 INT8 乘法打包与定点激活

每个 packed_gemm_p128 包含 64 个打包乘法单元，形成 128 路逻辑乘法通道。一个单元将两个 INT8 权重按 16 位间隔组合，再与共同的 INT8 激活进行一次 25×18 位有符号乘法。通过低字符号修正与高字进位修正，恢复两个有符号 16 位乘积。图 2-6 显示其真实 RTL 结构。

两个网络计算单元分别处理两条射线，各单元内部的 128 个乘积进入 32 位整数累加器。整幅空间谱按射线对和输出分组调度，由并行单元完成网络计算。DSP 乘法器的结构与配置依据器件用户手册[12]。

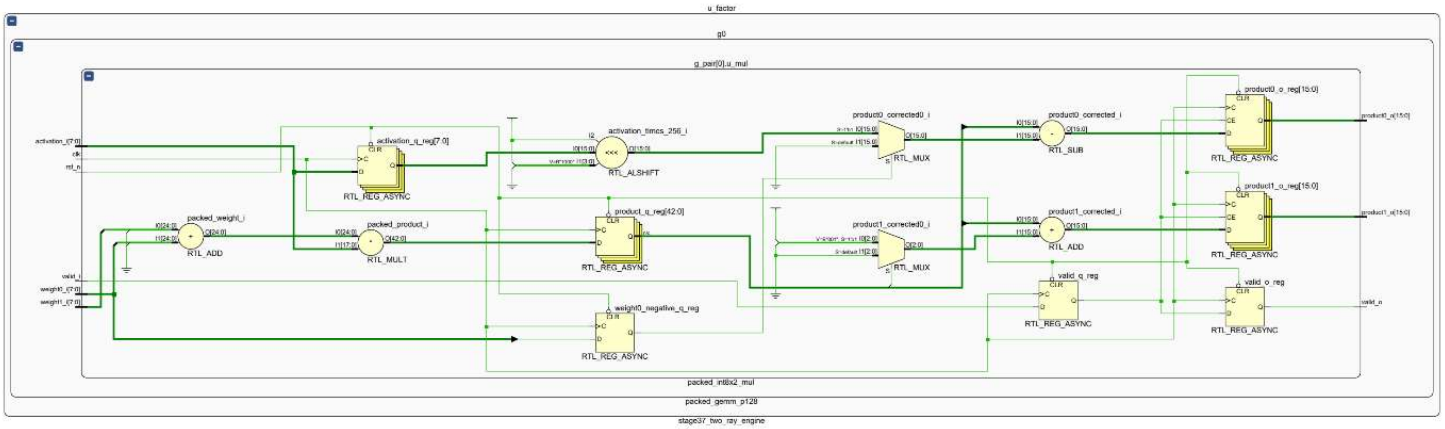


图 2-6 单个双 INT8 打包乘法单元的 Vivado RTL 视图

激活模块依次对累加和执行算术右移、加入 INT16 偏置、按层选择 ReLU，并将结果饱和到 INT8 范围。该过程可表示为：

$$y = sat_8(R((a \gg s) + b))$$

其中 a 为 32 位累加和，s 为本层移位量，b 为已转换到目标输出尺度的偏置。R 按层取 ReLU 或恒等映射。隐藏层启用 ReLU 时将负数置零，sat 将超出范围的数限制在−128 至 127 之间。QAT 前向必须模拟这一顺序，才能使导出模型与 RTL 使用相同的整数规则。

表 2-8 网络乘法与激活的关键数据接口

信号	含义	数据形式
act_in、w_in	一个输入激活与 128 个权重	INT8。1024 位权重字
dout、out_valid	一组矩阵向量累加结果	128×32 位及有效标志
shift、bias、relu_en	激活重定标控制	5 位移位、32×16 位偏置、1 位使能
tag_i、tag_o	输出分块归属	4 位标签随流水线传递

2.2.3.6 射线渲染与幅值计算

网络输出衰减参数与复信号参数后，射线渲染模块按顺序处理每条方向射线上的 64 个空间采样点。模块查表取得衰减系数、信号幅度和相位因子，递推累计衰减与传播相位，并结合路径损耗形成各采样点的复数贡献。各贡献累加后，再由实部和虚部计算幅值。

缓存推理模式下，固定 Rx 对应的衰减中间量由 DDR 输入，当前 Tx 的信号参数来自网络引擎。两者组合为 32 位 raw 记录进入渲染器。低 16 位保存衰减分支的两个 INT8 输出，高 16 位保存信号分支的两个 INT8 输出。6 组 256 项查找表和 64 项路径损耗表随场景模型装载，不在推理时调用浮点三角函数。

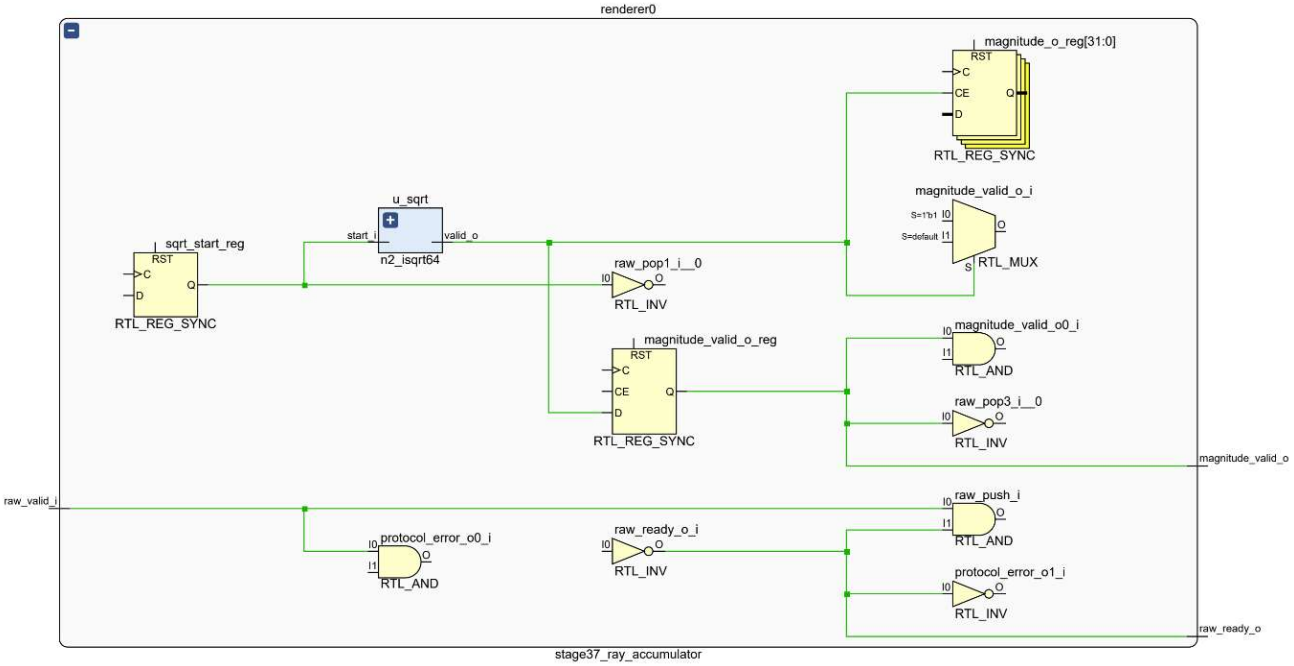


图 2-7 渲染器幅值开方控制与输出有效握手的 Vivado RTL 局部视图

图 2-7 展示整数开方模块 u_sqrt 的启动、结果有效和输出控制。该模块接收 64 位被开方数并输出 32 位结果。射线渲染还包括查表、乘法、移位、累加及 64 点计数，最终按 Q15 缩放关系恢复幅值。

渲染入口设置 64 项 raw FIFO，缓冲网络或缓存输入与多周期渲染之间的速率差。两条射线的网络计算可以并行，而当前实现复用一个渲染器，第二条射线的 64 项 raw 记录暂存后回放。渲染完成信号还受输出端可接收状态约束，结果再经顶层 FIFO 交给 DMA。

2.2.3.7 供电、时钟与复位

功率测量采用 5 V USB 供电链路，依次连接 5 V 电源、FNB58、USB-A 转 Micro-USB 供电线和板卡 Micro-USB 接口，电源选择 J9 置于 USB。仪表记录该连接位置的电压、电流与输入功率，测量范围覆盖 PS、PL、DDR 及实际启用的板载电路。

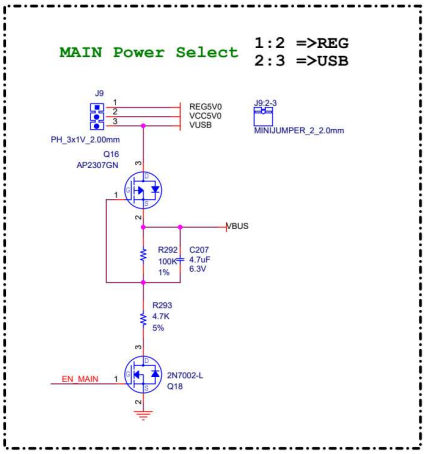


图 2-8 厂家原理图中的 J9 电源选择与 USB 供电通路局部

图 2-8 是厂家原理图的 J9 电源选择和局部 USB 供电通路。原理图中 J9 将 REG5V0 或 VUSB 接入 VCC5V0。具体档位以板卡丝印 USB、REG 及厂家手册为准。USB 路径使用 5 V。DC IN 是另一条经稳压器的外部供电路径，厂家手册给出的输入范围为 7～15 V、推荐 12 V。两种入口的电压要求不能互换，本作品 USB 测量链路不将高压快充电压送入 Micro-USB。

推理 Overlay 使用 PS 的 FCLK_CLK0 提供 95 MHz 工作时钟。

rst0 接收 PS 复位信号并向互连与外设分发复位。核心 rst_n 接入低有效外设复位，GPIO 另提供 1 位 soft_rst 信号，经两级寄存器同步后作用于内部状态机。重新加载 Overlay 或复位后，系统重新建立模型有效状态。

2.2.3.8 以太网与 MicroSD 接口

PC 与板端之间的数据通过板载千兆以太网 PHY 和 RJ45 传输。原理图 U5 为 RTL8211E-VL，PS 与 PHY 之间包含 ETH_TXD、ETH_RXD、时钟及控制信号，ETH_MDC 和 ETH_MDIO 承担管理配置。PHY 经差分线与 RJ45 磁性器件连接。本作品使用 Linux 网络栈发送 HTTP 请求及结果，不在自定义 PL 核心内重新实现以太网 MAC。 图 2-9 给出厂家原理图中的以太网 PHY 及关键收发信号局部。

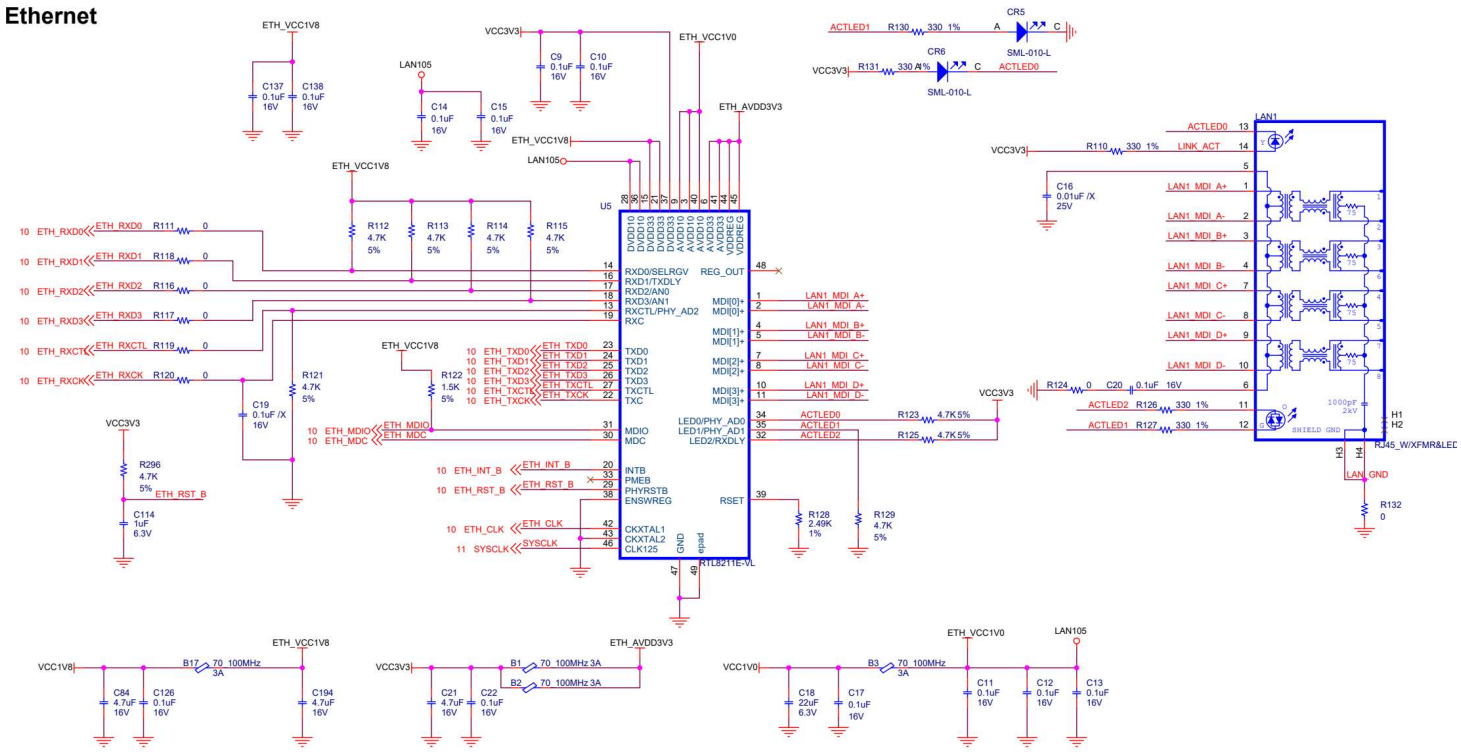


图 2-9 厂家原理图中的以太网 PHY 及关键收发信号局部

MicroSD 保存系统启动文件、板端代码、运行时模型和结果。板级电路通过电平转换连接 SD 接口，主要信号包括 SD_D0～SD_D3、SD_CCLK、SD_CMD 和 SD_CD。选择 SD 启动模式后，PS 从卡中加载启动环境。运行时文件先读入 DDR，再由驱动装载至 PL。

以太网连接电脑与板端服务，MicroSD 保存系统、代码、模型和结果文件。图 2-10 展示 MicroSD 接口与电平转换电路。

SD CARD

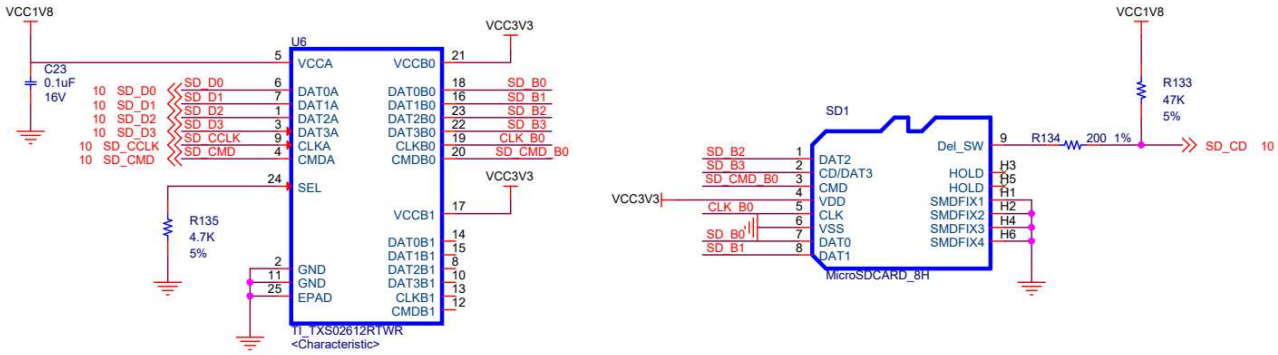


图 2-10 厂家原理图中的 MicroSD 接口及电平转换局部

2.2.3.9 DDR3 存储与共享缓冲

本作品所用 PYNQ-Z2 按 512 MB、16 位数据总线 DDR3 配置运行，连接 Zynq 的 PS 专用 DDR 接口。图 2-11 采用厂家公开原理图说明接口，图示器件料号不用于推导实物容量。关键网名包括 DDR3_D[15:0]、MEM_ADD、MEM_BANK、差分时钟 MEM_CLK0_P/N、数据选通 MEM_DQS 及读写控制线。本作品 PL 通过 HP0 访问 PS 内存系统，不直接用自定义 RTL 重新驱动 DDR 芯片引脚。

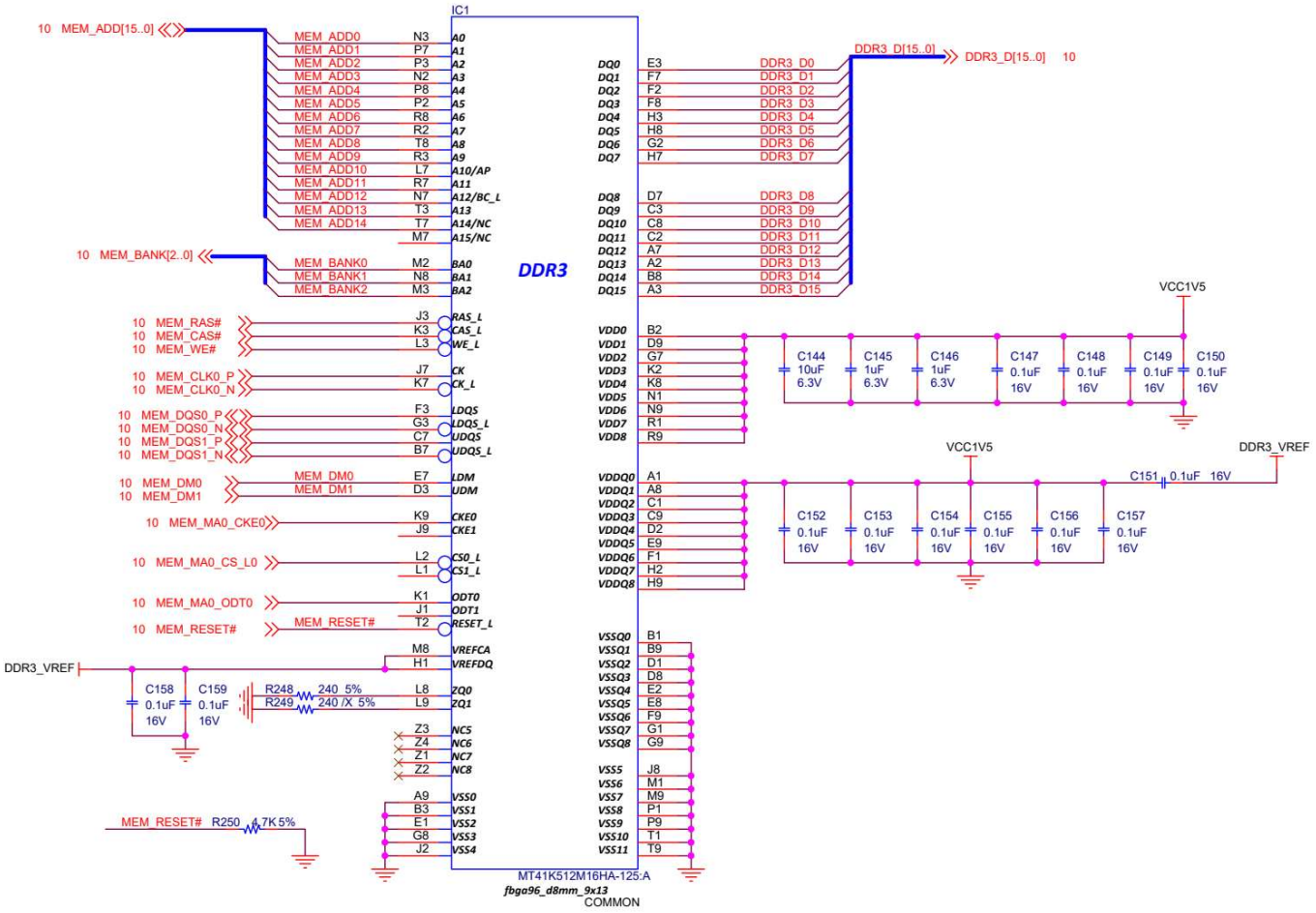


图 2-11 厂家原理图中的 DDR3 器件及地址数据控制信号局部

DDR 运行内容分为模型文件解析后的临时数组、固定方向常量、固定 Rx 衰减缓存、DMA 发送缓冲和 DMA 接收缓冲。每次查询只更新与 Tx 相关的编码并组合已有常量、缓存，分批送入 PL。固定 Rx 缓存与场景模型及采样配置匹配，切换模型或 Rx 配置时需读取或重新生成对应缓存。 表 2-9 给出物理总线与内部数据宽度的区别。

表 2-9 物理总线与内部数据宽度的区别

接口或存储	本作品中的数据宽度	需要区分的含义
板级 DDR3 DQ	16 位物理总线	DDR 器件与 PS 的电气连接
PS HP0、流接口	64 位通路	DMA 内存访问和每拍传输字
片上权重读取	1024 位并行字	16 个存储体汇合后供计算使用
最终原始幅值	32 位无符号整数	位于 64 位输出字的低 32 位

2.2.3.10 HDMI 显示与工作模式

HDMI 输出用于把已完成的空间谱显示到独立显示器。厂家电路中的 HDMI OUT 连接 PL 侧差分数据与时钟信号，关键网名为 HDMI_TX_D0/1/2_P/N、HDMI_TX_CLK_P/N。SCL、SDA 为显示器相关管理通路，另有热插拔检测及 5 V 接口供电。图 2-12 是 HDMI TX 局部，不能与板上 HDMI IN 混接。

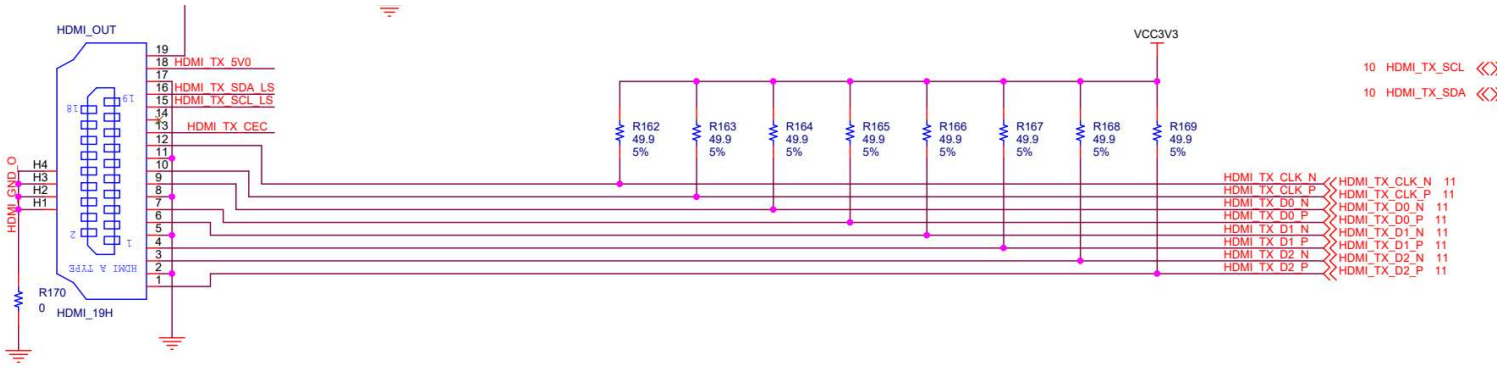


图 2-12 厂家原理图中的 HDMI OUT 差分数据时钟连接局部

当前服务在 PS 侧把 GT、PC 参考和 FPGA 结果组织为对比帧。数据集测试位置有 GT 时显示三幅图，自由 Tx 缺少 GT 时显示 PC 与 FPGA 两幅图。界面帧以 1280×720 组织，并按显示器比例选项进行适配。颜色映射和图像排版不改变原始定点空间谱。

本作品推理 Overlay 和 PYNQ 的 HDMI BaseOverlay 是两份完整位流。启用 HDMI 时先保存本次推理结果，再加载 BaseOverlay 输出图像。下一次 FPGA 请求先停止视频会话、恢复同一份推理 Overlay，再装载当前场景模型和所需缓存。因此，当前硬件是推理和显示分时使用 PL。 表 2-10 给出推理与 HDMI 显示的分时工作关系。

表 2-10 推理与 HDMI 显示的分时工作关系

阶段	当前 PL 配置	主要输入输出
空间谱推理	95 MHz 自定义推理 Overlay	DDR 定点记录→原始幅值
显示准备	PS 保存并组织对比图	数值数组→显示帧
HDMI 输出	PYNQ BaseOverlay	显示帧→HDMI OUT
继续推理	恢复自定义推理 Overlay	重新建立模型与硬件句柄

HDMI 是板端结果的现场查看接口。推理期间保持以太网链路用于提交任务和回传数据。使用浏览器查看结果与使用 HDMI 查看结果属于两种显示途径，均以 PL 已经生成的原始结果为基础。

2.2.3.11 硬件资源与时序实现

推理 Overlay 在 Vivado 2024.1 中针对 XC7Z020CLG400-1 完成布局布线。实现报告记录的实际时钟约为 94.118 MHz，时钟周期为 10.625 ns，最差建立裕量 WNS 为+0.015 ns，最差保持裕量 WHS 为+0.019 ns。表 2-11 列出该位流的资源和时序结果。

表 2-11 当前推理 Overlay 的布局布线资源使用

资源	使用量 / 器件总量	占用率
LUT	50069 / 53200	94.11%
寄存器	48464 / 106400	45.55%
BRAM 等效 36 Kb 块	127.5 / 140	91.07%
DSP48E1	178 / 220	80.91%
Tcl 目标频率	95 MHz	工程配置目标
实现报告时钟	约 94.118 MHz	周期 10.625 ns
WNS / WHS	+0.015 ns / +0.019 ns	归档布局布线报告

其中 BRAM 等效用量由 115 个 RAMB36 与 25 个 RAMB18 按容量折算而得，每个 18 Kb 块按 0.5 个 36 Kb 块计，因此等效计数可出现小数。网络乘法的双 INT8 打包和稀疏列组织用于减少计算及存储开销。寄存器、FIFO 和分阶段渲染用于切分组合路径并协调数据流。

新场景模型需要满足既有网络结构、描述符格式、整数数值范围和片上容量要求。部署时先离线完成训练、结构适配、量化及参数打包，再将生成的运行时资源装载到板端。

本节介绍了系统的板级连接、存储与互连以及自定义计算模块。PC 与 ARM 组织查询并准备数据，PL 依据整数规则生成完整方向空间谱，HDMI 显示在结果生成后进行。各模块通过数据格式、握手信号和复位状态共同构成运行链路。

2.3 软件系统设计

2.3.1 软件总体架构

软件系统把离线模型准备、电脑端交互与参考计算、板端任务控制和 FPGA 驱动连接为完整的空间谱查询流程。离线程序将已训练的 NeRF²模型参数转换为硬件可装载的运行时模型，并生成固定接收机对应的中间量。在线运行时，用户在浏览器中选择场景和测试位置，或输入场景范围内的发射机三维坐标。电脑端组织请求并计算 FP32 参考，板端 ARM 装载模型、准备数据和控制 DMA，PL 完成主推理，结果返回电脑并可输出到外接显示屏。

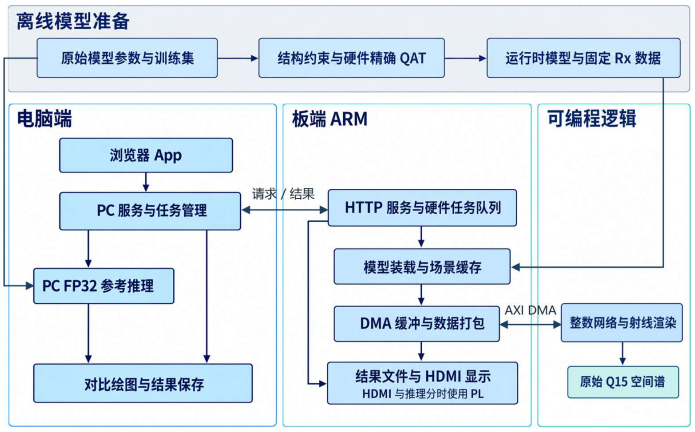


图 2-13 软件系统总体架构与离线在线数据关系

图 2-13 上部是部署前执行的离线流程，下部是每次查询使用的在线链路。原始浮点模型参数供 PC 参考推理使用。结构适配、QAT 和稀疏导出后的模型供板端使用。两条计算路径具有共同的场景、Tx 和方向网格，分别得到参考空间谱与硬件空间谱。PC 参考用于对比和显示，不替代 PL 输出，也不进入 PL 的神经网络计算。

当前实现采用本地 PC 与 PYNQ-Z2 通过有线网络协同运行，不依赖云端推理服务。浏览器承担显示和交互，实际 Python 计算发生在 PC 服务进程和板端服务进程中。图 2-13 中的 PL 计算模块用于说明软件驱动的对象，其内部电路见 2.2 节。

2.3.1.1 软件运行环境与交互界面

PC 端以 Python HTTP 服务组织场景目录、模型加载、后台任务和 Matplotlib 绘图，以 HTML、CSS 和 JavaScript 实现浏览器界面。板端使用 Linux、PYNQ、NumPy 与 Pillow 完成 Overlay 配置、连续物理内存分配、DMA 收发、整数复核及 HDMI 画面组织。PC 侧可使用 GPU 或 CPU 执行浮点参考推理。表 2-12 给出软件运行位置与主要依赖，PyTorch 与 PYNQ 接口分别参考文献[13]和[14]。

表 2-12 软件运行位置与主要依赖

位置	主要软件	承担任务
浏览器	HTML、CSS、JavaScript	输入参数、显示状态和结果图
PC 进程	Python、PyTorch、NumPy、Matplotlib	FP32 参考、任务协调、对比和保存
板端进程	PYNQ、NumPy、Pillow、HTTP 服务	模型装载、DMA、ARM 复核与 HDMI
离线准备环境	PyTorch、训练数据、导出脚本	结构适配、定标、QAT 和运行时编译

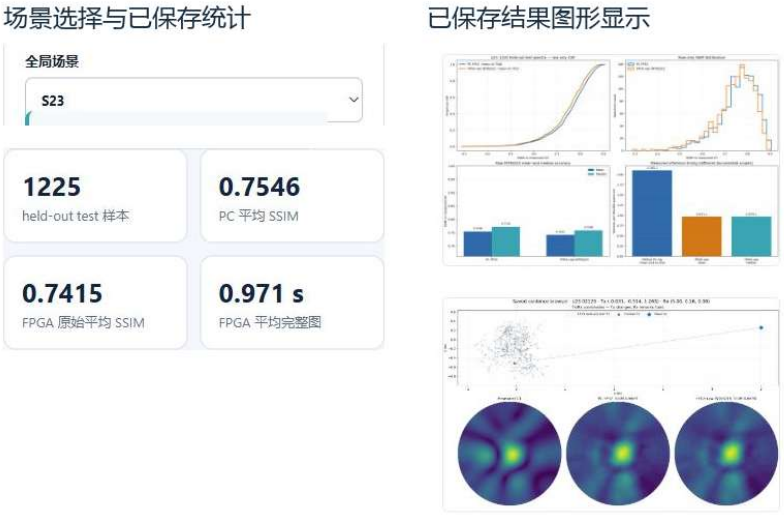


图 2-14 现有 App 的场景选择与总体统计界面局部

图 2-14 中截图来自现有 App 的实际本机界面，统计项读取已保存的场景结果。查询页、总体统计页和保存结果浏览页使用同一个场景选择器，但对应的计算行为不同：查询会发起新任务，保存结果浏览读取历史文件，总体统计负责批量运行与汇总。

2.3.1.2 软件目录与数据组织

PC 包和板端包均以 scene_registry.json 登记场景，以 scenes/<scene>/作为每场景独立目录。登记信息包含场景 ID、测试样本数、Tx 范围和硬件文件身份。场景目录进一步保存模型、配置、样本目录与运行结果。代码负责通用流程，场景文件负责不同环境下的参数与数据。表 2-13 给出主要目录及数据流向。

表 2-13 主要目录及数据流向

路径或文件	内容	读取位置
pc_app/	PC 路由、场景核心与静态网页	PC 服务进程
jupyter/	板端服务、驱动、装载器和整数参考	板端服务或 Notebook
scenes/<scene>/scene.json	场景配置、范围及模型信息	PC 与板端上下文
model/stage37_runtime_model.npz	整数权重、索引、偏置、描述符和 LUT	运行时模型装载器
model/quant_manifest.json	位置编码尺度与逐层定点规则	Tx 量化及模型解释
model/attenuation_u16.npy	固定 Rx 的 90×360×64 中间缓存	完整图驱动与 ARM 参考
model/constant_packets/	18 个方向常量基础包及清单	ConstantSource
data/stage32/all_test_catalog.json	测试 ID、Tx、GT 路径与参考指标	单样本和批量任务
evidence/	逐次报告、保存图及批量状态	结果浏览与续跑

文件名中的 stage 编号对应工程迭代阶段。运行时模型保存推理所需的数据，原始 PyTorch 模型用于训练和浮点参考计算，NeRF²参考实现见文献[15]。

2.3.2 软件模块与数据流程

软件由界面与场景路由、PC 任务协调、浮点参考、板端业务服务、运行时装载、数据打包、DMA 驱动、整数复核以及结果显示等模块组成。以下先介绍部署前的模型准备，再沿一次在线查询的调用顺序说明各模块。图 2-15 概括浏览器到硬件驱动的调用链，表 2-14 列出顶层模块的接口。

表 2-14 顶层模块的接口边界

函数或方法	关键输入	返回值或主要写入
startMeasured / startFree	样本 ID 或 tx、板端 URL、复核及显示选项	payload 与异步任务轮询
start_job	payload 字典	job_id。在 JOBS 登记任务
run_one_job	job_id、payload	写本次报告和对比图，更新 JOBS
run_request	scene、payload	板端报告、FPGA 图、Q15 哈希
run_map	scene、tx、可选 pc/gt/progress	report、fpga、result 三个返回值

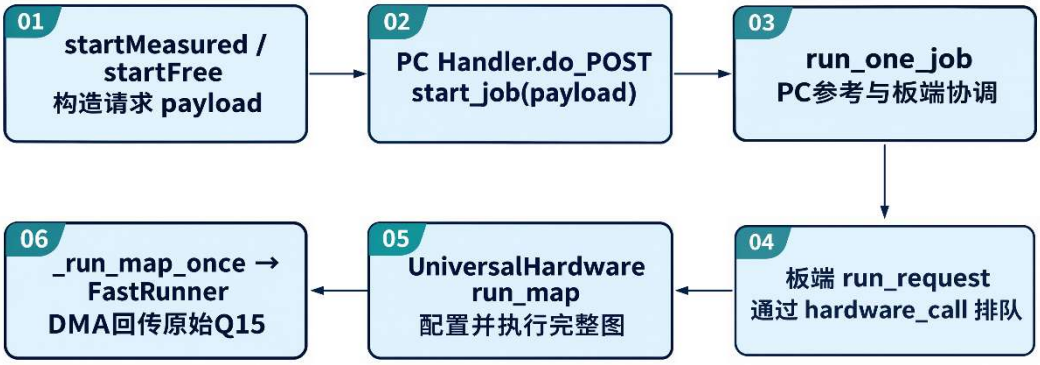


图 2-15 从浏览器请求到完整图驱动的顶层函数调用

图 2-15 给出了在线查询的数据流。浏览器提交任务后，PC 组织参考推理并向板端发送请求，板端硬件线程完成场景配置与空间谱计算，DMA 将原始 Q15 结果写回内存。

上述流程将任务协调与硬件执行分配给不同服务。PC 负责参考计算和结果整理，板端负责管理推理硬件及其运行状态。

2.3.2.1 离线模型准备

模型准备程序接收场景的原始训练模型参数、配置、数据集和既定训练测试划分。处理顺序是结构适配、训练集定标、硬件精确 QAT、整数参数导出、稀疏压缩和运行时资源生成。该过程在 PC 完成。板端在线查询只读取已经生成的文件。 表 2-15 给出模型准备调度函数的输入输出。

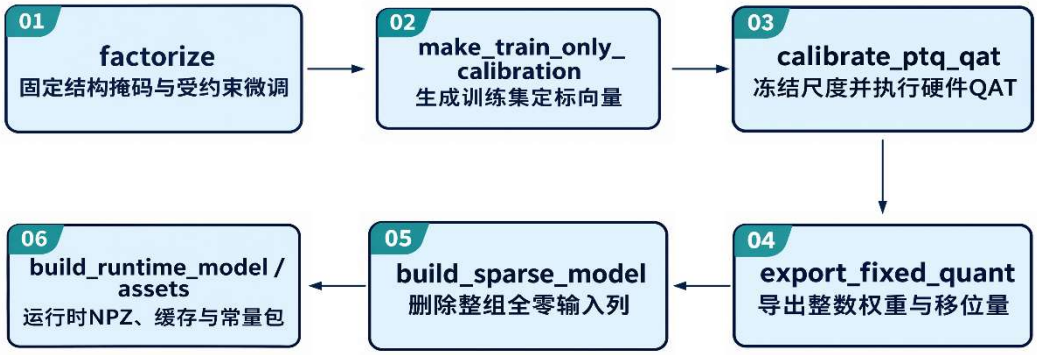


图 2-16 离线模型准备脚本的执行关系

如图 2-16 所示，结构适配确定模型的数据依赖，定标与量化感知训练使模型适应目标整数规则，稀疏导出进一步生成板端可装载的模型和运行时资源。该流程在部署前完成。

表 2-15 模型准备调度函数的输入输出

函数或方法	关键输入	返回值或主要写入
factorize	原始参数路径、场景配置、结构适配步数	结构适配后的模型参数文件
calibrate_ptq_qat	结构适配模型、训练集、定标与 QAT 配置	训练定标矩阵、QAT 模型与整数导出
evaluate	FP32 模型、整数导出、测试划分	完整评估报告和软件判定结果
build_runtime	冻结的 QAT 整数导出及量化清单	运行时 NPZ 和固定 Rx 资源

模型训练和 QAT 使用训练划分中的样本计算损失，测试划分用于评价空间谱预测质量。

2.3.2.2 结构约束与受约束微调

固定 Rx 时，同一方向上的空间采样位置不会随 Tx 改变。为使网络能够分离固定中间量和随 Tx 变化的计算，本作品对两个连接块施加固定零掩码。原始 NeRF²衰减层第 5 号连接的输入布局为 63 维位置编码与 256 维特征的拼接，保留前 63 列并清零后 256 列。信号首层清零前 256 列共享特征。这里的第 5 号对应代码索引 5，即从 0 开始编号。 表 2-16 给出结构适配中的关键操作。

表 2-16 结构适配中的关键操作

函数或语句位置	具体处理	目的
mask_feature_block (model)	attenuation_linears[5].weight[:,63:]置零	使该连接只依赖位置编码
同一掩码函数	signal_linears[0].weight[:,:256]置零	去除信号分支的共享位置特征输入

函数或语句位置	具体处理	目的
微调 main 中的参数选择	不优化 feature_layer 及衰减前 5 层参数	避免更新已经被断开的特征路径
每步反向与更新	清零对应梯度。Adam 后再次施加掩码	保持零连接在整个微调期间成立
validate(...)	前后分别计算评估图的 SSIM、MSE	记录结构变化造成的预测差异

受约束微调从原始训练参数初始化，每步从训练发射位置和方向射线中采样，以渲染结果与训练标签之间的均方误差更新保留参数，并采用梯度裁剪和余弦学习率。固定零掩码约束网络结构，微调用于改善该结构下的预测能力。

结构适配建立了固定接收端衰减计算与在线信号计算之间的数据依赖。随后对冻结整数权重删除整组全零列，可在保持整数模型输出一致的条件下压缩权重。

2.3.2.3 训练集定标与硬件精确 QAT

定标程序根据训练划分中的发射位置、方向网格和深度采样位置构造输入向量。默认输入为 8192×9 的 float32 矩阵，每行包含空间采样位置、视线方向及发射端坐标各 3 个分量。位置编码和层输出的统计量用于确定输入尺度、逐层输出尺度及右移量。图 2-17 展示硬件精确 QAT 的前向计算与参数更新过程，表 2-17 列出相关函数的输入和输出。量化感知训练的整数推理方法可参考文献[16]。

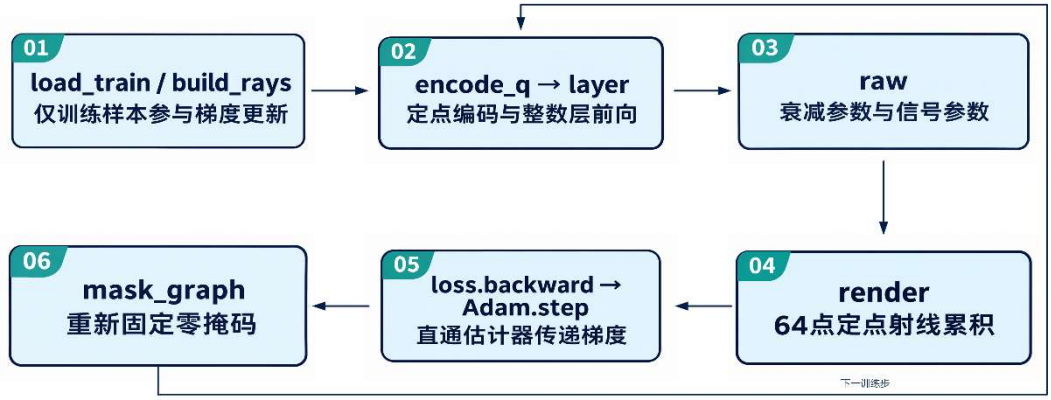


图 2-17 硬件精确 QAT 的前向和参数更新流程

如图 2-17 所示，训练循环利用射线预测结果与标签计算损失，通过近似梯度更新模型参数，并在每次更新后重新施加结构掩码，从而保持既定的分支依赖关系。结构相似度用于评价输出质量。

表 2-17 QAT 主要函数的输入输出

函数或方法	关键输入	返回值或主要写入
encode_q	三维坐标或方向、63 维尺度	63 维 INT8 规则的量化编码
HardwareQAT.layer	层索引、量化输入 xq、尺度 sx	模拟移位、偏置、饱和、激活的输出

函数或方法	关键输入	返回值或主要写入
HardwareQAT.raw	point、direction、tx，末维均为 3	每个采样点的四个网络参数
HardwareQAT.render	一批 origin、direction、tx	64 点定点渲染后的预测幅值
mask_graph	当前模型	再次保持两个连接块为零

QAT 前向模拟目标 INT8 权重、INT16 偏置、算术右移、饱和和查表渲染，并通过直通估计器为取整与查表操作传递近似梯度。训练更新浮点参数，导出阶段按照冻结的量化规则生成整数模型。量化尺度由训练数据在训练前确定。

2.3.2.4 QAT 数值顺序与训练接口

HardwareQAT.layer 按照输入和输出尺度将浮点权重量化为有符号 INT8，依次执行矩阵累加、指定算术右移和量化偏置相加，再按层配置执行 ReLU 并饱和至 INT8 范围。

HardwareQAT.raw 分别产生衰减和信号参数。render 对 64 个采样点执行相位递推、衰减因子递推、路径损耗和复数累加，最后用整数开方前向得到幅值。训练中的开方使用可导的替代梯度，前向值仍按目标整数结果组织。训练损失为一批训练射线预测与 GT 的均方误差。表 2-18 给出 QAT 输入与产物。

$$L = \frac{1}{B}\sum_{i=1}^{B}(\hat{y}_i - y_i)^2$$

其中 B 为当前批次射线数，$\hat{y}$表示定点规则前向输出的幅值，y 表示训练图对应像素的标签。该损失只对训练数据反向传播。SSIM 表示结构相似度指数，用于评价空间谱结构。结构相似度的定义见文献[17]。

表 2-18 QAT 输入与产物

项目	数据形式	用途
模型参数与 config	原始结构适配模型、网络配置	建立可训练前向
quant_manifest	输入尺度、层尺度、移位、最终尺度	固定目标整数契约
训练数据	训练图、训练 Tx、固定 Rx 与方向	提供监督幅值及几何位置
checkpoint_qat.tar	含更新参数的 PyTorch 模型文件	作为整数导出源
checkpoint_qat.json	步数、批次、学习率及输入身份	记录本次训练配置

2.3.2.5 整数模型导出与运行时资源生成

export_fixed_quant 按照冻结的量化清单把 QAT 模型转为整数权重、偏置和移位量。build_sparse_model 按 128 个输出的分组处理权重，删除组内全部为零的输入列，保存剩余列的原始索引。build_runtime_model 再将这些内容与渲染 LUT 封装为 stage37_runtime_model.npz，并生成配套 JSON 清单。

表 2-19 给出运行时模型 NPZ 的关键数组。

表 2-19 运行时模型 NPZ 的关键数组

数组名	类型与形状	含义
weights_u8	uint8，N×128	按位保存 INT8 权重，计算时解释为有符号数
indices_u16	uint16，N	压缩列对应的逻辑输入位置
biases_i16	int16，23×128	各输出分组的量化偏置
descriptors_u64	uint64，23	按位打包的层、分组、长度和基址
shifts_u8	uint8，13	各逻辑层使用的右移量
衰减和信号 LUT	6 组，每组 256 项	幅度、衰减和相位的定点查表值
path_loss_u16	uint16，64	每个深度采样的路径损耗因子
model_id / scene	uint32 标量 / 字符串	运行时提交令牌和场景名

当前四场景 NPZ 中 N 分别为 s23 的 1448、Meetingroom 的 1851、Office1 的 1824 和 Office2 的 2449，均低于硬件 3072 字的权重存储深度。这里一个权重字包含 128 个 INT8 权重，N 不是单个权重的数量。导出器同时检查 23 条描述符及关键分支的输入依赖约束，防止生成与现有硬件不兼容的模型。

build_scene_runtime_assets 生成 attenuation_u16.npy、方向常量基础包和 512 条缓存核对数据。衰减缓存的 90×360×64 个 uint16 元素共 4147200 字节，每个元素打包两个 INT8 衰减参数。Tx 变化时复用该缓存，模型、Rx 或几何配置变化时应使用重新生成的配套资源。

2.3.2.6 PC 服务与场景管理

PC 总入口 stage39_pc_server.py 读取注册表，为四场景建立各自的上下文。context(scene)通过独立模块命名空间执行场景核心代码，使目录、JOBS 任务字典和 MODEL_CACHE 等变量按场景隔离。这样切换场景时不需要复制整套 HTTP 服务器，也不把另一个场景的模型或保存路径混入当前请求。 图 2-18 给出 PC 服务入口与场景上下文的建立。表 2-20 给出场景路由函数的输入输出。



图 2-18 PC 服务入口与场景上下文的建立

图 2-18 中，各场景通过独立上下文隔离模型缓存、任务状态和文件目录，共享 GPU 计算与绘图入口由相应锁协调。该设计支持场景切换时复用通用服务

流程。

表 2-20 场景路由函数的输入输出

函数或方法	关键输入	返回值或主要写入
context	场景 ID 字符串	MODULES 中的场景模块对象
Handler.choose	当前 HTTP 请求路径	self.scene 和 self.module。改写相对路径
prefix_urls	JSON 结构、场景 ID	带/scene/<id>/前缀的资源和任务链接
Handler.do_GET / do_POST	HTTP 路径、JSON 请求体	分派后的 JSON 或文件响应

LOAD_LOCK 保护场景模块初始化。GPU_LOCK 串行化共享 GPU 上的 PC 参考任务。PLOT_LOCK 保护绘图入口。HTTP 可并发响应状态和文件请求，具体推理仍受模型、GPU 和板端硬件队列约束。

2.3.2.7 浏览器交互与输入校验

前端使用 scene 保存当前场景，用 sceneEpoch 区分场景切换前后的异步请求。changeScene 增加版本号、清除旧显示并调用 init 读取新目录。init 更新样本选择、Tx 边界、Rx 坐标和已有结果。轮询函数发现版本变化时结束旧任务的界面更新，避免迟到响应覆盖当前场景。 图 2-19 给出浏览器场景切换与查询请求流程。表 2-21 给出前端关键函数及变量。

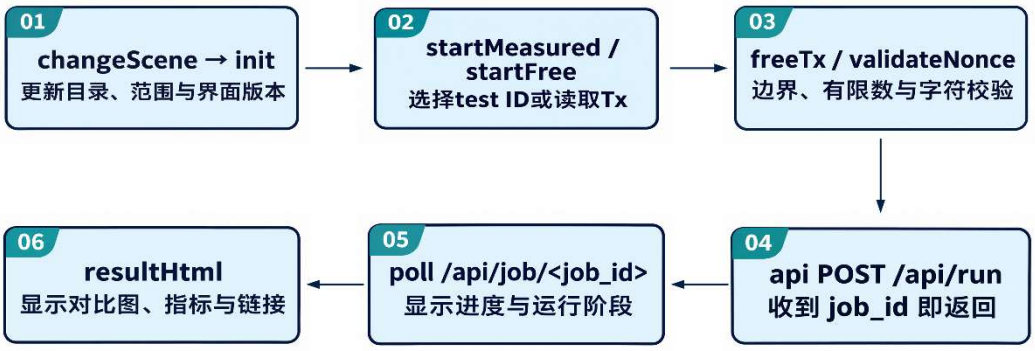


图 2-19 浏览器场景切换与查询请求流程

图 2-19 展示场景切换与异步查询过程。界面使用 sceneEpoch 区分不同场景版本，并通过轮询获取任务状态，以保证显示结果与当前场景对应。

表 2-21 前端关键函数及变量

函数或方法	关键输入	返回值或主要写入
init / applyScene	health、catalog、existing 结果	刷新候选样本、范围、统计和控件
freeTx	txX、txY、txZ 及 catalog.tx_bounds_m	三元素 tx 数组。越界或非有限数报错
validateNonce	空值或最多 64 个可打印 ASCII 字符	合法时通过。否则抛出输入错误

函数或方法	关键输入	返回值或主要写入
startMeasured / startFree	样本 ID 或 tx、复核和 HDMI 选项	向/api/run 提交 payload
poll	job_id、当前场景与版本	显示进度。complete、paused 或 error 时结束

measured 模式的 Tx 从测试目录查得，用户选择样本 ID 即可。free 模式直接提交三维坐标，并在 PC 和板端分别检查范围。范围是当前场景登记的坐标边界，不表示模型已覆盖任意未知环境。随机种子只影响复核位置，不改变预测空间。

2.3.2.8 测试样本与连续坐标查询



图 2-20 现有 App 中测试样本选择与任意 Tx 坐标输入局部

两种查询使用相同的推理驱动，差别在于输入来源和是否存在 GT。测试样本模式从完整测试目录读取 Tx 及原始标签，可比较 GT、PC 和 FPGA 三者。任意 Tx 模式没有该坐标对应的 GT，显示 PC 与 FPGA 两幅预测并比较二者差异。这里的 GT 统一指数据集原始标签，s23 来自测量，其余场景标签来自对应仿真数据。 表 2-22 给出查询请求 payload 的关键字段。

表 2-22 查询请求 payload 的关键字段

字段	数据形式	含义
mode / spectrum_id	measured 或 free / 测试 ID	决定按目录取 Tx 还是读取输入坐标
tx	长度 3 的数值数组，单位 m	free 模式的发射机坐标
board_url	http://板卡 IP:8311	板端业务服务地址

字段	数据形式	含义
nonce / challenge_count	字符串 / 8、16 或 32	可选随机射线复核
run_arm / show_hdmi	布尔值	是否运行整数复核与物理显示
hdmi_display_aspect	显示比例枚举	显示器比例适配方式

2.3.2.9 PC 任务调度

start_job 生成带时间戳和随机后缀的 job_id，在 JOBS 登记 running 状态后启动后台线程执行 run_one_job，HTTP 立即以 202 返回任务 ID。浏览器随后查询任务状态，因此推理过程不会长期占用一个同步页面操作。后台线程按 phase 和 progress 记录当前步骤。 图 2-21 给出 PC 单次任务的分支、协调和保存流程。表 2-23 给出单次任务相关函数的接口。



图 2-21 PC 单次任务的分支、协调和保存流程

图 2-21 中，测试样本模式读取既定坐标与标签，连续坐标模式读取用户输入。两种模式均先执行 PC 参考计算，再请求板端推理并保存结果。

表 2-23 单次任务相关函数的接口

函数或方法	关键输入	返回值或主要写入
run_one_job	当前任务 ID、模式及选项	更新 JOBS 并写入本次运行目录
live_pc_render	本场景 Tx	pc 图、pc_seconds、device
board_run_resilient	板端地址、PC 图和推理参数	板端 JSON 结果。连接失败时有限重试
wait_for_board	板端请求 Future、URL 及 job_id	返回完成结果，同时更新进度
array_from_b64	压缩 NPZ 的 Base64 字符串	有限数、90×360 的 float64 数组

板端回复后，PC 重新计算指标、组织对比图并保存 pc_fp32.npy、fpga.npy 和 report.json。若给出随机数，还会运行 PC 端独立复核。任何一步异常均会把任务标记为 error 并记录异常类型和说明。

2.3.2.10 PC 浮点参考推理

get_model 按场景加载原始模型参数和 config，通过 strict=True 检查参数结构匹配，设置 eval 模式并关闭梯度。首次加载后先用 64 条射线预热，再将模型和设备保存在 MODEL_CACHE。后续同场景查询复用模型，避免把重复加载开销混入每次参考计算。 图 2-22 给出浮点参考从模型装载到完整空间谱的流程。表 2-24 给出 FP32 计算函数的关键输入输出。

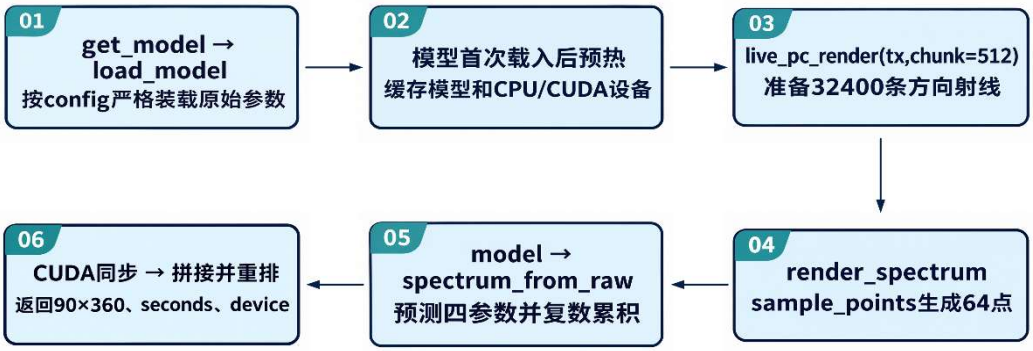


图 2-22 浮点参考从模型装载到完整空间谱的流程

图 2-22 展示 PC 浮点参考的计算过程。模型首次装载后进行预热并缓存，正式推理按射线分块执行，结果整理和传回 CPU 发生在对应计时阶段之后。

表 2-24 FP32 计算函数的关键输入输出

函数或方法	关键输入	返回值或主要写入
load_model	原始参数文件、网络配置和设备	NeRF²模型与配置字典
sample_points	每条射线原点、方向，0～5m、64 点	采样位置[...,64,3]与距离 t
render_spectrum	模型及一批 Tx、原点、方向	该批射线的预测幅值
spectrum_from_raw	四个网络参数及采样几何	复数路径贡献累积后的幅值
live_pc_render	三维 Tx，默认每批 512 条射线	90×360 空间谱、计时与设备

浮点参考按原始 NeRF²表达式处理衰减幅度、衰减相位、信号幅度和信号相位，并累加 64 点复数贡献。

2.3.2.11 网络接口与板端任务调度

电脑端默认在 127.0.0.1:8310 提供网页与协调 API，板端在 0.0.0.0:8311 提供硬件业务 API。normalize_board_url 强制使用 8311，拒绝把 PC 端口或 Jupyter 端口当作推理服务。json_request 根据当前 SCENE_ID 自动为板端请求补充 /scene/<id>/前缀。场景 ID 必须在注册表中存在。 表 2-25 给出板端线程与请求入口的接口。

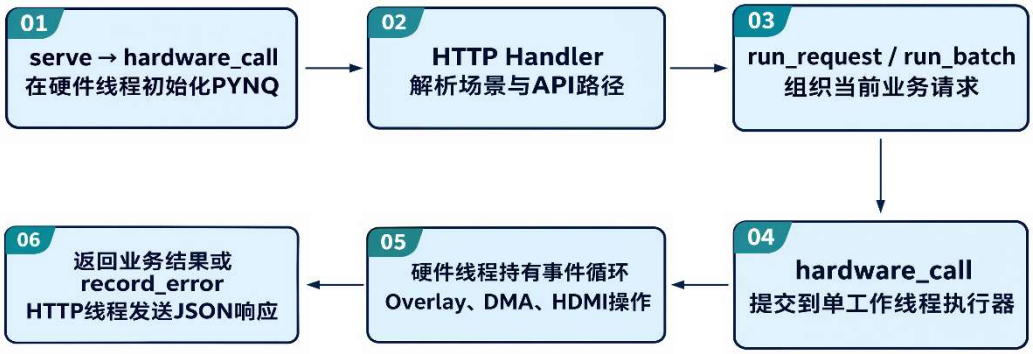


图 2-23 HTTP 业务线程与单一 PYNQ 硬件线程的关系

图 2-23 中，HTTP 线程接收并组织请求，硬件线程顺序执行 Overlay、DMA 和 HDMI 操作。集中管理硬件状态使多个业务请求能够共享同一套板端计算资源。

表 2-25 板端线程与请求入口的接口

函数或方法	关键输入	返回值或主要写入
serve	绑定地址与 8311 端口	建立 ThreadingHTTPServer 并持续服务
_hardware_thread_initializer	执行器初始化回调	为唯一硬件线程建立 asyncio 事件循环
hardware_call	待执行函数和参数	在硬件线程执行并返回函数结果
initialize_pynq_device	是否强制重新发现设备	PYNQ 活动 Device 对象
hardware	当前硬件线程	复用或建立 UniversalHardware 实例

板端使用 max_workers=1 的 HARDWARE_EXECUTOR 管理所有 Overlay、DMA 与 HDMI 操作，HTTP 线程和批量任务线程只提交工作。此设计保证 PYNQ 事件循环与硬件句柄归同一线程拥有，避免多线程同时修改 PL 和 DMA 状态。它不限制普通状态查询和文件读取必须串行完成。

2.3.2.12 板端推理请求处理

run_request 首先解析 mode。measured 模式核对测试 ID 并从本地目录取得 Tx 和 GT。free 模式把 tx 转为 float32 并检查三维形状、有限数和场景边界，GT 置空。PC 参考通过压缩 NPZ 传入，decode_array 要求其形状为 90×360 且全部为有限数。 图 2-24 给出板端 run_request 的业务处理顺序。表 2-26 给出板端单次请求的主要变量。

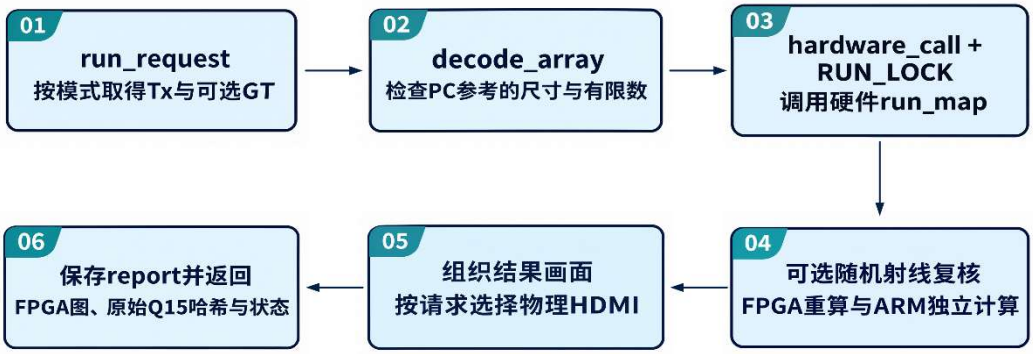


图 2-24 板端 run_request 的业务处理顺序

如图 2-24 所示，板端先校验输入，再调用硬件生成空间谱，并根据请求组织复核和显示。PC 参考结果用于比较和画面组织。

表 2-26 板端单次请求的主要变量

函数或方法	关键输入	返回值或主要写入
run_request	场景、模式、PC 图及显示复核选项	ok、scene_id、run_id、report、FPGA 图与哈希
decode_array	pc_map_npz_base64	float64[90,360]，只用于对比和画面
set_state	阶段、完成行数及时间	更新场景 STATE，供/api/active 读取
UniversalHardware.run_map	Tx、可选 PC/GT 及进度回调	完整空间谱和运行报告

run_request 在硬件线程中以 RUN_LOCK 保护完整业务事务：先做主推理，再做可选随机复核，最后组织显示。运行目录保存报告与 HDMI 画面，回复中的 fpga_map_npz_base64 是由原始 Q15 换算得到的空间谱。

2.3.2.13 场景配置与硬件状态管理

UniversalHardware 维护 overlay、runner、scene、cache 和 pl_owner 等状态。configure 遇到当前场景且推理 Overlay 仍有效时直接复用。首次运行、场景变化、异常恢复或 HDMI 占用 PL 后，则重新建立所需配置。场景切换通常只更新模型与缓存，HDMI 切换后则必须重新下载推理位流。 表 2-27 给出配置相关方法的输入输出。图 2-25 给出场景配置与模型缓存建立流程。

表 2-27 配置相关方法的输入输出

函数或方法	关键输入	返回值或主要写入
configure	场景 ID、是否强制配置	配置结果。建立 runner、cache 和 model_load
_program_accelerator	当前 PL 拥有状态与通用 bit 路径	新的 Overlay 对象并更新 pl_owner
selftest_cache_sentinel	场景目录及 512 条预存核对数据	一致则继续。不一致抛出异常

函数或方法	关键输入	返回值或主要写入
mark_hdmi_overlay_active	HDMI Overlay 已加载的事实	清除推理句柄，pl_owner 置 base_hdmi
recover_overlay	发生故障的现有驱动状态	重载同一推理位流并使旧配置失效

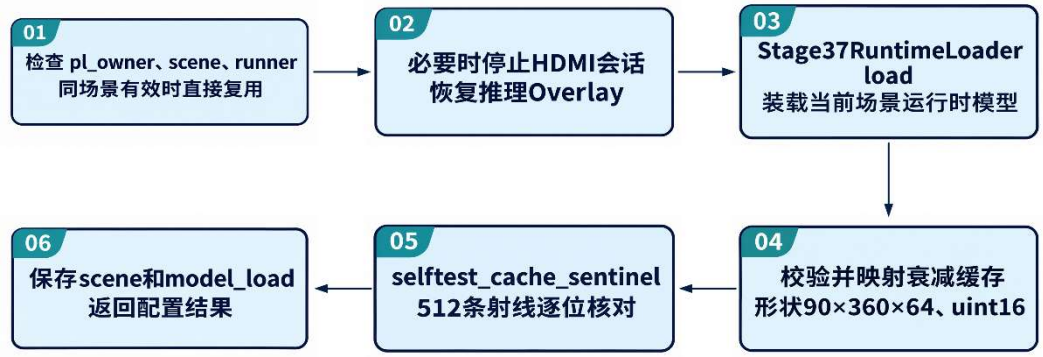


图 2-25 场景配置与模型缓存建立流程

图 2-25 展示场景资源的配置过程。同场景资源有效时直接复用，重新配置时装载运行时模型与衰减缓存，经缓存核对后记录当前状态。

衰减缓存先检查文件哈希，再以 mmap 只读映射，要求 shape=(90,360,64)、dtype=uint16。512 条射线核对把同一场景模型在 PL 计算的中间量与已保存缓存比较，确认两者配套后才标记 scene 有效。

2.3.2.14 运行时模型装载

Stage37RuntimeLoader 把 NPZ 转换为硬件流协议。build_load_packet 读取权重、索引、偏置、描述符、移位量和七组渲染表，为每项生成地址字和数据字。操作码 8 的命令头给出记录数和模型 ID。大权重字分成 16 个 64 位段写入，与硬件 16 个存储体对应。

图 2-26 展示模型装载和提交过程。分块传输保持地址与数据记录完整，全部记录写入后再提交模型，以建立后续推理所需的有效状态。

表 2-28 给出模型装载器的关键方法。

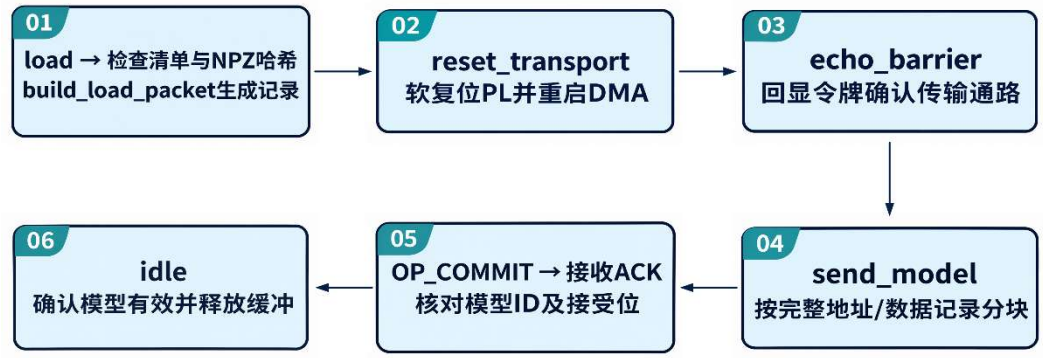


图 2-26 运行时模型装载与提交确认流程

图 2-26 展示模型装载和提交过程。分块传输保持地址与数据记录完整，全

部记录写入后再提交模型，以建立后续推理所需的有效状态。

表 2-28 模型装载器的关键方法

函数或方法	关键输入	返回值或主要写入
build_load_packet	运行时 NPZ 路径	uint64 packet、model_id、records
reset_transport	GPIO 和 RawDMA 对象	复位状态。异常时附诊断
_echo_barrier	回显缓冲、模型令牌与超时	核对收到的 64 位令牌
_send_model	已 flush 的 CMA 缓冲与字数	实际发送块数
load	NPZ 及配套 JSON 清单	模型 ID、记录数、耗时、状态和恢复记录

首块最多 1023 个 64 位字，即 1 个头字和 511 对地址数据，后续块最多 1024 字，即 512 对完整记录。底层输入 TLAST 不定义模型包边界，记录总数才是逻辑边界。全部写完后发送操作码 9 提交，检查 ACK 中的模型 ID、接受位和 model_valid 状态，未通过时不进入正式推理。

2.3.2.15 位置编码与射线数据打包

quantize_tx 对三维 Tx 计算原值、10 组正弦和余弦，组成 63 维位置编码，再按当前场景冻结的输入尺度四舍五入并限制到 INT8 范围。方向编码已在离线常量文件中保存，Tx 编码在每次查询时填入。三个空间分量各形成 21 个编码值，合计 63 维。补齐到 64 字节只为传输对齐，不新增神经网络输入维度。 图 2-27 给出 Tx 编码、固定方向与衰减缓存的记录组织。表 2-29 给出一对射线的缓存推理记录。

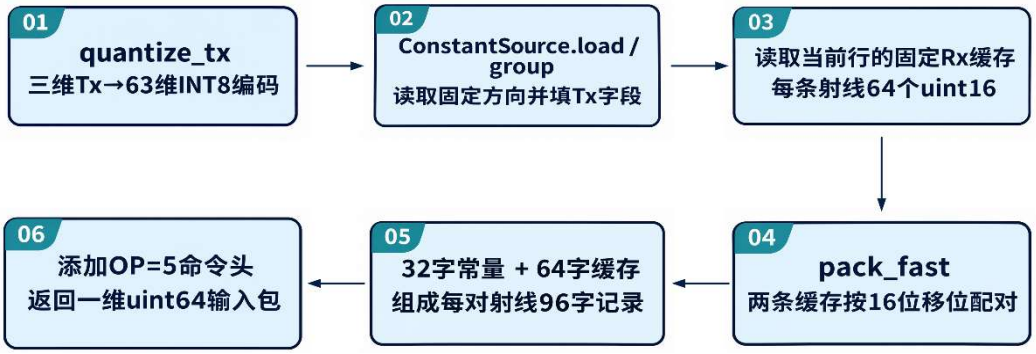


图 2-27 Tx 编码、固定方向与衰减缓存的记录组织

图 2-27 展示一对射线的输入记录。方向和发射端编码构成常量部分，64 点衰减缓存为射线渲染提供固定接收端中间量，后续硬件计算生成空间谱结果。

表 2-29 一对射线的缓存推理记录

位置	长度	内容
常量字 0～7 / 8～15	各 8 个 uint64	射线 0 方向编码 / 当前 Tx 编码
常量字 16～23 / 24～31	各 8 个 uint64	射线 1 方向编码 / 当前 Tx 编码

位置	长度	内容
缓存字 32～95	64 个 uint64	每采样点低 16 位射线 0 缓存，高 16 位射线 1 缓存
整个数据包首字	1 个 uint64	操作码 5 与本包射线总数

pack_fast 把偶数条射线两两配对，每对在 32 个常量字后追加 64 个缓存字。缓存配对及数据包字数按下式计算：

$$cw = cache_0 \mid (cache_1 \ll 16)$$
$$N_{words} = 1 + \frac{nrays}{2} \times 96$$

式中，|表示按位或，≪表示左移，nrays 为本包的偶数射线总数，N_words 为 uint64 字数。打包只组织输入记录，不在 CPU 生成最终 FPGA 空间谱。

2.3.2.16 完整空间谱调度与双缓冲

ConstantSource.load 核对基础常量包的文件长度和哈希，填入本次 Tx 编码。group 将最多 4 个相邻基础包拼接。18 个每包 5 行的文件因此组织为 20、20、20、20、10 行的 5 批生产传输。_run_map_once 按行号将每批结果写入同一个 90×360 的 uint32 数组。 表 2-30 给出完整图驱动的主要输入输出。

表 2-30 完整图驱动的主要输入输出

函数或方法	关键输入	返回值或主要写入
ConstantSource.load	基础包索引、63 维 Tx 编码	该包常量数组与清单条目
ConstantSource.group	0～4 生产批次索引与 Tx 编码	拼接常量及 first_row、row_count、ray_count
FastRunner.prepare	缓冲编号、输入包和射线数	slot、nwords、nrays 三元组
_run_map_once	场景、Tx 及可选比较对象	报告、换算后空间谱、原始 uint32 结果

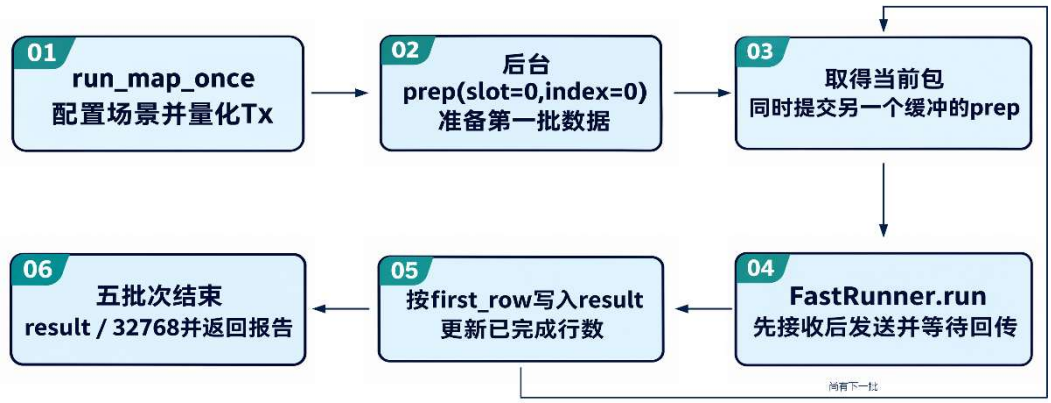


图 2-28 整图五批次调度与双发送缓冲的函数流程

图 2-28 将完整方向网格分为 20、20、20、20 和 10 行五批。双发送缓冲使

下一批 CPU 数据准备与当前批 DMA 传输及 PL 计算重叠，从而减少批次之间的等待。

驱动分配两个发送缓冲，后台 prep 线程准备下一批时，当前批在 DMA 和 PL 执行。两种工作在时间上重叠，减少 CPU 准备造成的空隙。每次只能有当前有效批次进入同一加速器。接收缓冲按当前批次复用，18 个基础包数、5 次传输和硬件双射线引擎是三个不同层次。

2.3.2.17 DMA 传输与缓存一致性

FastRunner 使用 PYNQ allocate 建立连续物理内存缓冲。ARM 把记录复制到发送缓冲后调用 flush，使 DDR 中的内容与 CPU 缓存一致。run 先检查核心空闲且模型有效，安排 S2MM 接收，再启动 MM2S 发送。发送和接收完成后重新检查状态，从每个 64 位回传字的低 32 位提取原始幅值。 表 2-31 给出 RawDMA 与 FastRunner 的关键接口。



图 2-29 DMA 收发与缓存一致性处理顺序

图 2-29 展示 DMA 收发与缓存维护顺序。发送前执行 flush，使 DDR 中的内容与 CPU 写入一致。接收后执行 invalidate，使 CPU 读取 DMA 写回的数据。

表 2-31 RawDMA 与 FastRunner 的关键接口

函数或方法	关键输入	返回值或主要写入
RawDMA.arm_recv	接收物理地址、目标字节数	写 S2MM 地址和长度并读回状态
send_preflushed	已刷新的发送缓冲、长度、超时与偏移	写 MM2S 并等待空闲或错误
wait_recv	接收缓冲与超时	等待 S2MM 完成，再 invalidate
FastRunner.run	准备结果三元组	uint32 射线幅值、传输段秒数
RawDMA.snapshot	当前 DMA 寄存器	故障诊断用寄存器快照

RawDMA 通过 MMIO 访问控制寄存器，错误位和超时均产生异常。发送前刷新与接收后失效的对象不同，不能互相替代。完整图结果转换为浮点显示值时直接除以 32768，保留原始 Q15 作为数值来源。

2.3.2.18 ARM 整数参考计算

Stage37ARMReference 读取运行时 NPZ，不调用 PL 计算。run_indices 根据射线索引找到基础常量包、记录中的 lane 和 64 点衰减缓存，再分别执行 signal

和 renderer。该路径用于少量射线的独立整数复核，也可作为完整 ARM 测量程序的计算基础。 图 2-30 给出 ARM 参考对一条选定射线的整数计算流程。表 2-32 给出 ARM 参考的下层函数接口。

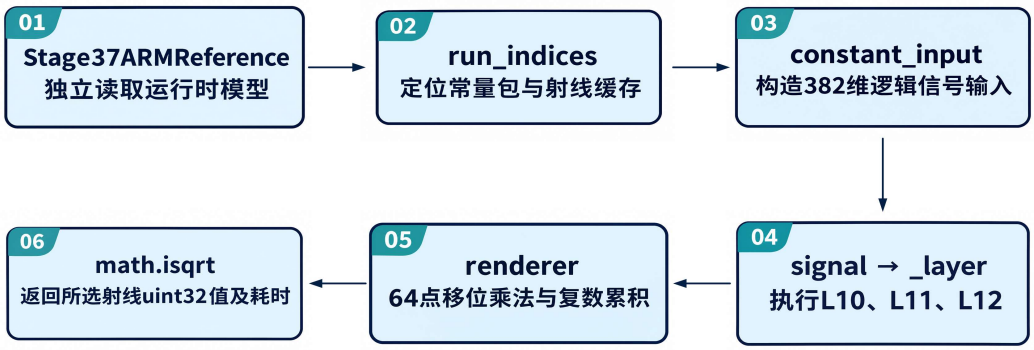


图 2-30 ARM 参考对一条选定射线的整数计算流程

图 2-30 中，ARM 参考路径独立读取运行时模型、常量和衰减缓存，按选中射线执行整数网络计算与 64 点渲染，并返回整数结果及耗时。该路径为板端输出提供数值复核依据。

表 2-32 ARM 参考的下层函数接口

函数或方法	关键输入	返回值或主要写入
_constant_input	一对射线的 32 字常量、lane=0 或 1	382 维 INT8 逻辑信号输入
_layer	层号、激活数组、是否 ReLU	该层 INT8 输出。按描述符稀疏列计算
signal	常量记录和射线编号	两个 INT8 信号参数
renderer	两个信号参数、64 个 uint16 缓存	单条射线 uint32 幅值
run_indices	常量源、缓存、Tx 编码、选定索引	所选射线 uint32 数组及总秒数

382 维逻辑输入由 256 个零共享特征、63 维方向编码和 63 维 Tx 编码组成。_layer 依次执行信号 L10、L11 和 L12，按稀疏索引取输入并累加权重，再移位、加偏置、激活和饱和。renderer 按 64 点进行 Q15 相位、幅度与衰减递推，最后使用 math.isqrt 得到整数幅值。

2.3.2.19 随机射线抽样复核

复核模块把随机数、场景 ID、Tx 的 float32 规范表示、射线数量和模型位流身份序列化成 canonical_input。challenge_indices 以 SHA-256 计数器生成 32 位数，拒绝落在不能均匀映射的尾部区间，再映射到 0～32399 并去重排序。相同输入产生相同索引，使 PC 和板端能够分别实现并核对同一选择过程。 图 2-31 给出公开选点、板端重算与 PC 独立复核流程。表 2-33 列出主要接口。

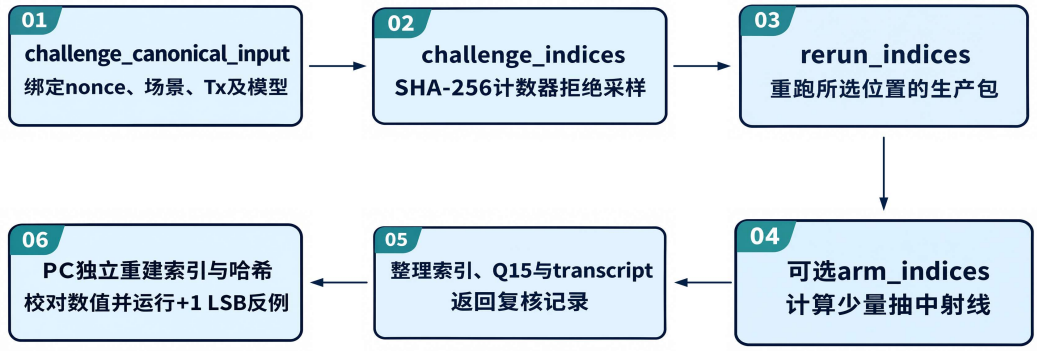


图 2-31 公开选点、板端重算与 PC 独立复核流程

图 2-31 中，公开规则根据输入生成抽查索引，FPGA 重跑对应生产包，ARM 计算抽中的射线。PC 独立核对索引、哈希及结果，并通过加 1 LSB 的反例检查比较过程。

表 2-33 随机复核主要函数的接口

函数或方法	关键输入	返回值或主要写入
challenge_canonical_input	随机数、场景、tx、数量及身份	规范化 JSON 字符串
challenge_indices	规范字符串、8/16/32 条	排序去重的 int64 射线索引
rerun_indices	Tx、所选位置、完整图原始数组	重算 Q15、完整图对应值及实际重跑工作量
arm_indices	Tx 及所选位置	独立 ARM 整数值与耗时
independently_verify_random_challenge	板端记录、原随机数/Tx/数量及 ARM 要求	PC 独立核对结果或明确异常

FPGA 复核会重跑抽中位置所在的 20 行或 10 行生产包，因此实际计算射线数可能远多于 8、16 或 32。ARM 只计算抽中的射线。电脑端还对首条重算值临时加 1 LSB，检查比较器能否发现差异。

2.3.2.20 测试集批量运行与断点续跑

批量模块用于对已登记场景进行完整评估。run_batch 从完整测试目录按顺序读取每个 Tx 和 GT，逐幅调用相同的 run_map 驱动。每幅先保存原始 Q15 的 NPZ，再追加 records.jsonl 并 flush/fsync，最后更新 status.json。这样只有已经完整提交的样本才计入续跑进度。 表 2-34 给出批量模块的关键函数。图 2-32 给出完整测试集的逐图保存与续跑流程。

表 2-34 批量模块的关键函数

函数或方法	关键输入	返回值或主要写入
start_batch	场景与是否重新开始	创建后台线程或返回现有状态
archive_batch_if_needed	当前模型、位流、目录身份	归档不兼容旧结果，写 identity.json

函数或方法	关键输入	返回值或主要写入
read_records	JSONL 及对应原始图文件	连续且通过文件哈希的已完成前缀
run_batch	完整测试目录与运行配置	maps、records、status 和最终 result.json
batch_status	当前场景	未开始、运行、暂停、完成或错误状态

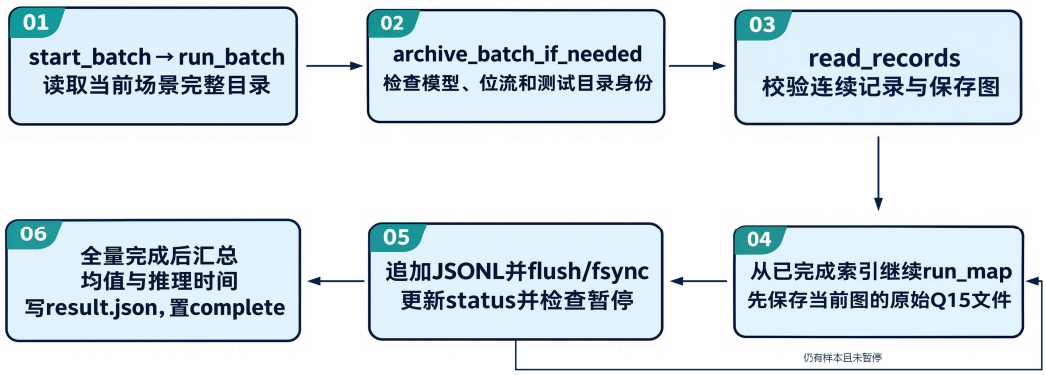


图 2-32 完整测试集的逐图保存与续跑流程

图2-32展示测试集任务的保存与恢复过程。系统先保存单幅原始Q15结果，再提交记录并更新状态。暂停在当前样本提交后生效，恢复时从已提交的连续样本之后继续。

read_records 遇到尾部 JSON 不完整、样本序号不连续或图文件校验失败时，只保留此前有效的连续记录。暂停请求在当前样本提交后停止。恢复从已提交数量继续。四场景样本数不同，执行数量来自注册表和实际目录。

2.3.2.21 历史结果读取与同步

carousel_frame 随机选择测试 ID，读取 GT，并分别取得 PC 和 FPGA 保存结果。PC 缓存缺失时可以现场计算后保存。FPGA 本地缓存缺失时请求板端 map_response。后者要求板端已存在完整测试摘要，并核对模型、位流、测试目录及原始图文件，才返回换算后的空间谱。 图 2-33 给出保存结果浏览中的 PC 与 FPGA 缓存读取流程。表 2-35 给出结果浏览与下载函数的接口。

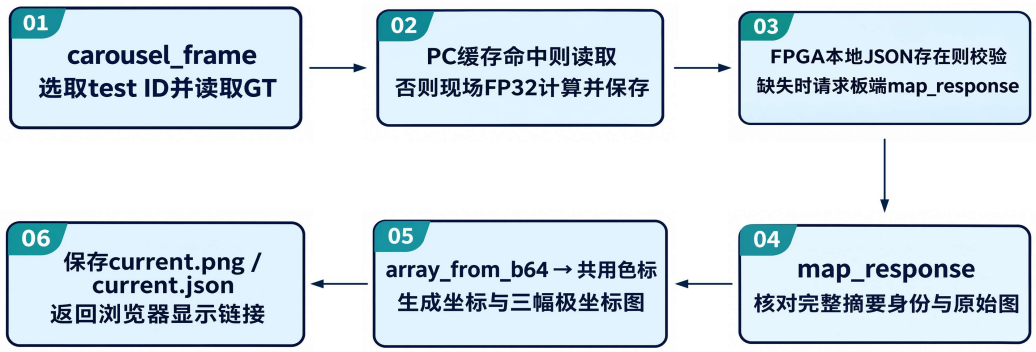


图 2-33 保存结果浏览中的 PC 与 FPGA 缓存读取流程

图 2-33 展示历史结果的读取过程。轮播从已有文件读取空间谱，sync_evidence 可预先下载当前场景的 FPGA 结果，以便后续浏览。

表 2-35 结果浏览与下载函数的接口

函数或方法	关键输入	返回值或主要写入
carousel_frame	板端 URL 及可选测试 ID	坐标图、GT/PC/FPGA 对比图和来源说明
map_response	场景与测试 ID	已保存 FPGA 空间谱、原始 Q15 哈希与来源
sync_evidence	场景模块、任务 ID 和板端地址	下载本场景所有 FPGA 图到 PC 缓存
validate_saved_map	下载 JSON 及预期样本 ID	核对身份与传输哈希，补记传输哈希

总体统计摘要、下载的 FPGA 图缓存和板端原始 Q15 文件分别用于统计展示、本地浏览及保存原始计算结果。系统按其各自的存储位置读取和管理这些数据。

2.3.2.22 空间谱显示与 HDMI 切换

save_comparison 在 PC 侧用 Matplotlib 生成对比图，render_polar_rgb 将 90×360 方向数据映射为极坐标显示。组图使用共同的颜色范围便于观察，不改变原始数组及其指标。测试位置显示 GT、PC、FPGA，任意 Tx 显示 PC 和 FPGA 并注明没有对应 GT。 表 2-36 给出图像显示函数的输入输出。

物理 HDMI 使用 PYNQ BaseOverlay，替换当前推理位流。服务在显示后调用 mark_hdmi_overlay_active 使旧推理句柄失效。下一次硬件请求先停止视频，再重新装载同一推理 Overlay 及当前场景数据。普通浏览器绘图无需进行这次位流切换。

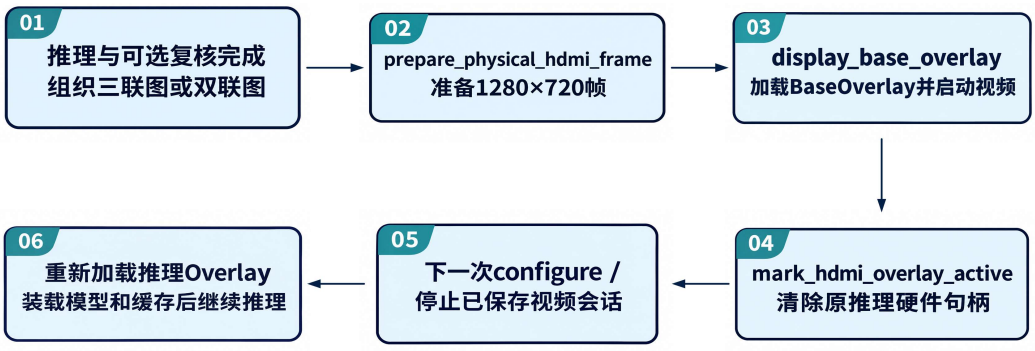


图 2-34 HDMI 结果显示与下一次推理恢复流程

图 2-34 展示结果显示与后续推理的衔接。浏览器通过软件绘图显示结果，物理 HDMI 加载 BaseOverlay，下一次推理前再恢复推理 Overlay 及场景资源。

表 2-36 图像显示函数的输入输出

函数或方法	关键输入	返回值或主要写入
save_comparison	保存路径与两幅或三幅空间谱	写 comparison.png
compose_clear_three_way	GT、PC、FPGA、1280×720 画布与样式	RGB 显示帧及布局信息
free_compare_frame	两幅预测及画布尺寸	带 GT N/A 说明的 RGB 帧
prepare_physical_hdmi_frame	RGB 帧与显示比例	适配后的物理帧和比例说明
_display_base_overlay	1280×720 RGB 物理帧	启动 HDMI 输出并返回驱动说明

2.3.2.23 Notebook 引导与服务管理

Notebook 用于初始化和管理板端程序，服务启动后运行于独立进程。第 1 单元确定绝对路径、固定协议模块导入并校验包。第 2 单元调用 start，连接已有同版本服务或启动监督器。监督器创建 HTTP 工作进程并记录其 PID 和退出状态，在异常退出后重启工作进程。

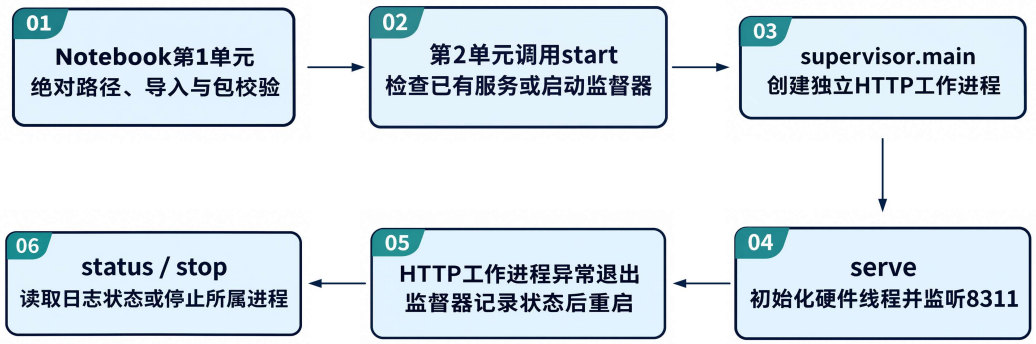


图 2-35 Notebook 启动与受监督服务的生命周期

图 2-35 中，Notebook 启动的服务运行于独立进程。监督器记录工作进程状态，并在其异常退出后重启。板卡重新上电后需要重新启动服务。

表 2-37 Notebook 六个单元格的职责

单元格	调用入口	运行目的
1	prepare_stage39_imports 与包校验器	建立正确路径与协议模块，验证源内容
2	stage39_service_launcher.start	启动或复用 8311 板端服务
3	run_selftests	四场景各运行一幅完整图核对
4	run_benchmarks(repeats=50)	四场景正式时延与输出稳定性测量
5	export_evidence	导出当前已保存的板端结果
6	stage39_service_launcher.status	查看服务、监督器与日志状态

日常演示在服务已正常运行时直接连接 App。板卡断电重启后通常执行前两个单元恢复服务，不需为重新连接而重复全部基准测试。服务可在 Notebook 页面关闭或内核重启后继续运行，但不会跨板卡断电存活。Notebook 校验以单元类型和源文本的规范化哈希为依据，忽略执行计数、输出与 Jupyter 运行元数据，因此正常执行与保存不会使源校验失效。

2.3.2.24 异常诊断与恢复

错误处理按输入、网络、服务和硬件层分别执行。非法 Tx、未知场景、模型身份不符等问题应先修正配置。连接中断可等待受监督服务恢复并重试。DMA 或 PL 异常则需要重新配置硬件。报告中的恢复次数保留在运行结果内，便于区分一次正常运行和经恢复完成的运行。 图 2-36 给出软件错误定位与硬件恢复的分层关系。表 2-38 给出诊断与恢复函数的关键输出。

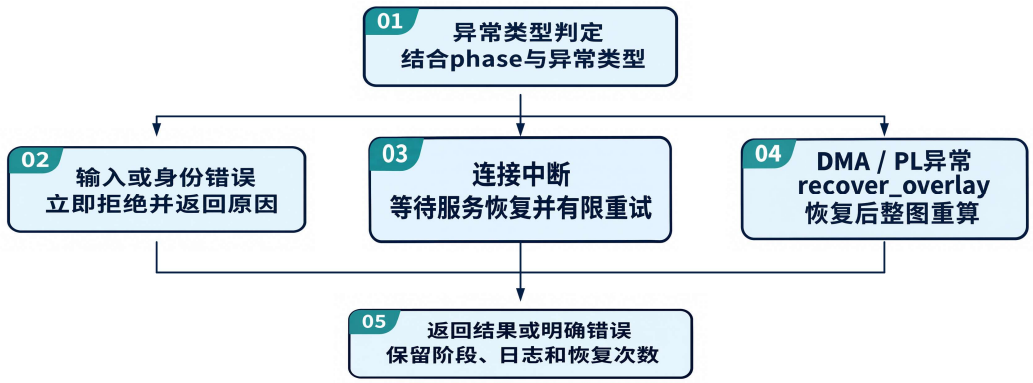


图 2-36 软件错误定位与硬件恢复的分层关系

输入或身份错误直接返回原因，连接中断进入有限重试，DMA 或 PL 异常进入硬件恢复和整图重算。处理结果保留阶段、日志和恢复次数。

表 2-38 诊断与恢复函数的关键输出

函数或方法	关键输入	返回值或主要写入
record_error	异常、场景和阶段	last_error.json、traceback 末尾与 STATE 错误态
wait_for_board_service	板端 URL、超时、空闲要求	服务重新就绪或明确超时
board_run_resilient / board_get_resilient	请求和有限尝试次数	恢复后的板端响应或连接异常
UniversalHardware.run_map	完整图任务	硬件失败时最多一次整图重算，记录恢复事件
status / stop（启动器）	端口与服务 PID 记录	返回状态。仅停止已识别的本服务进程

装载器首先通过软复位重试，必要时由 configure 或 run_map 重新加载推理 Overlay，并在复位硬件后释放旧 DMA 缓冲。处理结果通过异常和运行记录返回。

2.3.2.25 HTTP 接口与数据格式

接口中的 scene 由/scene/<id>/前缀确定。表 2-39 采用省略场景前缀的相对路径。网页链接由 PC 路由统一补齐。PC 的/api/run 是异步任务入口，板端同名接口则在本次硬件业务结束后回复。

表 2-39 主要 HTTP 业务接口

服务与接口	请求或参数	返回或作用
PC GET /api/scenes	无	场景注册表
PC GET /api/catalog	当前场景	目录、Tx 范围、Rx 和参考信息
PC POST /api/run	查询 payload	202 及 job_id
PC GET /api/job/<id>	job_id	status、phase、progress 及完成结果
PC POST /api/pc-preview	测试 ID 或 free Tx	仅 PC 参考图，不作为 FPGA 结果
PC POST /api/sync-evidence	board_url	异步下载本场景保存的 FPGA 图
板端 GET /api/health	当前场景	服务版本、输出模式与模型身份
板端 POST /api/run	mode、Tx/ID、PC 图和选项	硬件报告、FPGA 图及原始哈希
板端 POST /api/all-test/start	restart 布尔值	启动或继续批量任务
板端 GET /api/all-test/status	当前场景	已完成数量、状态和总量
板端 GET /api/all-test/result	当前场景	完整逐图摘要及统计
板端 GET /api/all-test/map	spectrum_id	某幅已保存的 FPGA 图
板端 GET /api/last-error	无	最近异常阶段、类型和日志末尾

数值数组在 HTTP 中使用压缩 NPZ 再进行 Base64 编码。解码端关闭 allow_pickle 并检查尺寸和有限数。文件读取接口将路径 resolve 后限制在当前场景资源或结果目录内，防止相对路径越出其数据根目录。 表 2-40 给出主要状态与结果文件。

2.3.2.26 结果存储与计时定义

表 2-40 主要状态与结果文件

文件或变量	保存内容	重启后的性质
PC JOBS / 板端 STATE	后台任务状态、阶段与进度	内存状态，不自动跨进程恢复
board live/<run_id>/report.json	一次请求的模式、模型、时间和结果说明	磁盘文件，可保留查阅
board all_test/maps/*_q15.npz	完整测试图的原始 uint32 Q15	后续取图与数值复核的依据
records.jsonl / status.json	逐图提交记录与当前进度	支持按完整记录续跑

文件或变量	保存内容	重启后的性质
all_test/result.json	完整逐图摘要与均值、时间统计	全量完成后才生成
PC carousel_fpga_cache/	下载的 FPGA 图与传输哈希	支持本地保存结果浏览
PC carousel_pc_cache/	已计算的 FP32 图	减少重复参考计算

atomic 和 atomic_json 先写入同目录临时文件，再通过 os.replace 替换目标文件。批量 JSONL 记录在追加后执行 flush 和 fsync，以提高已提交数据在中断后的可恢复性。表 2-41 列出主要计时字段的覆盖范围。

表 2-41 主要计时字段的覆盖范围

字段或返回值	覆盖部分	不应混入的含义
live_pc_render 的 seconds	分批 FP32 前向与 CUDA 同步	不含模型加载、预热、输入上卡及最后拷回 CPU
full_map_cached_pipeline_seconds	五批数据准备、流水调度及 DMA/PL 完成	不含前置模型配置及后续 HTTP、绘图、HDMI
fpga_stream_seconds_sum	各批发送接收等待段求和	不等于纯 PL 时钟周期计数
run_entry_to_result_ready_seconds	板端业务入口至本次结果就绪	可能含复核、显示和配置
coordinated_wall_seconds_to_board_result	PC 发起任务到板端结果返回的协调总过程	包含 PC 参考和通信。不等于 FPGA 核心时延

2.3.2.27 软件模块与源文件对应关系

表 2-42 给出软件模块与现有源文件的对应关系。文件以 PC 包、板端包或复现包为根目录，便于按相同逻辑查阅源码。软件流程图中的抽象处理步骤可由这里的真实函数继续向下定位。通用读写、哈希和状态辅助函数在表后说明。

表 2-42 模块到源文件及函数的对应

模块	现有源文件	主要入口
PC 总路由	pc_app/stage39_pc_server.py	context、Handler.choose、main
PC 任务与参考	pc_app/stage39_scene_core.py	start_job、run_one_job、live_pc_render
浏览器	pc_app/stage39_static/app.js	changeScene、startMeasured、startFree、poll
板端业务	jupyter/stage39_board_service.py	hardware_call、run_request、run_batch

模块	现有源文件	主要入口
完整图驱动	jupyter/pynq_stage38_raw.py	UniversalHardware、ConstantSource、FastRunner
模型装载	jupyter/stage37_runtime_loader.py	build_load_packet、Stage37RuntimeLoader.load
DMA 底层	jupyter/pynq_load_weights.py	RawDMA.arm_recv、send_preflushed、wait_recv
ARM 整数参考	jupyter/stage37_arm_reference.py	signal、_layer、renderer、run_indices
FP32 黄金参考	software/stage11/reference.py	load_model、render_spectrum、spectrum_from_raw
图像与 HDMI	jupyter/stage28_exact_common.py 等	render_polar_rgb、prepare_physical_hdmi_frame
服务启动监督	jupyter/stage39_service_launcher.py 等	start、status、stop、supervisor.main
离线流水	03_Package_Builder/run_model_pipeline.py	factorize、calibrate_ptq_qat、build_runtime

PC 端 get_model 和 MODEL_CACHE 只缓存当前场景的参考模型。板端 hardware 缓存唯一硬件管理对象。状态、缓存与磁盘文件的生命周期不同：文件仍在不表示 PL 仍保持已装载模型，Python 对象仍在也不表示 HDMI 切换后的旧 Overlay 有效。软件通过场景、模型身份和 pl_owner 共同维护这些关系。 表 2-43 给出辅助函数的输入输出与作用。

2.3.2.28 模块间数据约定

表 2-43 辅助函数的输入输出与作用

函数或方法	关键输入	返回值或主要写入
sha / sha256_file / _sha256	文件路径	SHA-256 字符串，用于模型、缓存和结果一致性检查
array_to_b64 / array_b64	90×360 数值数组	以 value 为键的 float32 压缩 NPZ，再编码为 ASCII
atomic / atomic_json	目标路径、JSON 兼容结构	临时文件替换后的完整 JSON 文件
set_state / state / update_job	场景或 job_id、需要更新的字段	线程锁保护下的状态更新或副本
catalogs / load_catalog / load_all_test_catalog	场景目录或已配置目录	样本列表及按 spectrum_id 建立的索引

函数或方法	关键输入	返回值或主要写入
metrics / comparison_metrics	reference、prediction 同尺寸数组	SSIM、PSNR、RMSE/MAE 等指标
decode_status / read_status	GPIO 状态字或 MMIO 对象	错误、空闲、model_valid 及剩余数量
_free_buffer / close	CMA 缓冲或驱动对象	停止使用后释放资源并清空句柄

模块间统一使用米作为 Tx 与 Rx 坐标单位，统一按 90 行仰角、360 列方位角组织空间谱。展平时将行号乘 360 后加列号，得到 ray_index。训练及 PC 参考使用浮点参数。传输记录使用 uint64。FPGA 结果数组使用 uint32 并按 32768 恢复 Q15 数值。固定衰减缓存使用 uint16 打包两个 INT8 参数。

相同显示布局可以服务测试样本和任意 Tx，但指标依据必须与数据来源一致，有 GT 时评价预测与 GT，没有 GT 时仅比较 PC 和 FPGA。保存结果轮播用于再次查看已生成的空间谱，现场查询用于重新执行模型。软件在接口、目录和结果说明中保留这种区别。

2.4 设计原理与软硬件任务划分

2.4.1 空间谱生成原理

系统以接收端 Rx 的位置与姿态、90×360 方向网格和每个方向的 64 个采样点确定场景几何。对方向 d，以接收机位置 r 为射线原点，按采样距离构造空间点。发射机坐标 tx 作为本次查询输入，位置编码将三维量扩展为 63 维特征，使网络能够表达传播场的空间变化。图 2-1 展示离线准备与在线推理的关系，图 2-16 展示部署参数的生成过程。

$$p_i = r + t_i d,\ i = 0, 1, \ldots, 63$$

式中 i 为采样点序号，t_i 为第 i 个采样距离。浮点参考采用 NeRF²模型预测采样位置的衰减幅度、衰减相位、信号幅度和信号相位，再将 64 个采样点的复数贡献累加为一个方向像素。该像素表示复信号累加后的相对幅值。

结构适配使衰减支路依赖采样位置，使信号支路依赖方向和 Tx。固定 Rx 及方向网格后，采样位置与衰减中间量可提前计算。每次新 Tx 查询再计算 32400 个方向的信号参数，并对每条射线的 64 个采样点执行渲染，从而复用固定部分的网络计算。

固定零掩码对原网络施加结构约束，受约束微调与硬件精确 QAT 用于恢复该结构下的预测质量。冻结整数模型后，再删除整组全零列以压缩存储，并保持对应整数结果一致。四场景中，s23 直接进行结构适配，其余场景各执行 3000 步微调。

2.4.2 计算任务与数据依赖

软硬件任务划分依据计算重复度和数据依赖确定。发射端编码每次生成 63 个 INT8 值，HTTP 解析和文件管理包含较多控制分支，适合由 PC 和 ARM 处理。神经网络矩阵向量运算及定点射线累积需要在完整方向网格上重复执行，适

合映射为 PL 专用数据通路。模型训练由 PC 完成。

表 2-44 软硬件任务划分及依据

任务	执行位置	划分依据	交换的数据
训练、结构微调、QAT	PC/GPU 离线	需浮点参数更新和训练框架。不属于每次查询	运行时 NPZ、定标清单及缓存
交互、FP32 参考、绘图	PC 在线	界面与参考计算便于操作和独立比较	场景、Tx、可选参考图与结果文件
场景控制、Tx 编码、DMA	ARM/PS	控制分支多，需调用 Linux 与 PYNQ 接口	63 维编码、uint64 输入流、uint32 结果
固定 Rx 中间量保存	DDR 与离线文件	每场景缓存约 4.15 MB，需跨 Tx 复用	90×360×64 个 uint16 元素
稀疏整数网络及射线渲染	PL	规则重复、可并行乘加，需固定数值顺序	每方向一个原始 Q15 幅值
少量射线独立复核	ARM/NumPy	形成独立实现用于发现数据或算术不一致	8、16 或 32 条所选整数输出

表 2-44 总结计算与验证任务的分工。PL 执行完整空间谱推理，ARM 对选中射线进行整数复核。固定接收端缓存由文件系统和 DDR 提供，模型参数驻留片上存储，并行运算由 DSP 资源支持。

2.4.3 接口协议与状态管理

控制和数据接口分别见图 2-2、表 2-5 及表 2-6。ARM 经 GP0 配置 DMA 和 GPIO。DMA 经 HP0 访问 DDR，并以 64 位 AXI4-Stream 与核心交换数据。模型装载使用地址—数据记录及提交确认，推理使用操作码、射线数量与成对输入记录。网络计算模块不是具有独立寄存器空间的 AXI-Lite IP。

接口正确性还依赖状态顺序：发送前刷新输入缓存，先启动接收再发送，完成后使接收缓存失效。只有提交令牌、模型有效位和场景资源一致才开始查询。重复请求由唯一硬件线程串行处理，防止多个 HTTP 线程同时更改同一块 PL。相关实际函数与变量分别见图 2-23、图 2-26 和图 2-29。

HDMI 显示使用 BaseOverlay。下一次推理前，服务恢复推理 Overlay 以及对应的模型和缓存状态，具体流程见图 2-34。

2.5 模型和硬件优化分析

2.5.1 优化思路与比较方法

优化首先利用数据依赖确定可复用部分，再通过结构约束和微调恢复预测能力。冻结定点规则后删除整组全零列，最后引入双射线计算、分批 DMA 与双缓冲调度。图 1-3 展示研发顺序，图 2-5 和图 2-28 分别展示计算结构与在线调度。模型压缩与 FPGA 计算组织的相关方法见文献[6]和[7]。

下文依据归档评估结果、稀疏模型清单和布局布线报告，依次比较量化精度、权重存储和器件资源。完整测试集的 PC 与 FPGA 逐图时间见表 3-7。

2.5.2 量化感知训练的精度影响

PTQ 和 QAT 均采用已结构适配的模型及训练集定标规则。QAT 通过训练射线损失更新参数，其前向模拟目标整数运算。表 2-45 对同一场景完整测试划分计算相对 GT 的平均结构相似度。PTQ 为量化后未进行该次 QAT 的基线，QAT 为最终选定整数导出。

表 2-45 完整测试集上 PTQ 与最终 QAT 的精度对比

场景	图数	PTQ 平均 SSIM	QAT 平均 SSIM	提升量
s23	1225	0.738094	0.741537	0.003443
Meetingroom	200	0.671796	0.682629	0.010832
Office1	200	0.629839	0.652706	0.022867
Office2	100	0.603049	0.605330	0.002281

四个场景经 QAT 后的平均结构相似度均高于对应的 PTQ 结果，提升量为 0.002281～0.022867，其中 Office1 的提升量最大。这表明在结构适配模型上模拟硬件整数运算并进行参数更新，有助于改善量化后的预测质量。最终 FPGA 结果与原始 PC 浮点模型的比较见表 3-5。训练参数更新与定标使用训练划分，测试划分用于独立评价。

2.5.3 稀疏存储的权重压缩效果

压缩以冻结的同一整数网络为对象：稠密表示含 687872 个有效 INT8 权重，权重有效载荷为 671.750 KiB。稀疏表示将 128 个输出分为一组，只删除组内整列均为零的输入。保留列含 128 个 INT8 值，即一个 1024 位权重字。AXI 传输字是 64 位，一个权重字需拆成 16 个传输段，这两个“字”不能混用。

表 2-46 冻结整数网络的权重压缩前后对比

场景	稠密 INT8/KiB	稀疏权重/KiB	保留权重字	载荷减少
s23	671.750	181.000	1448	73.06%
Meetingroom	671.750	231.375	1851	65.56%
Office1	671.750	228.000	1824	66.06%
Office2	671.750	306.125	2449	54.43%

表 2-46 比较稠密与稀疏表示的权重有效载荷，索引、偏置和描述符等另行存储。四场景稀疏权重载荷较稠密表示减少 54.43%～73.06%，均可装入 3072 个 1024 位字的硬件权重区。清单中的稠密重建与稀疏计算核对均达到逐位一致，说明全零列删除保持了冻结整数模型的计算结果。

2.5.4 运行时可配置设计的资源开销

早期固定场景实现与后续运行时可配置实现都针对 XC7Z020CLG400-1，均有已归档布局布线报告。后者增加模型写入、提交确认、描述符与可配置存储，以一份推理位流支持四场景参数切换。表 2-47 给出同类报告的资源及最差时序裕量，展示获得可配置能力所付出的实际资源代价。

表 2-47 固定场景版与运行时可配置版的实现资源和时序

指标	固定场景版	运行时可配置版	变化或说明
LUT	46832	50069	+3237
寄存器	45634	48464	+2830
BRAM/36 Kb 等效	89.5	127.5	+38.0
DSP48E1	178	178	保持不变
名义目标频率	95 MHz	95 MHz	最终实际约 94.118 MHz
WNS	+0.100 ns	+0.015 ns	均通过各自时序检查
WHS	+0.014 ns	+0.019 ns	均通过各自时序检查
场景参数更新	按场景固化生成物	同一推理位流装载 NPZ	新增可配置能力

由表 2-47 可知，与固定场景版本相比，运行时可配置版本增加 3237 个 LUT、2830 个寄存器和 38 个 36 Kb 等效 BRAM，DSP 数量均为 178 个。新增资源用于支持模型写入、提交确认和可配置存储，使四个场景能够共享同一推理硬件。两版设计时钟均为 95 MHz，最终实现报告中的实际时钟约为 94.118 MHz。

2.5.5 计算复用与流水调度

固定 Rx 分支缓存后，每幅仍进行 32400×64=2073600 个采样位置的射线渲染，但信号网络的输入在一条方向射线的 64 个采样点上相同，只需按 32400 个方向重新计算信号参数。相对于对每个采样点重复计算信号分支，信号分支调用次数由 2073600 次变为 32400 次。

双 INT8 打包用一个 DSP48E1 乘法产生两个目标 INT8 乘积，双射线引擎共享模型并分别累加。这些结构的端口与数据宽度见表 2-8。18 个五行基础包在软件中组成 20、20、20、20、10 行的 5 批传输，两个发送缓冲将下一批准备与当前 DMA/PL 运行重叠。18 个包、5 次传输和 2 条网络计算通路分别是文件组织、事务调度和硬件并行的层次。

上述结构和调度设计的综合效果通过完整空间谱的时延、输出稳定性和能耗实验评价，结果分别见表 3-6、表 3-7、表 3-11 和表 3-8。板端驱动计时从模型和缓存准备完成后开始，应用响应还涉及参考计算、通信及可选复核。

第三部分 实验结果与分析

3.1 实验系统与验证内容

本作品已形成由电脑、PYNQ-Z2 和外接显示屏组成的桌面式无线空间谱推理系统。训练与模型准备在 PC 侧完成，运行时由浏览器组织空间位置查询，PYNQ-Z2 执行完整方向网格的整数推理与射线渲染，结果返回电脑，并可通过板卡 HDMI OUT 输出至显示屏。系统已在 s23、Meetingroom、Office1 和 Office2 四个场景上运行，四场景采用同一推理位流，分别装载配套运行时模型与固定接收端数据。 系统实物连接已在图 2-3 给出，以下结合不同取景和运行结果说明完成情况。

实物中，笔记本电脑显示 App，开发板通过网线与电脑的 USB 转网口转换器相连，通过 HDMI 线与外接显示屏相连。日常展示使用套件电源适配器为板卡供电。照片中的 GT、PC 与 FPGA 分别表示数据集参考谱、电脑浮点预测和板端

整数预测。照片用于展示实际设备与运行状态，性能统计采用后文列明的独立测试记录。

3.1.1 系统实物与连接

系统全景由操作端、计算板卡、外接显示和连接线缆共同组成。正面照片用于观察界面与显示效果，约 45°斜视照片用于观察板卡位置、连接方向及各设备的空间关系。 图 3-1 给出整个系统正面全局照片。图 3-2 给出整个系统约 45°斜视全局照片。

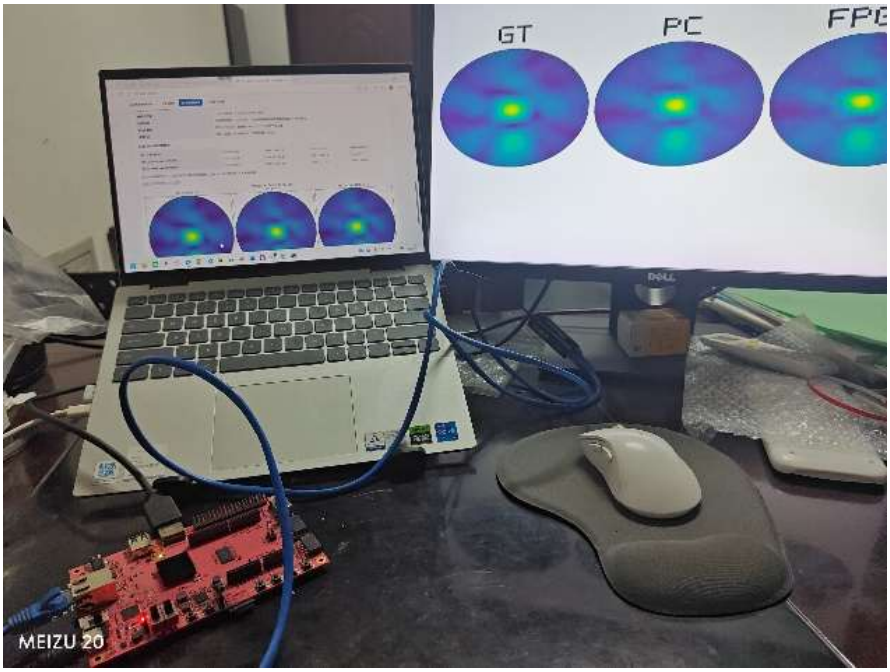


图 3-1 整个系统正面全局照片

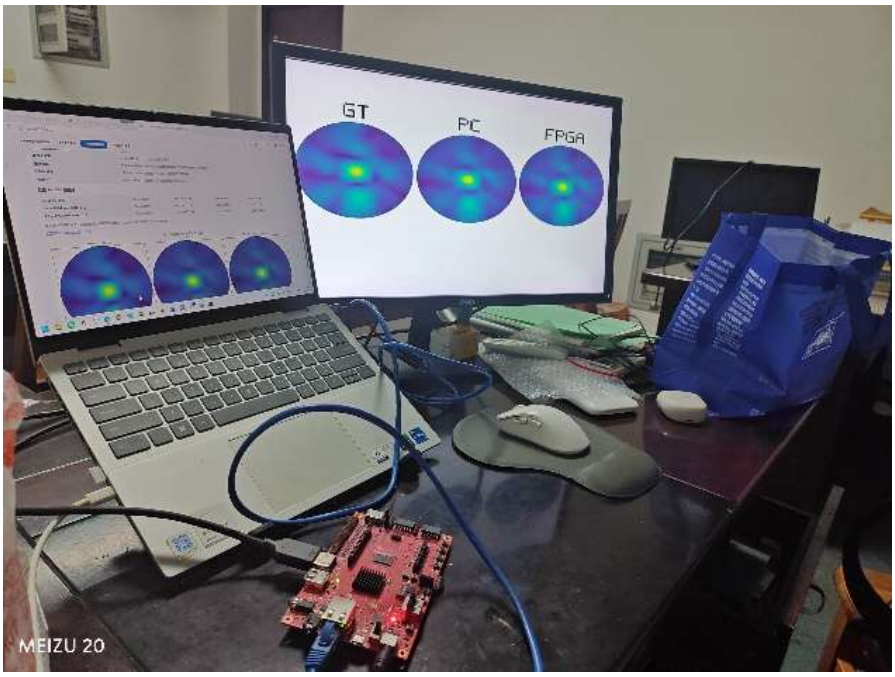


图 3-2 整个系统约 45°斜视全局照片

3.2 系统实现与功能验证

3.2.1 样机组成

样机由 PYNQ-Z2 开发板、电脑和显示屏组成，采用桌面布置。设备之间通过板载连接器及配套线缆完成供电、数据通信和显示连接。

板卡、电脑和显示屏可分别移动或替换，不需要拆装机械机构。板卡周围保留散热、MicroSD 与跳线操作空间，供电、网线和 HDMI 线沿板边引出，避免插头受拉。 样机部件及接口分工见表 2-3，板卡与线缆位置见图 2-3。 图 3-3 给出真实运行录像中的板卡 网络及显示设备连接。

3.2.2 硬件实现

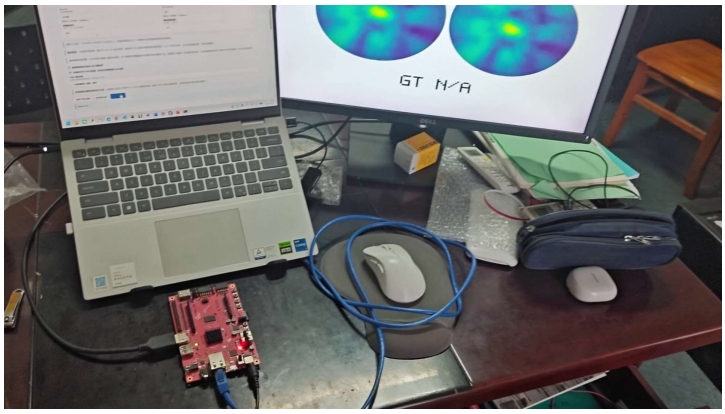


图 3-3 真实运行录像中的板卡 网络及显示设备连接

硬件载体采用 PYNQ-Z2 现成电路板，未另行绘制或加工自制 PCB。已完成的电路与逻辑成果包括可装载的推理 Overlay、自定义整数计算与射线渲染核心、AXI DMA 数据通路和板端接口联调。网口承担任务与数据通信，HDMI OUT 承担结果显示，MicroSD 承担系统及运行文件保存。 图 3-4 给出电源 网络和 HDMI 接口的实际连接近照。

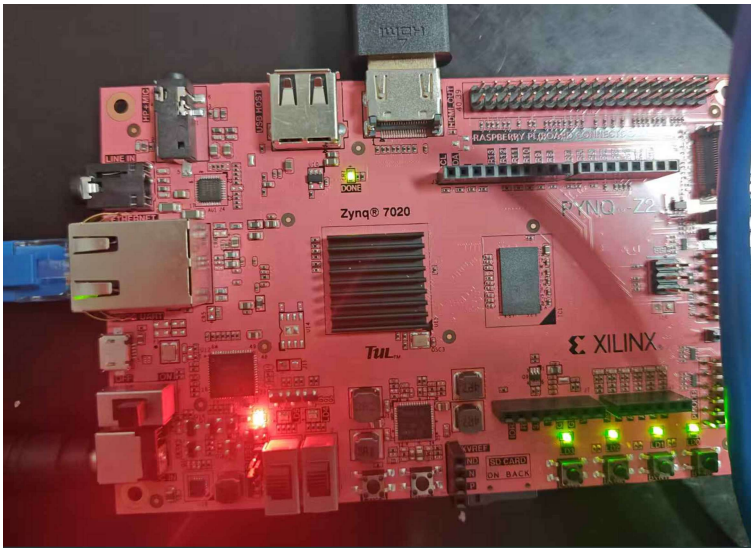


图 3-4 电源 网络和 HDMI 接口的实际连接近照

日常展示使用套件适配器供电，功率测试使用 5 V Micro-USB 输入链路。后文功率结果对应测试时的板卡输入测量位置。

3.2.2.1 硬件资源与时序结果

当前推理位流已在 Vivado 2024.1 中针对 XC7Z020CLG400-1 完成布局布线，并用于四场景板端推理。可装载文件包含 bit 位流和配套 hwh 硬件描述，场景切换更换运行时数据，不用为四个场景分别综合硬件。以下数值来自归档的实现报告。

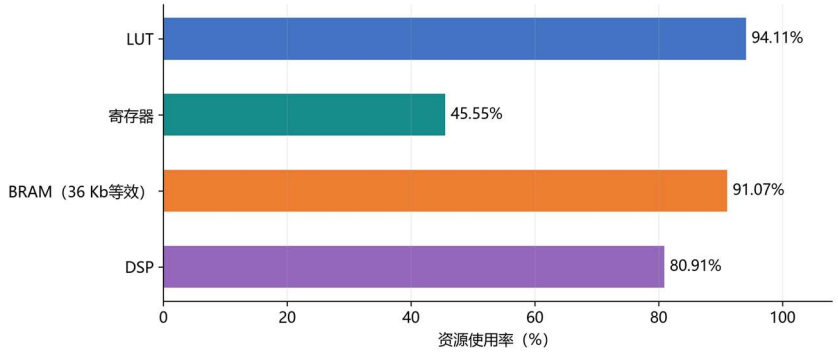


图 3-5 布局布线后主要逻辑资源使用率

图 3-5 给出推理 Overlay 布局布线后的主要逻辑资源使用率。各资源数量及实现时序见表 2-11，可据此评估运行时模型存储和并行计算结构对目标器件资源的需求。

3.2.3 软件实现与功能验证

软件已形成浏览器 App、PC 服务、板端服务、Notebook 引导与离线模型准备程序。用户在同一界面选择场景，进入测试样本查询或任意 Tx 查询。PC 保存原始浮点模型用于参考，板端装载整数运行时模型，界面显示预测结果、输入坐标及计算状态。

表 3-1 软件界面与交付程序的完成内容

成果	已实现内容
场景控制台	统一选择四场景、板端地址与连接状态
查询界面	测试位置选择、连续 Tx 输入、ARM 与 HDMI 选项
结果界面	GT/PC/FPGA 或 PC/FPGA 空间谱、指标及运行信息
统计与浏览	完整目录统计、代表案例及已保存结果轮播
板端程序	服务启动、模型切换、DMA、整数参考与显示
模型准备程序	结构适配、定标、QAT 和运行时文件导出

图 2-14 中的总体统计来自真实板端运行记录。读取统计和轮播不会自动重跑整套推理，正式查询则针对输入 Tx 现场进行计算推理，生成板端结果。

3.2.3.1 测试位置查询

测试位置查询从场景既定测试目录获取 Tx 和 GT，电脑与 FPGA 分别生成

完整预测空间谱，并与同一 GT 比较。以 s23 测试 ID 03835 为例，Tx 坐标为（−0.380，−0.379，1.386）m，左至右依次为原数据集测量 GT、PC FP32 和 FPGA 原始 Q15 换算结果。输入控件见图 2-20，输出图与具体比较结果见图 3-6 及表 3-2。

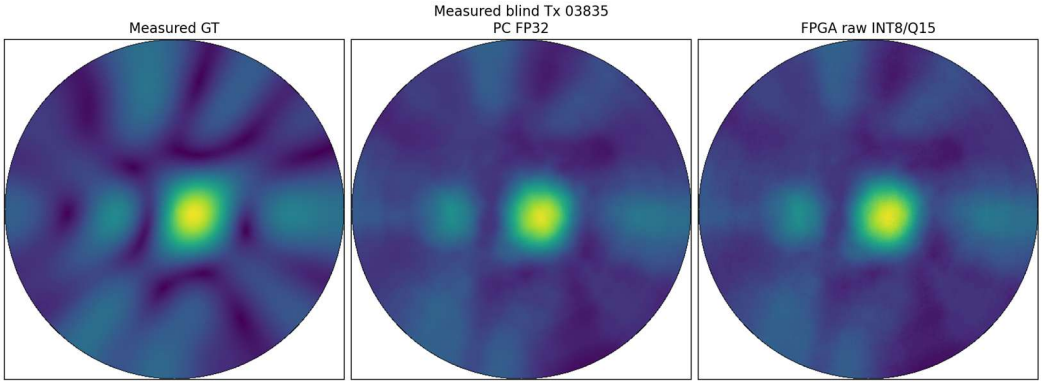


图 3-6 s23 测试位置 03835 的实际结果

表 3-2 测试位置 03835 的完整图比较

比较对象	结构相似度指数	均方根误差
PC 相对 GT	0.795577	0.061528
FPGA 相对 GT	0.780668	0.062206
PC 与 FPGA	0.979666	0.009319

该测试位置的 PC 与 FPGA 预测空间谱具有相近的角域分布，展示了同一输入下两条推理路径的输出形态。完整测试集的预测质量在后文通过结构相似度统计分析。

3.2.3.2 连续坐标查询

任意 Tx 查询接收当前场景允许范围内的任意三维坐标，不局限于测试目录中的离散位置。系统按同一方向网格分别执行浮点和整数推理。越界或非有限数输入由程序拒绝。以下结果来自 s23 坐标（0.654，0.2466，1.650）m 的一次实际运行。 图 3-7 给出 s23 连续 Tx 坐标的 PC 与 FPGA 实际导出结果。表 3-3 给出本次连续坐标查询的结果。

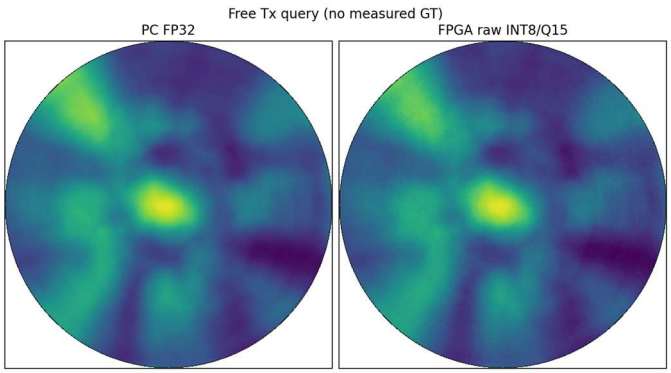


图 3-7 s23 连续 Tx 坐标的 PC 与 FPGA 实际导出结果

表 3-3 本次连续坐标查询的结果

项目	记录值	解释
输出尺寸	90×360	两端使用相同方向网格
PC 与 FPGA 结构相似度指数	0.975079	属于模型间比较
PC 与 FPGA 均方根误差	0.014314	对应同一归一化数值范围
FPGA 完整图驱动时延	0.988547 s	本次单次运行，不代替统计均值
GT 状态	不存在该坐标对应 GT	不计算相对 GT 的预测精度
原始输出	uint32 Q15 / 32768	保留完整数值再进行显示

连续坐标查询扩展了系统的输入方式，使已建模环境中的发射位置可按坐标直接指定，并生成相应的空间谱预测。

3.2.3.3 板端 HDMI 显示

系统通过 PYNQ-Z2 的 HDMI OUT 接口向独立显示屏输出结果。测试位置查询可显示 GT、PC 和 FPGA 三幅空间谱，连续坐标查询显示 PC 和 FPGA 预测，并以 GT N/A 标识该位置缺少参考标签。

图 3-8 截自原始实拍录像，保留屏幕取景与实际显示像素，画面中的 GT N/A 与连续坐标查询无 GT 的状态一致。HDMI 提供结果的独立物理展示，使电脑侧操作与板端显示形成完整演示链路。

物理 HDMI 采用 PYNQ 视频 Overlay，与推理 Overlay 分时使用 PL。显示后的下一次查询先停止视频并恢复推理 Overlay、模型与缓存。约 1 s 的完整图驱动时延对应推理阶段，不包含 HDMI 位流切换和视频初始化。

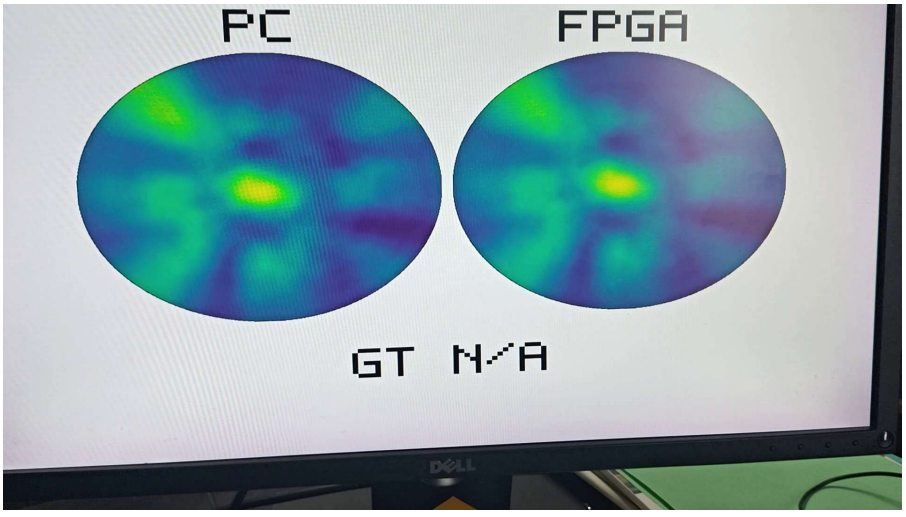


图 3-8 真实 HDMI 运行录像中的 PC 与 FPGA 显示画面

3.2.3.4 复现工具与工程材料

除运行界面外，工程已整理硬件 RTL 和 Tcl、电脑端 Python、板端 Notebook 与驱动、模型准备程序和数据契约。硬件 Tcl 负责重建工程并生成 bit 及 hwh。

模型准备程序以原始模型参数、数据与划分为输入，生成与硬件配套的运行时文件。文档化执行顺序与各模块职责已随代码包提供。 表 3-4 给出已整理的软件工程成果。

表 3-4 已整理的软件工程成果

成果	输出或作用	当前验证范围
硬件工程与 Tcl	重建 Overlay 及导出硬件文件	当前归档位流已经用于实板运行
电脑与板端源代码	任务、参考、驱动和显示	已有四场景运行记录
模型转换 Skill	运行时 NPZ 及描述性报告	非默认小规模端到端测试通过
通用 PYNQ Skill	HWH 解析、调用骨架、黄金参考与性能报告	离线解析、生成及正负 mock 用例通过
英文工程仓库	README、依赖与复现说明	代码与执行契约已组织

模型转换 Skill 的小规模验证使用 1 步结构适配、1024 行定标、1 步 QAT 和 1 幅评估图，生成 1456 个稀疏权重字，并且确保未超过 3072 字容量。

通用 PYNQ Skill 不绑定 NeRF²、固定图像尺寸或本作品场景，接口与寄存器信息通过配置和 hwh 读取。现有记录包括真实工程 bit/hwh 交叉检查、mock 正确输出与错误黄金参考反例。

3.3 性能测试与结果分析

为评价系统的预测质量和运行效率，本节对四个场景的完整测试集开展精度、时延和功率测试，并利用整数参考对硬件输出进行复核。测试对象、计时范围与功率测量位置分别随各项结果说明。

精度实验以各场景测试集的原始标签为参考，时延实验分别考察固定位置的重复运行和完整测试集的平均表现。功率实验依据板端输入链路的采样记录计算功率与能耗。

3.3.1 评价指标与测试对象

预测质量采用结构相似度指数评价，FPGA 空间谱由原始 Q15 整数除以 32768 得到。时延实验同时给出中位数和第 95 百分位数，其中第 95 百分位数用于表征该组测量中较慢一侧的时延水平。

3.3.2 完整测试集预测精度

精度实验覆盖四个场景的全部 1725 幅测试空间谱，其中 s23 的参考数据来自实测，Meetingroom、Office1 和 Office2 来自 CloudRT 仿真。PC 浮点结果和 FPGA 定点结果分别与同一参考图计算结构相似度，再按场景取算术平均。图 3-9 给出各场景的平均结构相似度。

由图 3-9 和表 3-5 可知，四个场景的 FPGA 平均结构相似度均低于对应 PC 浮点参考，差值为 0.0075～0.0130。其中，s23 场景的 PC 与 FPGA 均值分别为 0.754553 和 0.741537，Office2 分别为 0.612852 和 0.605330。结果表明，经过结构适配与定点部署后，模型在四个测试场景中保持了接近浮点参考的平均预测表现。各样本之间的差异进一步通过累计分布分析。

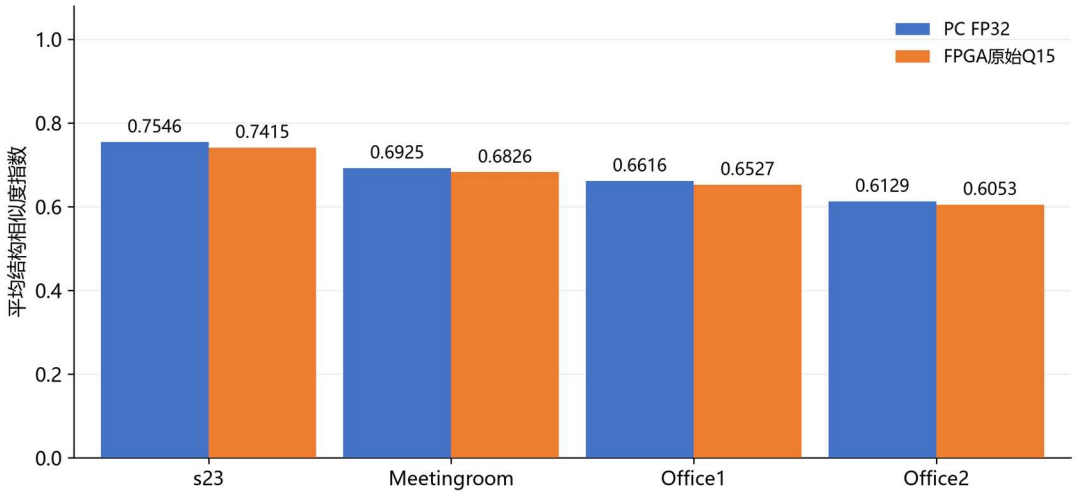


图 3-9 完整测试集平均结构相似度指数

表 3-5 四场景完整图预测精度

场景	图数	PC 均值	FPGA 均值	均值差
s23	1225	0.754553	0.741537	0.013016
Meetingroom	200	0.692512	0.682629	0.009884
Office1	200	0.661638	0.652706	0.008933
Office2	100	0.612852	0.605330	0.007523

3.3.3 精度分布与空间谱对比

为考察场景平均值之外的样本差异，图 3-10 给出各测试集结构相似度的累计分布，图 3-11 展示参考空间谱及两种预测结果的可视化对比。累计分布反映不同相似度区间内的样本比例，空间谱对比则用于观察预测结果的角域结构。

图 3-10 分别展示参考空间谱、PC 浮点预测和 FPGA 定点预测。三幅图使用相同的颜色范围，便于比较主要能量区域及其周围的角域分布。图 3-11 展示各场景完整测试集的结构相似度分布。

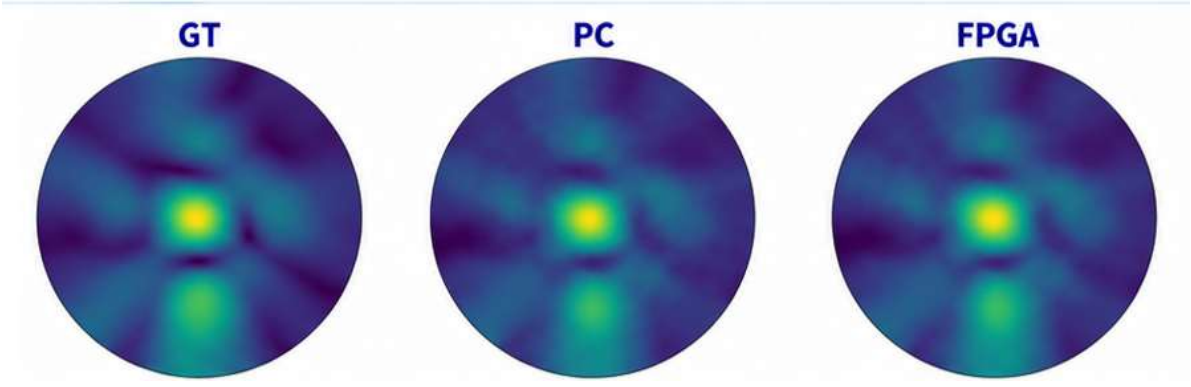


图 3-10 参考空间谱与 PC 及 FPGA 预测结果对比

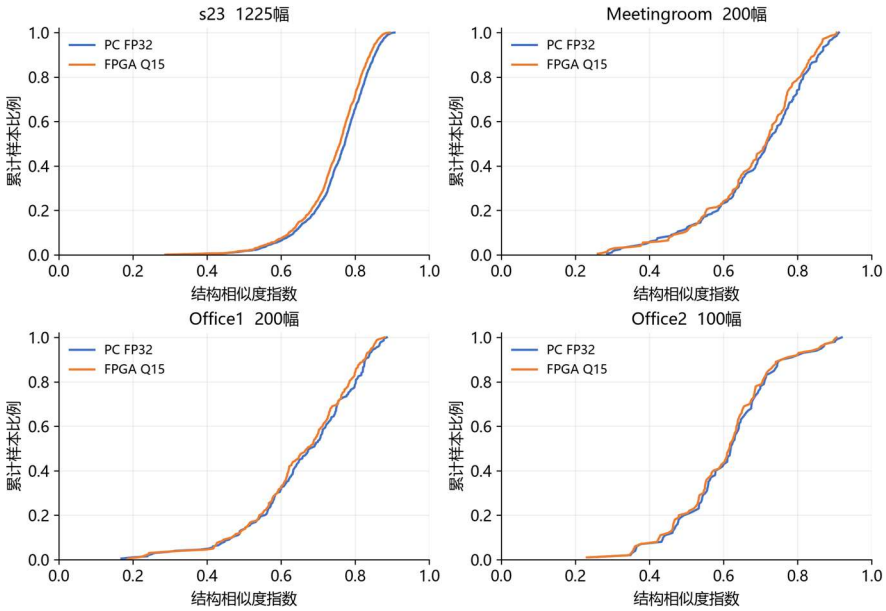


图 3-11 各场景完整测试集结构相似度的累计分布

3.3.4 固定位置连续推理时延

在每个场景选择一个固定测试样本，连续执行 50 次完整空间谱推理，共获得 200 次记录，用于分析驱动时延和输出稳定性。计时涵盖数据准备、五批流水调度及 DMA 与 PL 计算，模型与缓存准备、HTTP 通信、绘图、随机复核和 HDMI 切换不计入该时延。图 3-12 给出各场景连续推理的时延。

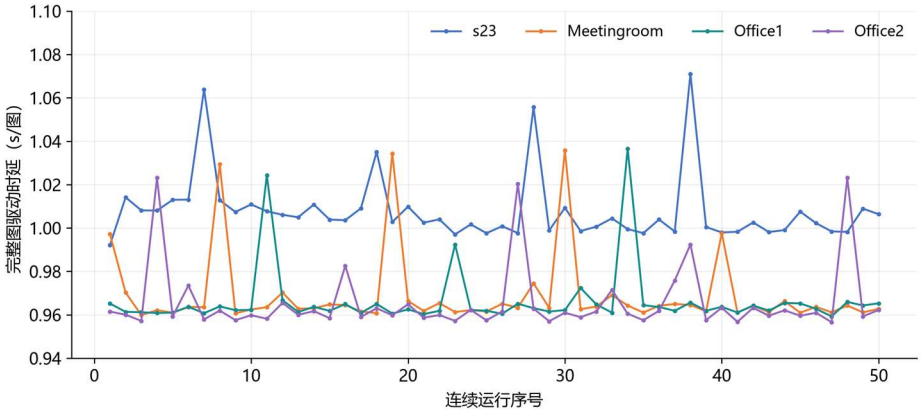


图 3-12 各场景连续 50 次完整图驱动时延

表 3-6 固定样本 50 次时延及输出稳定性

场景	中位数/s	P95/s	最小/s	最大/s
s23	1.003878	1.046368	0.992063	1.071055
Meetingroom	0.963501	1.015129	0.959842	1.035753
Office1	0.962617	0.983371	0.959241	1.036586
Office2	0.960714	1.007711	0.956580	1.023127

四个场景连续 50 次推理的中位时延为 0.961～1.004 s，第 95 百分位时延为 0.983～1.046 s。各场景的原始 Q15 输出哈希在重复运行中保持一致，并与可移植整数参考一致。本组记录未发生自动硬件重试，表明固定输入下的整数输出具有重复性。

3.3.5 完整测试集平均推理时间

为考察不同测试位置下的整体运行时间，四个场景的完整测试集分别重复执行 3 轮，PC 与 FPGA 各完成 5175 次完整空间谱推理。图 3-13 和表 3-7 给出逐图调用时间的算术平均值。

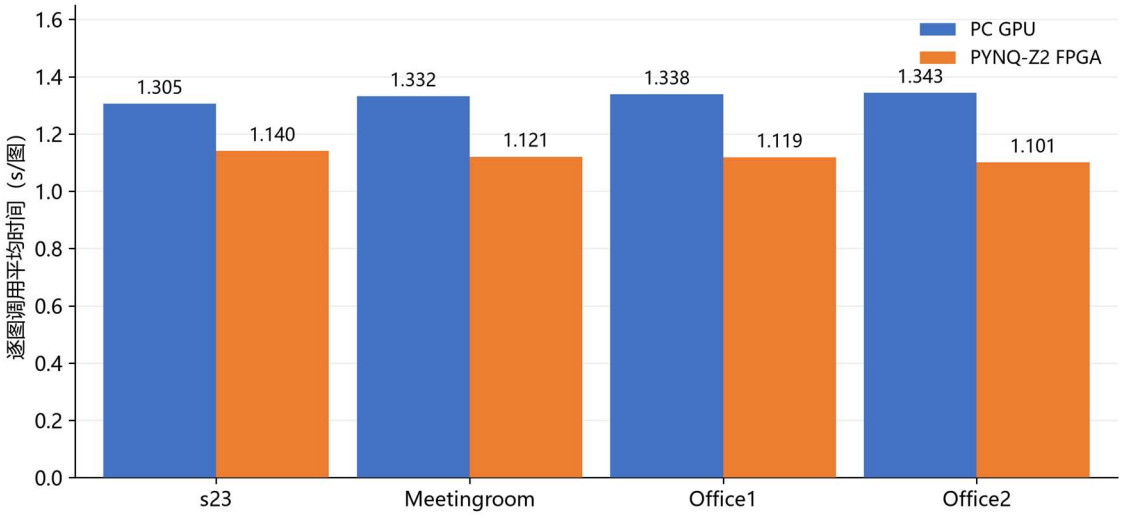


图 3-13 全测试集三轮逐图调用平均时间

表 3-7 三轮逐图平均时间的描述性比较

场景	每端推理数	PC/s	FPGA/s	时间缩短
s23	3675	1.305175	1.140460	12.62%
Meetingroom	600	1.331536	1.120950	15.82%
Office1	600	1.338199	1.118589	16.41%
Office2	300	1.343496	1.100565	18.08%

由图 3-13 和表 3-7 可知，FPGA 单幅平均推理时间为 1.101～1.140 s，较对应 PC 浮点参考缩短 12.62%～18.08%。PC 计时包含输入传输至 GPU、浮点渲染、同步和结果拷回，FPGA 计时对应板端逐图调用过程。两组结果反映所用平台及相应计时范围下的运行表现。

3.3.6 板端功率测量方法

板卡功率通过 FNB58 在 5 V Micro-USB 输入链路中采集，记录电压、电流和功率。四个场景分别完成 R1、R2 和 R3 三轮完整测试，每轮均包含空闲段与活动段。图 3-14 展示功率计及板端输入链路，图 3-15 展示一次测量中的运行进

度和仪表软件。

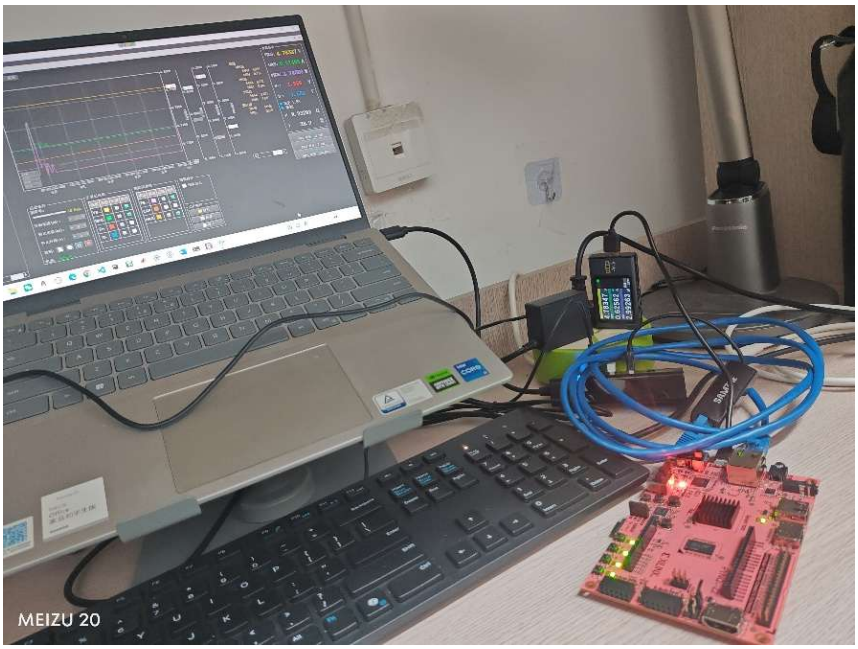


图 3-14 外接功率计及实际板输入测量链路

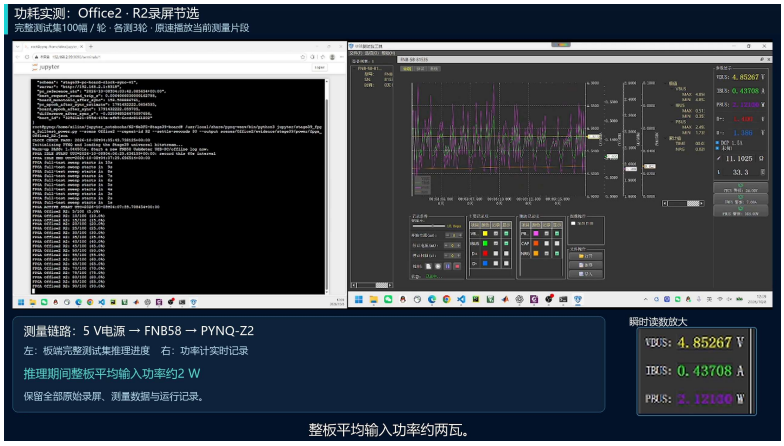


图 3-15 Office2 R2 功率测量录屏中的推理进度与 FNB58 软件

功率分析使用 CFN 中的 PBUS 记录，并通过录屏将仪表时间与板端 UTC 阶段标记对齐。图 3-16 展示 Office2 R1 的空闲与活动窗口功率、电压记录。

3.3.7 活动窗口功率与积分能耗

图 3-16 中的空闲窗口和活动窗口由仪表记录与运行阶段标记对应得到。对活动窗口内的输入功率按时间积分，再除以该轮完成的空间谱数量，可得到单幅空间谱分摊的板输入能耗。该能耗包含活动期间板卡各部分的输入能量。

$$E = \frac{1}{N}\int_{t_0}^{t_1} P(t)\,dt$$

式中，N 为该轮完整图数量，t_0与 t_1为活动窗口起止时刻，P 为板输入功率，E 为每幅图分摊的输入能量。

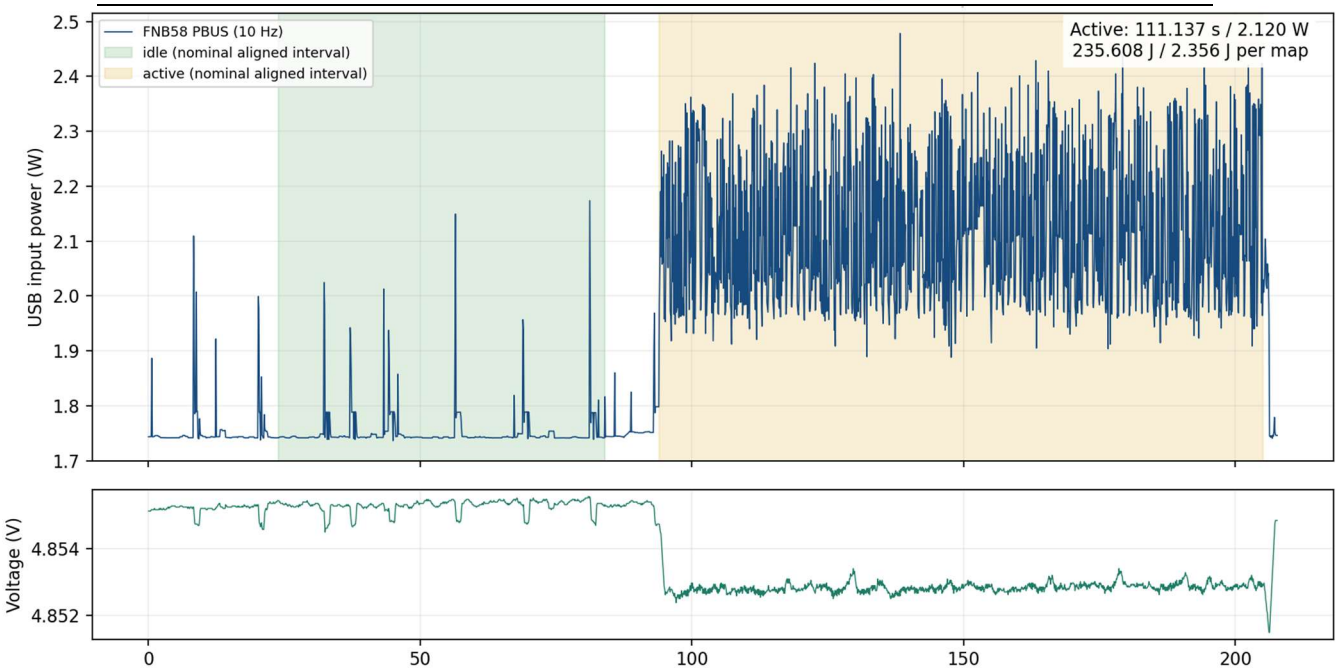


图 3-16 Office2 R1 的实际空闲与活动窗口功率电压记录

表 3-8 各场景三轮连续活动窗口统计

场景	均摊时间/s	平均功率/W	积分能耗/J 每图
s23	1.143991	2.104819	2.408904
Meetingroom	1.124870	2.098470	2.361410
Office1	1.122671	2.098009	2.354787
Office2	1.107694	2.119973	2.348907

表 3-8 分别给出三轮运行的统计量中位数。各列独立取中位数，因此同一行的时间、功率和能耗不一定对应同一轮测量。单幅积分能耗及其测量范围进一步列于表 3-9。

3.3.8 不同测量范围下的单图能耗

PC 侧通过 NVIDIA 驱动读取 RTX 4050 Laptop GPU 功率数据[18]，采样间隔设置为 50 ms。FPGA 侧通过外接仪表测量 PYNQ-Z2 整板直流输入功率，覆盖处理系统、可编程逻辑及其他板载负载。

在上述测量范围内，PC GPU 的单图积分能耗为 56.7～57.5 J，PYNQ-Z2 板输入的单图积分能耗为 2.35～2.41 J。两项测量覆盖的硬件范围不同，分别用于描述对应设备的运行能量。图 3-18 进一步给出功率采样中位数。

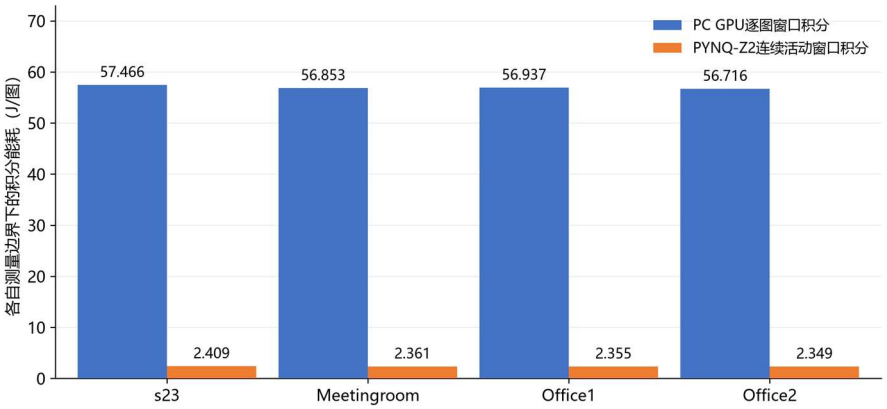


图 3-17 各自测量边界下的单图积分能耗

表 3-9 两端单图能耗及测量边界

场景	PC GPU/J 每图	PYNQ 板输入/J 每图
s23	57.466180	2.408904
Meetingroom	56.853408	2.361410
Office1	56.937200	2.354787
Office2	56.716016	2.348907

3.3.9 功率采样与能耗估算

表 3-10 依据每个场景的 10 个空闲采样点和 10 个推理采样点，取推理功率中位数并乘以对应场景的 50 次基准时延中位数，得到 1.900～2.051 J/图的代表性估算值。

由于统计窗口和计算方式不同，功率采样中位数为 1.974～2.043 W，连续活动窗口平均功率为 2.098～2.120 W。相应的单图能耗乘积估算为 1.900～2.051 J，活动窗口积分结果为 2.349～2.409 J。本文使用表 3-8 的积分结果评价连续整图任务的板输入能耗，表 3-10 给出功率与基准时延的乘积估算。

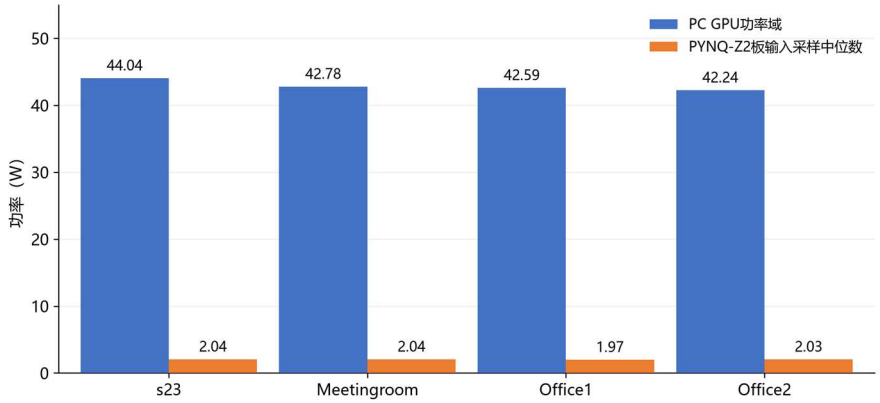


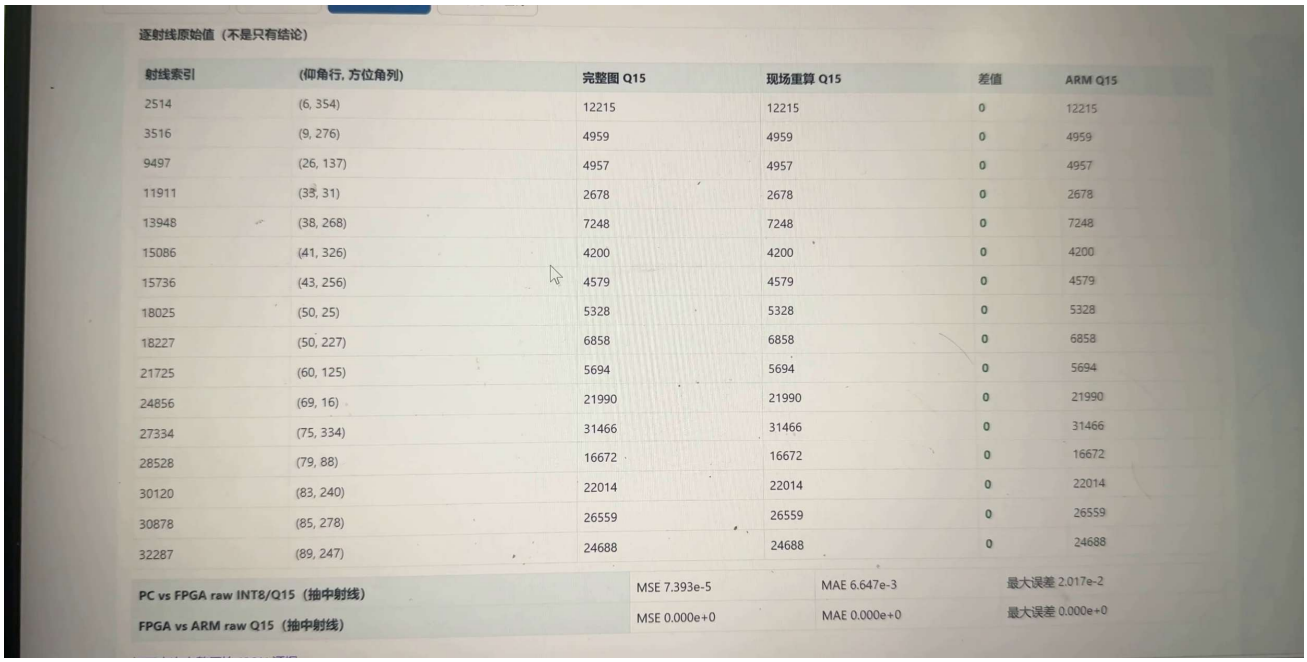
图 3-18 PC GPU 功率与板输入采样功率中位数的描述性比较

表 3-10 既有功率采样与基准时延乘积

场景	采样中位数/W	50 次中位时延/s	乘积估算/J
s23	2.042939	1.003878	2.050862
Meetingroom	2.039729	0.963501	1.965280
Office1	1.974068	0.962617	1.900271
Office2	2.027761	0.960714	1.948099

3.3.10 整数输出复核

随机射线功能已在实际查询中形成逐射线数值表。以 s23 测试 ID 03835、nonce 为 6765 的一次查询为例，公开规则选择 16 条射线，FPGA 重跑所涉及生产包，ARM 独立计算抽中位置，16 项整数差均为 0。电脑端重新计算选点和记录哈希，并成功发现临时+1 LSB 反例。 图 3-19 给出实际查询录像中的随机射线 Q15 与 ARM 复核区域。


逐射线原始值（不是只有结论）

射线索引	(仰角行, 方位角列)	完整图 Q15	现场重算 Q15	差值	ARM Q15
2514	(6, 354)	12215	12215	0	12215
3516	(9, 276)	4959	4959	0	4959
9497	(26, 137)	4957	4957	0	4957
11911	(33, 31)	2678	2678	0	2678
13948	(38, 268)	7248	7248	0	7248
15086	(41, 326)	4200	4200	0	4200
15736	(43, 256)	4579	4579	0	4579
18025	(50, 25)	5328	5328	0	5328
18227	(50, 227)	6858	6858	0	6858
21725	(60, 125)	5694	5694	0	5694
24856	(69, 16)	21990	21990	0	21990
27334	(75, 334)	31466	31466	0	31466
28528	(79, 88)	16672	16672	0	16672
30120	(83, 240)	22014	22014	0	22014
30878	(85, 278)	26559	26559	0	26559
32287	(89, 247)	24688	24688	0	24688

PC vs FPGA raw INT8/Q15（抽中射线）	MSE 7.393e-5	MAE 6.647e-3	最大误差 2.017e-2
FPGA vs ARM raw Q15（抽中射线）	MSE 0.000e+0	MAE 0.000e+0	最大误差 0.000e+0

图 3-19 实际查询录像中的随机射线 Q15 与 ARM 复核区域

表 3-11 随机复核与已有运行成果

项目	已记录结果	结论范围
本次随机选择	16 条，索引公开	只涉及抽中的位置
重算与完整图差	16 条均为 0 LSB	原始定点值一致
ARM 与 FPGA 差	16 条均为 0 LSB	同一整数模型逐位一致
PC 独立核对	选点、数值对、哈希通过	不依赖板端自报结论
+1 LSB 反例	差异被检测	验证比较器能拒绝错误

复核时，FPGA 重新执行抽中位置所在的生产包，ARM 独立计算选中射线。该例中 16 条射线的整数差均为 0，表明两条计算路径在所选样本上的输出一致。

第四部分 结论与展望

4.1 研究结论

本文面向固定接收端条件下的无线角域空间谱预测，基于 PYNQ-Z2 构建了 NeRF²软硬件协同推理系统。系统接受发射端三维坐标，生成 90×360 方向网格的完整空间谱，并支持四个场景的运行时切换。

在模型层面，依据固定接收端条件施加结构约束，分离衰减与信号计算，并复用衰减中间量。结合训练集定标和硬件精确量化感知训练，使模型适应目标整数运算。冻结整数参数后，通过删除分组全零列压缩权重。

在硬件层面，系统采用双 INT8 乘法打包、双射线并行和稀疏权重组织实现网络计算，由可编程逻辑完成射线渲染与幅值计算。ARM 负责场景配置、数据打包及 DMA 调度，双发送缓冲用于重叠数据准备与硬件执行。

四场景共 1725 幅测试空间谱的实验结果表明，FPGA 结果相对真值的平均结构相似度较 PC 浮点参考下降 0.0075～0.0130。模型及缓存准备完成后，完整测试集三轮推理的单幅平均时间为 1.101～1.140 s，较对应 PC 参考缩短 12.62%～18.08%。

各场景固定位置连续推理 50 次的中位时延为 0.961～1.004 s，整板输入功率中位数为 1.974～2.043 W。重复运行和抽样射线复核结果表明，系统能够稳定生成与整数参考一致的输出。

上述结果验证了固定接收端条件下模型计算复用和软硬件协同部署的可行性，为已建模场景的角域空间谱查询提供了实现基础。

4.2 后续研究方向

后续可将现有模型转换流程扩展到更多室内场景，研究结构适配和量化参数对不同传播条件下预测精度的影响。接收端配置变化时，还需重新生成与模型和几何条件匹配的缓存。

在硬件方面，可探索更高的射线并行度以及缓存和 DMA 调度优化。在应用方面，可将多个发射位置的预测结果接入信道知识地图，并进一步研究多接收端支持和在线数据更新。

此外，可在不同 PYNQ 硬件设计中完善接口适配与实板验证流程，提高模型部署和系统复现的效率。

参考文献

[1] X. Zhao, Z. An, Q. Pan, and L. Yang, “NeRF² : Neural radio-frequency radiance fields,” in Proc. 29th Annu. Int. Conf. Mobile Comput. Netw. (ACM MobiCom), 2023, pp. 393–407, doi: 10.1145/3570361.3592527.

[2] Y. Zeng et al., “A tutorial on environment-aware communications via channel knowledge map for 6G,” IEEE Commun. Surveys Tuts., vol. 26, no. 3, pp. 1478–1519, 3rd Quart.,

2024, doi: 10.1109/COMST.2024.3364508.

[3] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, “NeRF: Representing scenes as neural radiance fields for view synthesis,” in Proc. Eur. Conf. Comput. Vis. (ECCV), 2020, pp. 405–421, doi: 10.1007/978-3-030-58452-8_24.

[4] Y. Zeng and X. Xu, “Toward environment-aware 6G communications via channel knowledge map,” IEEE Wireless Commun., vol. 28, no. 3, pp. 84–91, Jun. 2021, doi: 10.1109/MWC.001.2000327.

[5] 胡泽杨，成国梁，游昌盛. 基于信道知识地图的智能通信与感知技术[J]. 无线电工程，2025，55(4)：757-766. DOI：10.3969/j.issn.1003-3106.2025.04.008.

[6] 高晗，田育龙，许封元，仲盛. 深度学习模型压缩与加速综述[J]. 软件学报，2021，32(1)：68-92. DOI：10.13328/j.cnki.jos.006096.

[7] 任仕伟，刘朝钧，李剑锋，等. 面向端到端目标检测神经网络的高效硬件加速系统设计[J]. 北京理工大学学报，2022，42(12)：1312-1320. DOI：10.15918/j.tbit1001-0645.2022.004.

[8] TUL Corp., PYNQ-Z2 Reference Manual, ver. 1.1, Oct. 28, 2019. Accessed: Oct. 31, 2026. [Online]. Available: https://dpoauwgwqsy2x.cloudfront.net/Download/PYNQ_Z2_User_Manual_v1.1.pdf

[9] TUL Corp., “PYNQ-Z2 schematic,” Jan. 9, 2018. Accessed: Oct. 31, 2026. [Online]. Available: https://dpoauwgwqsy2x.cloudfront.net/Download/TUL_PYNQ_Schematic_R12.pdf

[10] Advanced Micro Devices, Zynq 7000 SoC Technical Reference Manual, UG585, ver. 1.15, Feb. 6, 2026. Accessed: Oct. 31, 2026. [Online]. Available: https://docs.amd.com/r/en-US/ug585-zynq-7000-SoC-TRM

[11] Advanced Micro Devices, AXI DMA LogiCORE IP Product Guide, PG021, ver. 7.1, Jun. 24, 2025. Accessed: Oct. 31, 2026. [Online]. Available: https://docs.amd.com/r/en-US/pg021_axi_dma

[12] Xilinx, 7 Series DSP48E1 Slice User Guide, UG479, ver. 1.10, Mar. 27, 2018. Accessed: Oct. 31, 2026. [Online]. Available: https://docs.amd.com/api/khub/documents/gu4oRPFEh_Pm2uaAlfY6Kg/content

[13] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in Advances in Neural Information Processing Systems, vol. 32, 2019, pp. 8024–8035. [Online]. Available: https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html

[14] PYNQ, “PYNQ libraries,” ver. 3.0.0. Accessed: Oct. 31, 2026. [Online]. Available: https://pynq.readthedocs.io/en/v3.0.0/pynq_libraries.html

[15] X. Zhao, “NeRF2: Neural radio-frequency radiance fields,” GitHub repository. Accessed: Oct. 31, 2026. [Online]. Available: https://github.com/XPengZhao/NeRF2

[16] B. Jacob et al., “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), 2018, pp. 2704–2713, doi: 10.1109/CVPR.2018.00286.

[17] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: From error visibility to structural similarity,” IEEE Trans. Image Process., vol. 13, no. 4, pp. 600–612, Apr. 2004, doi: 10.1109/TIP.2003.819861.

[18] NVIDIA, “NVIDIA System Management Interface (nvidia-smi).” Accessed: Oct. 31, 2026. [Online]. Available: https://docs.nvidia.com/deploy/nvidia-smi/index.html

[19] Advanced Micro Devices, Vivado Design Suite User Guide: Using Tcl Scripting, UG894, ver. 2024.1, May 30, 2024. Accessed: Oct. 31, 2026. [Online]. Available: https://docs.amd.com/r/2024.1-English/ug894-vivado-tcl-scripting

附录 A 大模型协作记录与技能包提炼

A.1 大模型协作方式

研发中使用大模型协助分析网络依赖、辅助整理并核对接口、修改脚本的错误及辅助编写文档。工作流程为：提出输入和约束→读取源码或日志→形成修改建议→执行软件检查或实板操作→根据失败结果修正→保留生成物和验证记录。模型回答作为待验证方案，最终数值依据代码、实现报告和仪表记录。

以下结合工程中的典型问题，说明大模型协作如何辅助问题定位、方案修订与结果验证。

A.2 典型协作案例

案例一为 Notebook 保存后的重复校验。首次执行六个单元后，重新运行首

单元曾出现 immutable file verification failed 错误。原因是 Jupyter 会写回输出和执行计数，从而改变 Notebook 文件的字节。修订后的校验按单元类型与源文本生成规范化摘要，排除输出和运行元数据。

相关实现位于 software/stage39/verify_stage39_board_package.py 及 Notebook 首单元导入流程。保存运行输出后可再次执行首单元，源代码改变时则触发校验。服务生命周期与恢复流程见图 2-35。

案例二为 s23 场景的精度检查与参数恢复。一次 1225 幅图评估中，PC 平均结构相似度为 0.7545534930，整数 QAT 为 0.7341017923，二者相差 0.0204517007。核对配置后，将激活定标百分位恢复为 99.5，QAT 学习率恢复为 1×10^{-5}，并重新生成定标、PTQ、QAT、评估及运行时文件。

相应修订在 run_model_pipeline.verify_qat_inputs 中加入场景、来源哈希、步数、学习率、定标和量化契约交叉检查，防止混用失败运行的旧导出。2026 年 10 月 13 日 cold_reproduction_final_gate.json 记录 17 项电脑端检查通过，包含 1725 幅覆盖、RTL 及 Vivado 检查。同时 physical_gate 仍为 PENDING_NEW_BOARD_EXECUTION，说明新生成包仍需在板端重新执行，不把旧板端记录移植为新包的实测结果。

案例三为功率测量数据的组织。NVIDIA 遥测覆盖 GPU 功率域，FNB58 记录 PYNQ 板输入功率。为对应不同测量方法，报告分别列出功率采样中位数、功率与时延乘积估算，以及连续活动窗口的积分能量，结果见表 3-8 至表 3-10。

案例四：时钟与数据宽度核对。设计采用 95 MHz 时钟，模型权重字宽为 1024 位，AXI 传输字宽为 64 位。核对硬件元数据、RTL 和模型数组后，报告统一使用上述接口与容量单位，布局布线结果见表 2-11。

将工程问题整理为可复用技能时，需要同时记录输入条件、执行步骤和验证结果。表 A-1 概括典型观察与相应的处理规则。

表 A-1 经验到可复用规则的提炼

失败或观察	提炼的规则	验证方法	适用边界
Notebook 运行后文件变化	校验源内容，分离运行产物	输出变化应通过，源变化应拒绝	针对 Notebook 源身份
失败 QAT 产物与新配置混用	导出前核对参数谱系和契约哈希	不符则在打包前停止	不能据通过推断未知场景精度
权重字与传输字混淆	明确位宽、形状和存储容量	NPZ 形状与 16 段装载协议交叉核对	3072 字是 PL 容量而非 SD 卡
HWH 含不可软件寻址的流模块	契约区分寄存器 IP 和纯流模块	真实 HWH 解析与接口生成检查	适配器仍须按本题事务实现
DMA 和 CPU 缓存易不同步	先接收后发送，flush/invalidate 分工	黄金向量逐轮比较及错误反例	示例针对简单 DMA 事务

失败或观察	提炼的规则	验证方法	适用边界
软件 mock 可能被误当实板	分离 PLANNED、PASS_MOCK、PASS_BOARD	正例通过，错误黄金参考必须失败	跨板卡须重新得到实板结果

A.3 NeRF²模型转换技能包 nerf2-checkpoint-fpga-export

nerf2-checkpoint-fpga-export 面向同结构 NeRF²模型的 FPGA 部署，接收模型、配置、数据和固定划分。该技能包依次执行结构适配、训练集定标、QAT、完整测试集评估和运行时打包，输出运行时 NPZ、逐图指标、部署报告及哈希清单。默认配置为 3000 步结构适配、8192 行训练定标和 1800 步 QAT。

在含 02_Model_Pipeline 等依赖的完整英文仓库根目录使用以下模板，将 NEW_SCENE 及路径替换为实际输入。先 dry-run 确认形状和路径，再删除--dry-run 执行正式流程，正式运行使用新的空输出目录。

```
python skill/nerf2-checkpoint-fpga-export/scripts/run_nerf2_checkpoint_export.py --project-root . --scene NEW_SCENE --checkpoint INPUT/model.tar --config INPUT/config.yml --dataset INPUT/dataset --output-dir work/generic_exports/NEW_SCENE_DRYRUN --dry-run

python skill/nerf2-checkpoint-fpga-export/scripts/run_nerf2_checkpoint_export.py --project-root . --scene NEW_SCENE --checkpoint INPUT/model.tar --config INPUT/config.yml --dataset INPUT/dataset --output-dir work/generic_exports/NEW_SCENE
```

该包以原始 s23 模型和未预置的场景标识完成小规模验证，执行 1 步结构适配、1024 行定标、1 步 QAT 及 1 幅图评估，九阶段通过并生成 1456 个 1024 位权重字，容量上限为 3072 字。技能包适用于 D=8、W=256、skips=[4]以及本作品约定的位置编码、采样与渲染结构。

A.4 PYNQ Overlay 验证技能包 pynq-overlay-validation

pynq-overlay-validation 面向 PYNQ Overlay 的接口检查与功能验证。框架负责 BIT/HWH 身份核对、IP 与地址盘点、测试输入、黄金结果比较、重复事务和报告生成。项目专有操作放在 adapter.py 中，通过修改适配器、接口契约和测试向量，可应用于图像滤波、FFT、通信基带和寄存器控制等有限事务型设计。

```
python -m unittest discover -s skill/pynq-overlay-validation/tests -v

python skill/pynq-overlay-validation/scripts/inspect_hwh.py --hwh PATH/design.hwh --bit PATH/design.bit --output work/pynq_validation/inventory.json

python skill/pynq-overlay-validation/scripts/generate_validation_bundle.py --hwh PATH/design.hwh --bit PATH/design.bit --project-name my_overlay --output-dir work/pynq_validation/my_overlay

python skill/pynq-overlay-validation/scripts/run_board_validation.py --contract work/pynq_validation/my_overlay/overlay_validation.json --output work/pynq_validation/my_overlay/reports/dry --dry-run
```

补齐适配器和向量后，在目标 PYNQ 环境中执行 run_board_validation.py，并使用生成目录内的 overlay_validation.json。只有真实加载 Overlay、完成事务并通过黄金比较才产生 PASS_BOARD。mock 固定标为 PASS_MOCK。现有记录包

括真实工程 7 个 HWH 模块、DMA 地址 0x40400000、GPIO 地址 0x41200000 的盘点，以及 5 轮最大绝对误差 0 的 mock 和错误黄金参考负例。

不受 PYNQ 支持的目标、缺少配套 HWH、散布聚集或多通道 DMA、未知长度连续外设流、缺少可靠黄金参考等情形不能直接套用示例。时序、资源、功率和 HDMI 等外部行为仍需各自报告或现场测量。适用范围、失效条件和已验证效果分别存入 README.md、VERIFICATION.md 及 verification/verified_effects.json，提交时保留 skill/完整层级。

附录 B 实验复现方法

B.1 复现输入与工程文件

复现起点仅为原始 NeRF²模型参数、原始数据、固定训练测试划分及 config，不使用已有 bit、hwh、量化模型、固定 Rx 缓存或历史板端结果。采用 Stage39_Cold_Reproduction_Kit_NoPS1_20261012 代码包。根目录 README_从零复现 Stage39.md 为配套说明。代码包内保留全部需要的 Python、RTL 和 Tcl。

四场景输入相对于 NeRF2-main 根目录的模型路径及划分见表 B-1，每个场景还须具备 data/RFID/<scene>中的 spectrum、tx_pos.csv、gateway_info.yml、train_index.txt 与 test_index.txt。输入哈希由 00_Config/reproduction_config.json 锁定。不能把后来修改的其他 Python 文件当作本次复现的参考实现。

表 B-1 复现输入模型与固定划分

场景	logs/RFID 下的模型路径	训练/测试数
s23	s23/spectrum-exp1/ckpts/010000.tar	4898 / 1225
Meeting room	Meetingroom/spectrum-meetingroom/ckpts/005000.tar	800 / 200
Office1	Office1/spectrum-Office1/ckpts/005000.tar	800 / 200
Office2	Office2/spectrum-Office2/ckpts/005000.tar	400 / 100

B.2 运行环境准备

在 Windows 的 Anaconda Prompt 进入代码包根目录。默认使用 Python 3.8、PyTorch 2.0.1 与 CUDA 11.8。FPGA 工具使用 Vivado 2024.1。环境文件和脚本均在包内，不需要自行拼接低层模型转换命令。以下命令一行一条。已有 nerf2 环境时用 conda env update 代替 create。

```
conda env create -f 00_Config/environment.yml

conda activate nerf2

python scripts/verify_kit.py

python -c "exec('import torch\nprint(torch.__version__)\nprint(torch.cuda.is_available())\nprint(torch.cuda.get_device_name(0))')"
```

检查 00_Config/reproduction_config.json 中的 input_root 和 vivado_executable 分别指向实际 NeRF2-main 与 Vivado 2024.1 的 vivado.bat。其余模型哈希、训练划分、正式参数和容量上限保持不变。校验输出应为 STAGE39 COLD REPRODUCTION KIT VERIFY PASS，CUDA 应可用。依赖和 GPU 环境未通过时不进入长时间运算。

B.3 输入校验与硬件重建

python 03_Package_Builder/reproduce_stage39.py preflight

python 03_Package_Builder/reproduce_stage39.py vivado

预检检查四场景数据数量、划分、图像尺寸和模型哈希，输出 work/reports/00_preflight.json。第二条命令调用 01_Vivado/tcl/build_stage39.tcl 新建工程并执行综合与布局布线[19]。成功后 01_Vivado/output 中生成 n2_stage38_raw.bit、同名 hwh 及资源时序报告。也可在 Vivado Tcl Console 进入 01_Vivado 目录后执行 source tcl/build_stage39.tcl。

核对实现报告中的时序通过标志及 WNS、WHS，并检查配套 hwh 与位流。设计时钟为 95 MHz。其他 Vivado 版本可能需要升级 IP 或调整实现流程。

B.4 模型转换与完整测试集评估

python 03_Package_Builder/reproduce_stage39.py models --profile full --scene all

full 模式执行结构适配、定标/PTQ/QAT、完整评估和运行时打包。该四场景复现配置中 s23 结构适配 0 步，其余三个场景各 3000 步。每场景 8192 行训练定标，QAT 为 1800 步、batch=192、学习率 1×10^{-5}，激活定标百分位均为 99.5。此配置用于复现现有作品。附录 A 的新场景 Skill 默认所有场景 3000 步且不设精度门限，二者目的不同。

生成物分别位于 work/full/factorized/<scene>、qat/<scene>、maps/<scene>及 runtime/<scene>。四场景累计评估 1225+200+200+100=1725 幅，不能用 smoke 或部分图替代。该四场景复现的软件检查要求 PC 与 QAT 整数平均结构相似度差不超过 0.015、QAT 不低于 PTQ，且模型不超过 3072 个 1024 位权重字。

若旧错误配置在 s23 得到 0.0204517007 差值，应先确认代码包为参数修订版，保留已完成的 factorized 后按以下顺序重新生成 QAT 和后续产物，不修改门限或手工把状态改为通过。

python 03_Package_Builder/run_model_pipeline.py qat --profile full --scene all

python 03_Package_Builder/run_model_pipeline.py evaluate --profile full --scene all

python 03_Package_Builder/run_model_pipeline.py runtime --profile full --scene all

B.5 RTL 验证与部署文件生成

python 03_Package_Builder/reproduce_stage39.py rtl --profile full

python 03_Package_Builder/reproduce_stage39.py assemble

python 03_Package_Builder/reproduce_stage39.py verify

RTL 检查应出 STAGE39_RUNTIME_LOAD_CACHE_AND_INFERENCE_PASS

和 STAGE39_FOUR_SCENE_SWITCH_AND_RETURN_PASS。打包后 outputs 生成 PC-App 与 board 目录及各自 ZIP。最终 work/full/reports/cold_reproduction_final_gate.json 的电脑端 status 应为 PASS。physical_gate 仍为 PENDING_NEW_BOARD_EXECUTION，要求下一步使用本次新生成包完成实物运行。

B.6 板端部署与服务启动

按套件方式准备 PYNQ-Z2 的 PYNQ 系统 MicroSD，设置 SD 启动并使用套件电源适配器供电。电脑通过 USB 转网口适配器和网线连接板卡，外接显示屏连接 HDMI OUT。在电脑浏览器打开板卡 Jupyter 地址，上传本次 outputs 中的 N2-NeRF2-Stage39-board.zip，解压到/home/xilinx/jupyter_notebooks/N2-NeRF2-Stage39-board，避免双层同名目录。若已有旧目录，先另行备份结果再改名或移走，防止新旧文件混合。

打开 Stage39_QAT_Four_Scenes.ipynb，依次执行第 1 至第 4 单元：包与导入检查、启动 8311 服务、四场景各一幅完整图逐位自检、四场景各 50 次时延与稳定性测试。具体入口见表 2-37。第 1 单元允许在正常保存后再次执行。第 6 单元用于查看服务状态。

B.7 应用启动与完整测试

```
python outputs/N2-NeRF2-Stage39-PC-App/pc_app/stage39_pc_server.py --port 8310
```

上述命令在已激活 nerf2 环境、代码包根目录执行，浏览器打开 http://127.0.0.1:8310。在 App 填写本机实际板卡地址和端口 8311。报告截图中的 192.168.2.99 仅为本次连接示例。依次选择 s23、Meetingroom、Office1、Office2 并运行总体精度，完成数量分别为 1225、200、200、100。每幅结果应由本次板端推理生成，保存原始 Q15 与逐图记录。

完成后分别执行测试位置查询、场景范围内任意 Tx 查询和 16 条随机射线 ARM 复核，检查原始整数差。修改 nonce 可改变复核位置。启用 HDMI 后观察板卡独立显示屏，再运行一次查询确认推理 Overlay 恢复。自由 Tx 没有对应 GT，只比较 PC 与 FPGA，不进行准确率的比较。

B.8 功率测量与数据归档

PC 端测量在代码包 06_Power_Measurement 目录执行 python run_pc_all_scenes.py，四场景完整测试集各 3 轮，输出 results/pc_gpu_full_test_authoritative。其功率来自 NVIDIA GPU 遥测，不能作为电脑插座整机功率。

FPGA 测量按 06_Power_Measurement/README_功耗测量入口.md 及配套执行手册，将已验证 5 V USB 电源、FNB58 与 Micro-USB 供电线串接到板卡输入，并按厂家要求选择 USB 供电。这与日常套件 DC 适配器展示为两种接法。保留网络连接，在板端终端进入 board 根目录，先停止占用同一硬件的后台服务，再执行测量脚本。不要并发运行 App 推理。

```
/usr/local/share/pynq-venv/bin/python3 -c "exec('import
sys\nsys.path.insert(0,\x27jupyter\x27)\nfrom stage39_service_launcher import
```

```
stop\nprint(stop(port=8311))')"

/usr/local/share/pynq-venv/bin/python3 jupyter/stage39_fpga_fulltest_power.py --scene Office2 --repeat-id R1 --settle-seconds 30 --output scenes/Office2/evidence/stage39/power/fpga_Office2_R1.json
```

该测量命令为一轮模板。依次将 scene 替换为四个场景，repeat-id 替换为 R1、R2、R3，并同步修改输出路径，共 12 轮。每轮在 FNB58 电脑软件新建独立记录，记录空闲段至脚本完成后的尾段，保存原始 CFN。录屏同时保留终端阶段与仪表时间。程序 JSON 记录任务与阶段时间，仪表记录电压电流功率，二者以录屏和阶段标记关联，不能只保存一张瞬时读数截图。

测量后将本次 12 份 JSON、CFN 及录屏成对保存。使用 06_Power_Measurement 中的 convert_usbmeter_cfn.py、audit_board_power_capture.py 与 summarize_board_power.py 按配套执行手册转换、核对窗口并积分。电脑端用 summarize_pc_full_test.py 汇总。连续窗口积分除以本轮图数得到单图分摊能量，计算方法见表 3-8 附近公式。ARM 完整图外接测量属于独立扩展实验，未执行时保持未测，不影响如实复现本报告已完成的 FPGA 部分。

恢复板端服务后，在 Notebook 第 5 单元重新导出 Stage39_physical_evidence.zip，核对四场景完整测试摘要均已包含。PC 还应逐场景下载全部已保存 FPGA 图，连同模型、bit/hwh、JSON、CFN、视频和本次生成的哈希清单一起备份。关闭浏览器不会删除文件。断电后通常运行 Notebook 前两个单元恢复服务，删除板端目录则不能依靠重新上传代码自动找回结果。

B.9 复现结果判据

一次从零复现成立需要同时满足：本次输入与环境检查、Tcl 实现及配套硬件文件、1725 幅软件评估、运行时装载与切换 RTL 检查、四场景实板自检及 50 次稳定性、1725 幅板端完整图、HDMI 及交互查询，以及本次采集的功率记录。